

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Wydział Informatyki
Filia w Gdańsku

Otrombke Natalia
Nr albumu s26828
Nazwa specjalizacji: Cyberbezpieczeństwo

Kasprowicz Oliwier
Nr albumu s30290
Nazwa specjalizacji: Cyberbezpieczeństwo

Gutowski Michał
Nr albumu s30572
Nazwa specjalizacji: Cyberbezpieczeństwo

Parkitny Miłosz
Nr albumu s30287
Nazwa specjalizacji: Cyberbezpieczeństwo

Wronisz Wojciech
Nr albumu s26999
Nazwa specjalizacji: Cyberbezpieczeństwo

RacePortal — platforma wydarzeń motorsportowych

Rodzaj pracy
inżynierska

Imię i nazwisko promotora
dr hab. Marek Bednarczyk

Gdańsk, miesiąc, rok obrony

Streszczenie: Praca przedstawia projekt i implementację systemu RacePortal — platformy wspierającej katalogowanie wydarzeń motorsportowych oraz proces zapisów uczestników. System obejmuje aplikację webową, backend REST oparty o Spring Boot i MySQL oraz aplikację mobilną Expo pełniącą rolę publicznej wizytówki katalogu. W pracy opisano problem dziedzinowy, wymagania, architekturę, realizację kluczowych modułów (wydarzenia, garaż, zgłoszenia, role), mechanizmy bezpieczeństwa (JWT, RBAC) oraz weryfikację testami automatycznymi. Wynikiem jest działające, konteneryzowane środowisko demonstracyjne zgodne z zakresem MVP projektu.
Słowa kluczowe: motorsport, katalog wydarzeń, system zapisów, zarządzanie wydarzeniami, aplikacja mobilna

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Karta projektu

Temat projektu: RacePortal — platforma wydarzeń motorsportowych **Temat projektu po angielsku:** RacePortal — a platform for motorsports events	**Akronim:** RP **Data ustalenia tematu** 2025-10-01
Promotor: dr hab. Marek Bednarczyk	**Konsultanci:** 1. Paweł Maryszczuk — Senior SecOps (testy penetracyjne)

Cele projektu:
Celem projektu jest zaprojektowanie i zaimplementowanie systemu RacePortal — centralnego katalogu wydarzeń motorsportowych w Polsce, umożliwiającego przeglądanie wydarzeń (lista, mapa, kalendarz), zarządzanie garażem pojazdów, zgłoszenia udziału oraz obsługę ról organizatora i administratora, wraz z aplikacją mobilną typu wizytówka oraz konteneryzowanym środowiskiem uruchomieniowym.

Rezultaty projektu:
Aplikacja webowa (React/Vite), API REST (Spring Boot) z bazą MySQL, aplikacja mobilna Expo (katalog wydarzeń z CTA do weba), wspólne typy API (`packages/api-types`), dokumentacja projektowa oraz zestaw testów automatycznych (JUnit, Vitest, Playwright).
Katalog wydarzeń z filtrami oraz widokami lista/mapa/kalendarz; szczegóły wydarzenia i wyznaczanie trasy (OSRM); konta użytkowników (JWT, RBAC); garaż pojazdów i dopasowanie kategorii do wydarzenia (`carMatch`); system zgłoszeń ze statusami i powiadomieniami e-mail; panele organizatora i administratora; archiwum; mobilna wizytówka z przekierowaniem zapisu na web.

Miary sukcesu:
Działający stos Docker Compose (web, API, MySQL, backup) z SMTP Mailbux; realizacja funkcji MVP katalogu, zapisów i ról; pozytywne wyniki testów automatycznych API/web/mobile unit; możliwość demonstracji pełnego scenariusza: gość → kierowca → organizator → administrator oraz mobilka → CTA web.

Ograniczenia:
Brak natywnej bramki płatności (PSP); aplikacja mobilna bez pełnej równoważności funkcji z webem (świadoma wizytówka — bez logowania, garażu i uploadu); brak publikacji w sklepach App Store/Google Play; ograniczony budżet operacyjny (Oracle Always Free, Cloudflare Tunnel) oraz ramy czasowe projektu akademickiego; produkcyjny TLS poza domkniętym zakresem lokalnego MVP (w prod: Cloudflare Tunnel).

Wykonawcy	**Numer albumu**	**Specjalizacja**	**Tryb studiów**
Otrombke Natalia	s26828	Cyberbezpieczeństwo	Stacjonarny
Kasprowicz Oliwier	s30290	Cyberbezpieczeństwo	Stacjonarny
Gutowski Michał	s30572	Cyberbezpieczeństwo	Stacjonarny
Parkitny Miłosz	s30287	Cyberbezpieczeństwo	Stacjonarny
Wronisz Wojciech	s26999	Cyberbezpieczeństwo	Stacjonarny

Data ukończenia projektu:	**Recenzent:**
7 listopada 2026	mgr Alex Polin

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Otrombke Natalia**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s26828**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Kasprowicz Oliwier**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30290**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Gutowski Michał**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30572**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Parkitny Miłosz**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30287**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Wronisz Wojciech**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s26999**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

Spis treści

1 Informacje wstępne **23**

1.1 Cel i zakres pracy 23

1.2 Wizja rozwiązania 24

1.3 Struktura pracy 24

2 Słownik pojęć **27**

3 Opis problemu **33**

3.1 Charakterystyka rynku wydarzeń motorsportowych 33

3.2 Problemy związane z organizacją wydarzeń i zapisami uczestników 33

3.3 Problemy informacyjne po stronie kierowców i organizatorów 34

3.4 Uzasadnienie potrzeby stworzenia systemu RacePortal 34

3.5 Aspekty techniczne i organizacyjne problemu 35

4 Kontekst biznesowy **36**

4.1 Segmentacja użytkowników 36

4.2 Interesariusze systemu 37

4.3 Propozycja wartości systemu 39

4.4 Kanały komunikacji i dystrybucji 41

4.5 Kluczowe zasoby i działania 43

4.6 Model działania platformy 43

4.7 Realistyczność wdrożenia i dalsze perspektywy 44

4.8 Kontekst prawny i etyczny 45

5 Analiza SWOT **47**

5.1 Mocne strony (Strengths) 47

5.2 Słabe strony (Weaknesses) 48

5.3 Szanse (Opportunities) 48

5.4 Zagrożenia (Threats) 49

5.5 Podsumowanie analizy SWOT 49

6 Planowanie projektu **51**
6.1 Cele projektu . . . 51
6.2 Zakres projektu . . . 52
6.3 Metodyka pracy . . . 53
6.4 Organizacja zespołu . . . 54
6.5 Harmonogram pracy . . . 54
6.6 Infrastruktura i środowisko pracy . . . 55
6.7 Analiza ryzyka projektowego . . . 56
6.7.1 Analiza zagrożeń . . . 56

7 Analiza wymagań **58**
7.1 Źródła i sposób pozyskania wymagań . . . 58
7.2 Wymagania ogólne i dziedzinowe . . . 59
7.3 Wymagania funkcjonalne . . . 60
7.4 Wymagania niefunkcjonalne . . . 62
7.5 Wymagania bezpieczeństwa . . . 63
7.6 Wymagania interfejsu użytkownika . . . 64
7.7 Diagramy UML . . . 64
7.7.1 Diagram przypadków użycia . . . 64
7.7.2 Diagram sekwencji: rejestracja i logowanie . . . 64
7.7.3 Diagram sekwencji: zapis na wydarzenie . . . 64
7.7.4 Diagram sekwencji: zarządzanie garażem . . . 64

8 Projekt systemu **66**
8.1 Założenia architektoniczne . . . 66
8.2 Architektura logiczna systemu . . . 67
8.3 Architektura fizyczna i wdrożeniowa . . . 67
8.4 Projekt bazy danych . . . 68
8.4.1 Diagram klas . . . 68
8.5 Projekt interfejsu REST API . . . 69
8.6 Projekt warstwy webowej . . . 70
8.6.1 Prototyp UI w Figma Make . . . 72
8.7 Projekt aplikacji mobilnej . . . 73
8.8 Projekt mechanizmów bezpieczeństwa . . . 73

9 Realizacja **75**
9.1 Organizacja implementacji . . . 75
9.2 Podział systemu na komponenty . . . 75
9.2.1 Backend . . . 76

9.2.2 Frontend webowy . . . 76
9.2.3 Aplikacja mobilna . . . 77
9.3 Integracja komponentów . . . 77
9.4 Konteneryzacja i uruchamianie środowiska . . . 77

10 Szczegóły implementacji backendu **79**
10.1 Struktura projektu backendowego . . . 79
10.2 Model domenowy i encje . . . 80
10.3 Rejestracja użytkowników i logowanie . . . 80
10.4 Zarządzanie profilami użytkowników . . . 81
10.5 Obsługa wydarzeń . . . 82
10.6 Obsługa zgłoszeń . . . 83
10.7 Obsługa garażu . . . 84
10.8 Panel administratora i organizatora . . . 84
10.9 Integracja z bazą danych . . . 85
10.10 Obsługa wyjątków i walidacja . . . 86

11 Szczegóły implementacji frontendu webowego **88**
11.1 Struktura aplikacji webowej . . . 88
11.2 Strona główna i katalog wydarzeń . . . 89
11.3 Widok szczegółów wydarzenia . . . 89
11.4 Logowanie i rejestracja . . . 89
11.5 Dashboard użytkownika . . . 90
11.6 Widok garażu . . . 90
11.7 Panel organizatora . . . 90
11.8 Panel administratora . . . 91
11.9 Mapa i elementy lokalizacyjne . . . 91
11.10 Responsywność i doświadczenie użytkownika . . . 91

12 Szczegóły implementacji aplikacji mobilnej **92**
12.1 Założenia aplikacji mobilnej . . . 92
12.2 Architektura rozwiązania mobilnego . . . 93
12.3 Lista wydarzeń i widok szczegółów . . . 93
12.4 Nawigacja i interfejs użytkownika . . . 94
12.5 Integracja z API . . . 95
12.6 Przekierowanie do wersji webowej . . . 96
12.7 Ograniczenia wersji mobilnej . . . 96

13 Bezpieczeństwo systemu **97**
13.1 Założenia bezpieczeństwa . . . 97
13.2 Klasyfikacja danych . . . 98
13.3 Uwierzytelnianie użytkowników . . . 98
13.4 Autoryzacja i kontrola dostępu RBAC . . . 99
13.5 Wykorzystanie tokenów JWT . . . 99
13.6 Walidacja danych wejściowych . . . 100
13.7 Ochrona zasobów użytkowników . . . 101
13.8 Bezpieczeństwo komunikacji . . . 101
13.9 Analiza zagrożeń . . . 101
13.10 Analiza ryzyka . . . 102
13.11 Ograniczenia przyjętych zabezpieczeń . . . 103
13.12 Testy penetracyjne i remedacje . . . 103

14 Testowanie i weryfikacja systemu **105**
14.1 Strategia testowania . . . 105
14.2 Środowisko testowe . . . 105
14.3 Testy jednostkowe . . . 106
14.4 Testy integracyjne backendu . . . 106
14.5 Testy End-to-End aplikacji webowej . . . 106
14.6 Testy aplikacji mobilnej . . . 107
14.7 Testy mechanizmów bezpieczeństwa . . . 107
14.8 Automatyzacja uruchamiania testów . . . 107
14.9 Analiza wyników testów i testy wydajnościowe k6 . . . 107
14.10 Ograniczenia procesu testowania . . . 108

15 Prezentacja systemu **109**
15.1 Prezentacja głównych funkcjonalności . . . 109
15.2 Scenariusz użycia z perspektywy kierowcy . . . 111
15.3 Scenariusz użycia z perspektywy organizatora . . . 111
15.4 Scenariusz użycia z perspektywy administratora . . . 111
15.5 Podsumowanie działania systemu . . . 111

16 Nakład pracy **112**
16.1 Liczba godzin przeznaczona na realizację projektu . . . 112
16.2 Podział zadań w zespole . . . 112
16.3 Narzędzia wspierające zarządzanie pracą . . . 112
16.4 Problemy organizacyjne i ich rozwiązanie . . . 112

17 Wkład własny **113**
17.1 Natalia Otrombke 113
17.2 Oliwier Kasprowicz 113
17.3 Michał Gutowski 113
17.4 Miłosz Parkitny 114
17.5 Wojciech Wronisz 114

18 Podsumowanie **116**
18.1 Największe wyzwania i sposób ich rozwiązania 116
18.2 Zmiany i adaptacje w trakcie realizacji 117
18.3 Ograniczenia końcowej wersji systemu 119
18.3.1 Ograniczenia budżetowe 120
18.3.2 Ograniczenia czasowe 120
18.4 Możliwości dalszego rozwoju 121
18.5 Wnioski końcowe 122

Bibliografia **123**

Spis rysunków **125**

Spis tabel **126**

Załączniki **128**

Rozdział 1

Informacje wstępne

1.1 Cel i zakres pracy

Celem pracy jest zaprojektowanie oraz implementacja systemu **RacePortal** — platformy internetowej (z uzupełnieniem mobilnym) służącej jako centralny katalog wydarzeń motorsportowych w Polsce oraz narzędzie do obsługi zapisów kierowców. Projekt stanowi odpowiedź na rosnące zapotrzebowanie społeczności motorsportowej na ujednolicone narzędzie eliminujące rozproszenie informacji na grupach społecznościowych i niespójnych stronach organizatorów.

Zakres projektu obejmuje:

- **Aplikację webową** — interfejs użytkownika w architekturze SPA (React/Vite), z modułami katalogu wydarzeń, kont użytkowników, wirtualnego garażu pojazdów, dopasowania kategorii (`carMatch`), zgłoszeń na imprezy oraz paneli zarządzania dla organizatorów i administratorów.

- **API REST oraz warstwę danych** — backend wykonany w oparciu o framework Spring Boot oraz relacyjną bazę danych MySQL (z wydzielonymi strukturami encji i wspólnymi typami API w pakiecie `packages/api-types`), zapewniający bezpieczne uwierzytelnianie (JWT) oraz kontrolę dostępu opartą o role (RBAC: `USER`, `ORGANIZER`, `ADMIN`).

- **System powiadomień i map** — zautomatyzowane powiadomienia e-mail (realizowane m.in. przez serwer SMTP Mailbux) dotyczące potwierdzeń rejestracji, statusów zgłoszeń oraz weryfikacji kont, a także integrację map wydarzeń z usługą wyznaczania tras (domyślnie OSRM z możliwością rozszerzenia o Google Maps API).

- **Aplikację mobilną (Expo)** — lekką, wieloplatformową aplikację mobilną opartą na Expo, pełniącą rolę mobilnej wizytówki katalogu z przyciskiem CTA przekierowującym użytkownika do pełnego serwisu webowego.

- **Infrastrukturę i testowanie** — pełną konteneryzację całego środowiska uruchomieniowego przy użyciu Docker Compose (web, API, baza danych, backupy) z zabezpiecze-

niem produkcyjnego ruchu przez Cloudflare Tunnel oraz przygotowanie podstawowego zestawu testów automatycznych (jednostkowych i integracyjnych).

Poza zakresem MVP pozostają m.in. natywne płatności online, push notifications oraz publikacja aplikacji mobilnej w sklepach. Upload plików (obrazy JPG/PNG/WebP oraz PDF) jest dostępny po stronie **weba** i API (`/api/uploads/...`); aplikacja mobilna Expo nie udostępnia własnego formularza wgrywania — jedynie wyświetla obrazy już opublikowane w katalogu.

1.2 Wizja rozwiązania

RacePortal ma być **jednym miejscem** odkrywania i obsługi startów motorsportowych w Polsce: kierowca znajduje wydarzenie, sprawdza wymagania względem własnego garażu i składa zgłoszenie; organizator publikuje imprezę i zarządza listą uczestników; administrator moderuje treści i role.

Wizja produktowa zakłada:

- publiczny katalog z trzema widokami (lista, mapa, kalendarz) na wspólnych filtrach;
- ujednolicony proces zapisu ze statusami i powiadomieniami e-mail;
- wyraźny podział ról `USER` / `ORGANIZER` / `ADMIN` (gość przegląda katalog bez konieczności zakładania konta);
- mobilną aplikację Expo do przeglądania katalogu, z przekierowaniem zapisu na wersję webową;
- uruchamialne środowisko demonstracyjne (Docker Compose) oraz wdrożenie produkcyjne realizowane za pośrednictwem Cloudflare Tunnel.

Szczegóły warstw technicznych (web React [1]/Vite [2]/TypeScript [3], API Spring Boot [4] z Spring Security [5], MySQL [6], mobile Expo [7], `packages/api-types`) opisano w rozdziale 8 oraz w rozdziałach implementacyjnych. Kod źródłowy utrzymywany jest w repozytorium projektu [8].

1.3 Struktura pracy

Praca została podzielona na rozdziały opisujące kolejno koncepcję, analizę, projekt oraz implementację systemu RacePortal:

- **Rozdział 1 (Informacje wstępne)** — przedstawia cel i zakres pracy, wizję rozwiązania opartą na architekturze MVP, a także strukturę całego dokumentu.

- **Rozdział 2 (Słownik pojęć)** — definiuje kluczowe terminy dziedzinowe i technologiczne wykorzystane w pracy (wydarzenie, katalog, garaż, zgłoszenie, `carMatch`, dokumenty KP/DZW/SWS) oraz pojęcia bezpieczeństwa i stacku (JWT, RBAC, ASVS, testy penetracyjne, Playwright, OSRM, MVP).

- **Rozdział 3 (Opis problemu)** — charakteryzuje specyfikę rynku amatorskiego i półprofesjonalnego motorsportu w Polsce, wskazuje na problemy organizacyjne i informacyjne po stronie kierowców oraz organizatorów, a także uzasadnia potrzebę stworzenia zintegrowanej platformy.

- **Rozdział 4 (Kontekst biznesowy)** — omawia segmentację użytkowników, mapę interesariuszy, propozycję wartości systemu, kanały komunikacji, model działania platformy oraz uwarunkowania prawne i etyczne.

- **Rozdział 5 (Analiza SWOT)** — analizuje mocne i słabe strony projektu oraz identyfikuje rynkowe szanse i zagrożenia związane z wdrożeniem systemu.

- **Rozdział 6 (Planowanie projektu)** — przedstawia cele, zakres, metodykę pracy, organizację zespołu, harmonogram, zastosowaną infrastrukturę oraz szczegółową analizę ryzyka projektowego.

- **Rozdział 7 (Analiza wymagań)** — zawiera źródła pozyskania wymagań, wymagania ogólne, dziedzinowe, funkcjonalne, niefunkcjonalne, bezpieczeństwa oraz interfejsu użytkownika, a także diagramy UML (w tym przypadki użycia).

- **Rozdział 8 (Projekt systemu)** — omawia założenia architektoniczne, architekturę logiczną i fizyczną, projekt relacyjnej bazy danych (model ERD), projekt interfejsu REST API, warstwy webowej (Figma Make), aplikacji mobilnej oraz mechanizmów bezpieczeństwa.

- **Rozdział 9 (Realizacja)** — przedstawia organizację prac implementacyjnych, podział systemu na komponenty (backend Spring Boot, frontend React/Vite, aplikacja mobilna Expo), integrację oraz konteneryzację środowiska za pomocą Docker Compose.

- **Rozdział 10 (Szczegóły implementacji backendu)** — szczegółowo opisuje strukturę projektu backendowego, model domenowy, mechanizmy uwierzytelniania i rejestracji (JWT, rate limiting), obsługę wydarzeń, zgłoszeń, garażu oraz integrację z bazą danych.

- **Rozdział 11 (Szczegóły implementacji frontendu webowego)** — omawia strukturę aplikacji webowej, kluczowe widoki (katalog, szczegóły, dashboard użytkownika, garaż, panele administratora i organizatora) oraz responsywność i doświadczenie użytkownika.

- **Rozdział 12 (Szczegóły implementacji aplikacji mobilnej)** — przedstawia założenia, architekturę oraz implementację mobilnej wizytówki systemu opartej na Expo.
- **Rozdział 13 (Bezpieczeństwo systemu)** — prezentuje założenia ochrony danych, klasyfikację informacji, uwierzytelnianie, kontrolę dostępu (RBAC), walidację wejścia, analizę zagrożeń i ryzyka oraz wnioski z testów penetracyjnych z remedacjami.
- **Rozdział 14 (Testowanie i weryfikacja systemu)** — opisuje strategię testowania, środowisko testowe, testy jednostkowe, integracyjne, E2E (Playwright) oraz weryfikację mechanizmów bezpieczeństwa.
- **Rozdział 15 (Prezentacja systemu)** — opisuje prezentację głównych funkcjonalności MVP oraz scenariusze użycia z perspektywy kierowcy, organizatora i administratora.
- **Rozdział 16 (Nakład pracy)** — podsumowuje liczbę przepracowanych godzin, podział zadań w zespole oraz narzędzia wspierające zarządzanie pracą.
- **Rozdział 17 (Wkład własny)** — opisuje indywidualny wkład członków zespołu (UI/UX, analiza, backend, modelowanie, fullstack/testy/bezpieczeństwo i wdrożenie).
- **Rozdział 18 (Podsumowanie)** — przedstawia największe wyzwania, zmiany i adaptacje w trakcie realizacji, końcowe ograniczenia systemu (w tym budżetowe i czasowe), kierunki rozwoju oraz wnioski.

Rozdział 2

Słownik pojęć

Wydarzenie (event) Zorganizowana impreza motorsportowa (np. track day, drift, rally), opisana datą, lokalizacją, kategorią i wymaganiami udziału.

Katalog Publiczny wykaz wydarzeń z widokami lista, mapa i kalendarz oraz filtrami.

Garaż Zbiór pojazdów przypisanych do konta kierowcy, wykorzystywanych przy zapisie na wydarzenie.

Zgłoszenie (registration) Wniosek o udział w wydarzeniu. Statusy dyplomowe: `PENDING` (oczekuje), `ACCEPTED` (wstępnie przyjęte), `CONFIRMED` (potwierdzone), `CANCELED` (anulowane).

Model „zapłać później” Ścieżka płatnego wydarzenia bez bramki online: po `ACCEPTED` kierowca dokłada dowód przelewu, a organizator potwierdza miejsce (`CONFIRMED`).

carMatch Mechanizm dopasowania kategorii / klasy pojazdu do wymagań wydarzenia (m.in. aliasy nazw).

Workspace organizatora Panel zarządzania pojedynczym wydarzeniem (zgłoszenia, klasy, wyniki, komunikacja, wskaźniki).

Lista startowa Publiczny wykaz potwierdzonych uczestników wydarzenia (po stronie weba).

Organizator Użytkownik z rolą `ORGANIZER`, uprawniony do tworzenia i zarządzania własnymi wydarzeniami oraz zgłoszeniami.

Administrator Użytkownik z rolą `ADMIN`, moderujący wydarzenia, wnioski o rolę organizatora oraz role kont.

Backoffice Osobny host administracyjny (`bo.raceportal.pl`) przeznaczony wyłącznie dla roli `ADMIN`.

Wizytówka mobilna Aplikacja Expo ograniczona do przeglądania katalogu; zapis i konto realizowane są w wersji webowej.

CTA Call to Action — wezwanie do działania; w mobilce przycisk otwierający zapis na stronie webowej.

PWA Progressive Web App — aplikacja webowa instalowalna na urządzeniu (manifest, service worker), uzupełniająca ścieżkę mobilną obok Expo.

API REST Interfejs HTTP backendu RacePortal (JSON) współdzielony przez web i mobile.

JWT JSON Web Token — token dostępu API; w warstwie webowej w ciasteczkach httpOnly (`rp_at` / `rp_rt`), nie w `localStorage`.

RBAC Role-Based Access Control — kontrola dostępu oparta o role (`USER` / `ORGANIZER` / `ADMIN`).

ASVS Application Security Verification Standard (OWASP) — poziom wymagań bezpieczeństwa aplikacji; w pracy przyjęto orientacyjnie poziom L1 adekwatny do MVP.

OWASP Open Worldwide Application Security Project — zbiór praktyk i kategorii zagrożeń aplikacji webowych wykorzystywanych przy analizie bezpieczeństwa.

IDOR Insecure Direct Object Reference — dostęp do cudzego zasobu poprzez manipulację identyfikatorem w żądaniu API.

XSS Cross-Site Scripting — wstrzyknięcie złośliwego skryptu do treści wyświetlanej w przeglądarce użytkownika.

Stored XSS Trwały XSS: szkodliwy skrypt zapisany w danych (np. URL wydarzenia) i wykonywany u innych użytkowników przy odczycie.

CSRF Cross-Site Request Forgery — wymuszenie niechcianego żądania w sesji zalogowanego użytkownika.

SameSite Atrybut ciasteczka ograniczający wysyłanie go w żądaniach cross-site (łagodzi CSRF).

CSP Content Security Policy — nagłówek HTTP ograniczający źródła skryptów i zasobów ładowanych przez przeglądarkę.

CORS Cross-Origin Resource Sharing — reguły zezwalające (lub blokujące) żądania API z innych originów przeglądarki.

HSTS HTTP Strict Transport Security — polityka wymuszająca połączenia HTTPS po stronie przeglądarki.

TLS Transport Layer Security — protokół szyfrowania transportu; w produkcji RacePortal terminowany na Cloudflare.

httpOnly Atrybut ciasteczka uniemożliwiający odczyt wartości przez JavaScript w przeglądarce (ochrona przed kradzieżą tokenu przez XSS).

Access token / refresh token Krótkotrwały token dostępu JWT oraz dłuższy token odświeżający (w RacePortal: ciasteczka `rp_at` / `rp_rt`; refresh w bazie jako skrót).

tokenVersion Wersja tokenu w payloadzie JWT; podnoszona przy wylogowaniu lub zmianie hasła, by unieważnić wcześniejsze sesje.

BCrypt Algorytm skrótu haseł używany przy przechowywaniu poświadczeń lokalnych.

Brute-force Automatyczne odgadywanie hasła wieloma próbami; chronione m.in. rate limitingiem (HTTP 429).

Credential stuffing Próby logowania listą skradzionych par e-mail/hasło z innych serwisów.

Enumeracja kont Ustalanie, czy dany e-mail/login istnieje, na podstawie różnic odpowiedzi API (czas, komunikat, kod).

HTTP 429 Too Many Requests — odpowiedź limitująca nadużycia; w RacePortal z nagłówkiem `Retry-After`.

Rate limiting Ograniczanie liczby prób / żądań w jednostce czasu (m.in. ochrona logowania przed brute-force).

SafeHttpUrls Walidator serwerowy akceptujący wyłącznie bezpieczne URL-e (`http`/`https` lub ścieżki uploadu), blokujący m.in. `javascript:`.

AttemptRateLimiter Mechanizm limitujący nieudane próby logowania per e-mail i IP.

Seed Wstępne dane demonstracyjne (konta, wydarzenia) ładowane przy starcie API; na ekspozycji publicznej powinny być wyłączone.

Hardening Wzmocnienie konfiguracji i kodu pod kątem bezpieczeństwa (nagłówki, sekrety, limity, redukcja powierzchni ataku).

Obrona wielowarstwowa Stosowanie wielu niezależnych zabezpieczeń (tu: Cloudflare $\rightarrow$ nginx $\rightarrow$ Spring $\rightarrow$ MySQL), tak by awaria jednej warstwy nie otwierała całego systemu.

Zasada najmniejszych uprawnień Nadawanie użytkownikowi i komponentowi tylko tych uprawnień, które są niezbędne do roli.

Remedacja Naprawa lub obejście luki znalezionej w teście bezpieczeństwa, wraz z weryfikacją, że problem nie wraca.

SecOps Security Operations — praktyki utrzymania i weryfikacji bezpieczeństwa systemów (w tym testy penetracyjne).

SAST Static Application Security Testing — automatyczna analiza kodu źródłowego pod kątem luk; poza pełnym zakresem MVP.

WAF Web Application Firewall — filtr ruchu HTTP chroniący aplikację przed typowymi atakami; poza zakresem MVP.

2FA Two-Factor Authentication — dwuskładnikowe uwierzytelnianie; świadomie poza zakresem MVP.

Testy penetracyjne Kontrolowane próby znalezienia luk bezpieczeństwa na powierzchni ataku (tu: HTTP(S) produkcji RacePortal) wraz z rekomendacjami remedacji.

MockMvc Warstwa testów integracyjnych Springa symulująca żądania HTTP do kontrolerów bez pełnego serwera.

OAuth Google Opcjonalne logowanie kontem Google (obok logowania hasłem).

OSRM Open Source Routing Machine — publiczny serwis wyznaczania trasy dojazdu (domyślny provider).

Docker Compose Narzędzie uruchamiania zestawu kontenerów (web, API, MySQL, backup) jako wspólnego środowiska lokalnego i produkcyjnego.

Cloudflare Tunnel Tunel zapewniający publiczny dostęp HTTPS do usług na maszynie wirtualnej (bez otwierania portów na firewallu VM).

nginx Serwer HTTP / reverse proxy na VM RacePortal (nagłówki bezpieczeństwa, proxy do API i frontu).

Mailbux Zewnętrzny SMTP używany do wysyłki maili transakcyjnych RacePortal.

PSP Payment Service Provider — natywna bramka płatności online; świadomie poza zakresem MVP.

MVP Minimum Viable Product — minimalny zakres funkcji przyjęty do odbioru projektu.

KP Karta projektu — formalny dokument planistyczny (cele, miary sukcesu, ramy odbioru).

DZW Dokument Założeń Wstępnych — opis problemu, celów i zakresu MVP RacePortal.

SWS Specyfikacja Wymagań Systemowych — uszczegółowienie wymagań jako podstawa implementacji i testów.

BMC Business Model Canvas — szablon opisu modelu biznesowego (segmenty, wartość, kanały, koszty itd.).

SWOT Strengths, Weaknesses, Opportunities, Threats — analiza mocnych/słabych stron oraz szans i zagrożeń.

SPA Single Page Application — aplikacja webowa ładująca UI po stronie klienta (tu: React/Vite).

DTO Data Transfer Object — obiekt kontraktu API oddzielający wejście/wyjście HTTP od modelu bazy.

Bean Validation Jakarta Bean Validation — deklaratywna walidacja pól DTO (adnotacje, `@Valid`).

Spring Security Framework zabezpieczeń Springa (uwierzytelnianie, autoryzacja, filtry HTTP) używany w API.

JJWT Biblioteka Java do wystawiania i weryfikacji tokenów JWT (HMAC-SHA).

TypeScript Nadzbiór JavaScriptu z typami statycznymi; używany we froncie webowym i w `packages/api-t`

Vite Bundler / serwer deweloperski frontendu RacePortal.

Tailwind CSS Biblioteka utility-first CSS do stylowania UI webowego.

Leaflet Biblioteka map interaktywnych w przeglądarce (katalog / szczegóły wydarzenia).

SMTP Simple Mail Transfer Protocol — protokół wysyłki poczty; w projekcie przez Mailbux.

Cloudflare Brzeg sieci produkcji RacePortal (DNS, TLS/HSTS, Tunnel); bez hostowania frontu na Pages.

Playwright Framework E2E do automatycznych testów przeglądarkowych aplikacji webowej.

Vitest Runner testów jednostkowych w ekosystemie Vite (m.in. `api-types`, mobile).

JUnit Framework testów jednostkowych/integracyjnych Javy (backend RacePortal).

E2E End-to-End — test pełnej ścieżki użytkownika w przeglądarce (tu: Playwright).

WebAuthn Standard silnych kluczy / passkeys; poza zakresem MVP (wzmianka przy 2FA).

CAPTCHA Test odróżniający człowieka od bota; poza zakresem MVP (wzmianka przy hardeningu).

RPS Requests Per Second — metryka obciążenia (liczba żądań na sekundę); w projekcie mierzona m.in. narzędziem k6.

k6 Narzędzie Grafana do testów obciążeniowych HTTP (pomiary RPS na VM).

Expo Framework do budowy aplikacji React Native; wykorzystany w kliencie mobilnym RacePortal.

Track day Impreza torowa bez klasycznej rywalizacji wyścigowej — jazda treningowa na torze.

USP Unique Selling Proposition — wyróżnik oferty; w RacePortal m.in. garaż + `carMatch` oraz panel organizatora na jednym katalogu.

Live demo Demonstracja systemu na żywo (produkcja lub Compose) według ustalonego scenariusza obrony.

Konto seed Konto demonstracyjne ładowane przy starcie API (kierowca / organizator / admin) do powtarzalnych pokazów i testów.

Oracle Always Free Warstwa darmowych zasobów Oracle Cloud; w projekcie VM Ampere hostująca Compose za Cloudflare Tunnel.

Filtr „pod moje auto” Widok katalogu ograniczony do wydarzeń, do których pasuje wybrany pojazd z garażu (wykorzystuje `carMatch`).

Monorepo Jedno repozytorium Git zawierające wiele pakietów/aplikacji (tu: `web/`, `backend/`, `mobile/`, `packages/`).

Fullstack Zakres prac obejmujący więcej niż jedną warstwę systemu (API, web, integracja, wdrożenie) — w praktyce RacePortal przypisany m.in. do roli testów i bezpieczeństwa.

DataInitializer Komponent startowy API ładujący dane seed (konta demo, przykładowe wydarzenia) przy uruchomieniu aplikacji.

Object storage Zewnętrzna usługa przechowywania plików (np. S3); w MVP zamiast niej lokalny upload na dysk serwera.

Cloudflare Pages Usługa hostingu frontendu statycznego Cloudflare; rozważana we wczesnych eksperymentach, ostatecznie porzucona na rzecz Compose + Tunnel.

Waitlista Kolejka rezerwowa przy braku miejsc; w MVP poza zakresem (kierunek rozwoju).

CTA webowe Przekierowanie z wizytówki mobilnej do pełnej ścieżki zapisu w aplikacji webowej / PWA.

Rozdział 3

Opis problemu

3.1 Charakterystyka rynku wydarzeń motorsportowych

Amatorski i półprofesjonalny motorsport w Polsce przeżywa w ostatnich latach dynamiczny rozwój. Obejmuje on liczne, zróżnicowane formy aktywności, takie jak wydarzenia typu *track day*, zawody w drifcie, rajdy amatorskie (KJS, Super OES), wyścigi górskie czy zmagania *time attack*. Infrastruktura, od dużych obiektów pokroju Toru Poznań czy Silesia Ring, po mniejsze tory szkoleniowe i lokalne obiekty, pozwala na organizację imprez o różnym stopniu zaawansowania.

Mimo rosnącej popularności tej formy spędzania czasu, rynek ten charakteryzuje się dużym rozproszeniem organizacyjnym. Informacje o terminach, regulaminach oraz procedurach rejestracji są często rozrzucone pomiędzy indywidualnymi stronami internetowymi organizatorów, fanpage'ami w mediach społecznościowych (głównie na platformie Facebook) oraz tematycznymi forami dyskusyjnymi. Brakuje jednego, wspólnego miejsca, które zbierałoby te informacje w przejrzysty sposób i ułatwiało ich odnalezienie.

3.2 Problemy związane z organizacją wydarzeń i zapisami uczestników

Z perspektywy organizatorów, zarządzanie mniejszymi i średnimi wydarzeniami motorsportowymi wiąże się z szeregiem wyzwań logistycznych. Większość podmiotów wykorzystuje do zbierania zgłoszeń podstawowe i niepołączone ze sobą narzędzia, takie jak formularze Google, arkusze kalkulacyjne czy wiadomości e-mail.

Ręczne przetwarzanie zgłoszeń jest nie tylko czasochłonne, ale i wysoce podatne na błędy ludzkie (np. zgubienie zgłoszenia, błędne przypisanie do klasy startowej). Poważnym problemem jest również weryfikacja techniczna uczestników — organizator musi ręcznie oceniać, czy dany pojazd spełnia wymagania techniczne dla danej kategorii wydarzenia (np. pojemność silnika, napęd, klatka bezpieczeństwa). Dodatkowo, komunikacja z uczestnikami odnośnie statusu

ich zgłoszenia (oczekujące, zatwierdzone, odrzucone) wymaga indywidualnej wysyłki wiadomości, co staje się trudne do skalowania przy rosnącej liczbie uczestników na listach startowych i rezerwowych.

3.3 Problemy informacyjne po stronie kierowców i organizatorów

Rozproszenie danych o imprezach dotkliwie uderza również w samych uczestników. Kierowcy mają ogromną trudność ze znalezieniem aktualnych wydarzeń startowych w jednym miejscu. Aby zaplanować sezon lub chociażby weekendowy wyjazd na tor, muszą oni na bieżąco monitorować wiele niezależnych źródeł.

Często proces poszukiwania informacji sprowadza się do przedzierania się przez nieczytelne posty w mediach społecznościowych, gdzie kluczowe dane, takie jak koszty wpisowego, harmonogram czy limit miejsc, są trudne do odnalezienia. Następnie, uczestnik staje przed problemem oceny, czy jego samochód w ogóle kwalifikuje się do wybranej imprezy. Brak centralnego kalendarza oraz ujednoliconego, przejrzystego interfejsu sprawia, że cały proces — od znalezienia wydarzenia, po udany start — jest niepotrzebnie skomplikowany, a komunikacja na linii organizator–kierowca jest chaotyczna i opóźniona.

3.4 Uzasadnienie potrzeby stworzenia systemu RacePortal

Projekt platformy RacePortal jest bezpośrednią odpowiedzią na wyżej zidentyfikowane problemy. Jego głównym założeniem jest dostarczenie **centralnego katalogu wydarzeń** oraz zautomatyzowanego, **ujednoliconego procesu zapisów**, w którym obie strony (kierowca i organizator) operują na tym samym zbiorze i formacie danych.

Stworzenie dedykowanego rozwiązania pozwoli na:

- zebranie wszystkich typów wydarzeń motorsportowych w łatwej do przeszukiwania bazie z widokiem mapy, listy i kalendarza;
- wprowadzenie mechanizmu „wirtualnego garażu” dla uczestników, z poziomu którego ich pojazdy mogą być automatycznie zestawiane z wymogami wydarzeń (mechanizm *carMatch*);
- uporządkowanie ról w systemie (Gość, Użytkownik/Kierowca, Organizator, Administrator), co usprawni przepływ informacji, walidację zgłoszeń oraz moderację treści;
- zapewnienie użytkownikom dostępu poprzez ścieżkę webową (pełna funkcjonalność) oraz mobilną wizytówkę (ułatwione odkrywanie wydarzeń z poziomu smartfona).

3.5 Aspekty techniczne i organizacyjne problemu

Rozwiązanie zdefiniowanych problemów biznesowych pociąga za sobą konieczność sprostania konkretnym wyzwaniom technicznym. Głównym zadaniem jest zaprojektowanie wydajnego i spójnego modelu relacyjnej bazy danych, który sprawnie powiąże wydarzenia, użytkowników, pojazdy z ich parametrami oraz listę zgłoszeń i ich statusy.

Kluczowe zagadnienia techniczne podjęte w ramach projektu obejmują:

- **Bezpieczeństwo i autoryzację:** implementację bezstanowego uwierzytelniania opartego o tokeny JWT oraz ścisłej, opartej na rolach kontroli dostępu (RBAC) do poszczególnych zasobów API;
- **Walidację logiczną:** realizację mechanizmów sprawdzających wymagania wydarzenia względem pojazdu (moduł *carMatch*);
- **Niezawodną komunikację asynchroniczną:** wdrożenie systemu powiadomień e-mail dostarczającego uczestnikom zmiany statusów, również w środowiskach izolowanych.

Rozdział 4

Kontekst biznesowy

Celem niniejszego rozdziału jest przedstawienie biznesowych i organizacyjnych uwarunkowań funkcjonowania systemu RacePortal. Analiza obejmuje identyfikację grup docelowych oraz interesariuszy przedsięwzięcia, określenie propozycji wartości, kanałów komunikacji i dystrybucji, niezbędnych zasobów, a także potencjalnego modelu biznesowego i możliwości dalszego rozwoju platformy.

RacePortal stanowi odpowiedź na problem rozproszenia informacji o wydarzeniach motorsportowych organizowanych na terenie Polski. Głównym założeniem przedsięwzięcia jest utworzenie scentralizowanej platformy umożliwiającej wyszukiwanie wydarzeń, zarządzanie zgłoszeniami uczestników oraz usprawnienie komunikacji pomiędzy kierowcami i organizatorami.

Analiza biznesowa opiera się na założeniach projektowych oraz funkcjonalnościach opracowanego systemu. Przedstawione możliwości komercjalizacji i rozwoju stanowią propozycje kierunków działania, a nie potwierdzone wyniki badań rynkowych ani prognozy finansowe.

4.1 Segmentacja użytkowników

Segmentacja użytkowników stanowi istotny element analizy biznesowej, ponieważ pozwala określić potrzeby poszczególnych grup odbiorców oraz dopasować do nich oferowane funkcjonalności systemu. W przypadku RacePortal wyróżniono trzy podstawowe segmenty rynku: uczestników wydarzeń motorsportowych, organizatorów wydarzeń oraz kibiców i pasjonatów motoryzacji.

Uczestnicy wydarzeń motorsportowych (kierowcy) stanowią jedną z głównych grup docelowych platformy. Segment obejmuje osoby zainteresowane zarówno amatorskim, jak i bardziej zaawansowanym udziałem w wydarzeniach, takich jak track day, drifting, rajdy czy inne formy rywalizacji samochodowej. Do ich najważniejszych potrzeb należą łatwy dostęp do aktualnego kalendarza wydarzeń, możliwość wyszukiwania imprez według lokalizacji i kategorii oraz sprawna rejestracja udziału.

Istotnym elementem z punktu widzenia tej grupy jest możliwość utworzenia profilu kierowcy i przypisania do niego pojazdów. Pozwala to wykorzystać informacje o posiadanym samochodzie podczas wyboru wydarzeń oraz wstępnego sprawdzania zgodności pojazdu z określonymi wymaganiami.

Organizatorzy wydarzeń motorsportowych stanowią drugi kluczowy segment platformy. Są to między innymi kluby motoryzacyjne, podmioty zarządzające torami wyścigowymi, organizatorzy imprez amatorskich oraz przedsiębiorstwa prowadzące wydarzenia komercyjne.

Podstawową potrzebą tej grupy jest skuteczna promocja wydarzeń oraz usprawnienie obsługi uczestników. RacePortal umożliwia organizatorom publikowanie informacji o imprezach, określanie warunków uczestnictwa oraz zarządzanie zgłoszeniami. Centralizacja tych działań może ograniczyć konieczność korzystania z wielu niezależnych kanałów komunikacji.

Kibice i pasjonaci motoryzacji tworzą trzeci segment użytkowników. Ich głównym celem jest pozyskiwanie informacji o nadchodzących wydarzeniach, terminach, lokalizacjach i charakterze imprez. W przeciwieństwie do kierowców nie muszą oni korzystać z mechanizmu zgłoszeń. Dostęp do publicznego katalogu bez konieczności rejestracji pozwala tej grupie korzystać z podstawowych funkcjonalności platformy.

Niezależnie od przedstawionej segmentacji biznesowej w systemie funkcjonuje również rola administratora. Nie stanowi ona odrębnego segmentu rynku, lecz reprezentuje podmiot odpowiedzialny za nadzór nad prawidłowym działaniem platformy, moderowanie treści oraz zarządzanie uprawnieniami użytkowników.

Przyjęta segmentacja pozwala wyróżnić dwie grupy o największym znaczeniu dla funkcjonowania platformy: kierowców, którzy poszukują wydarzeń i zgłaszają udział, oraz organizatorów, którzy dostarczają informacje o wydarzeniach. Pozostali odbiorcy zwiększają potencjalny zasięg i rozpoznawalność systemu.

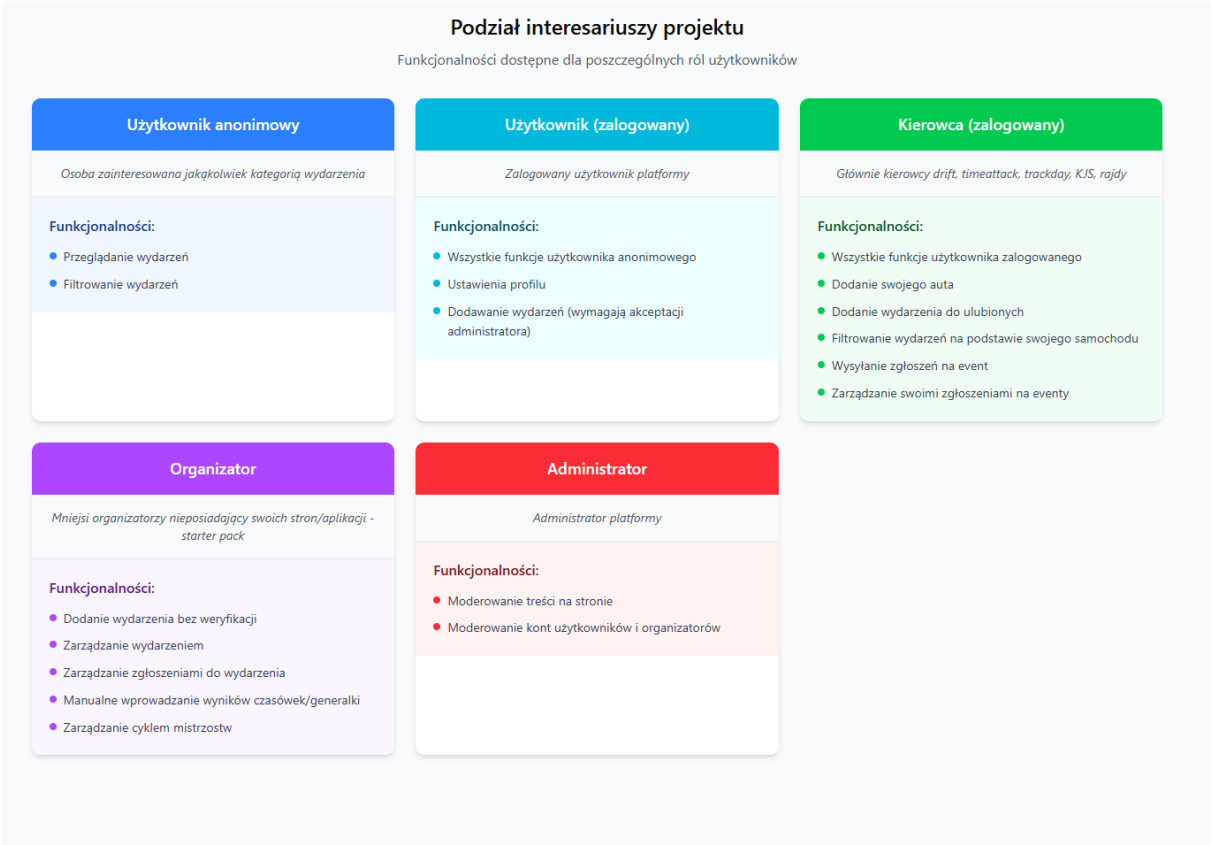
Wczesny, koncepcyjny podział funkcjonalności według poziomów dostępu (materiał projektowy zespołu) przedstawia rys. 4.1. Należy go interpretować jako mapę intencji produktowej; szczegółowe mapowanie na wymagania i projekt systemu opisano w rozdziałach 7–8.

4.2 Interesariusze systemu

Interesariuszami systemu RacePortal są osoby, organizacje i podmioty, na które funkcjonowanie platformy może wywierać bezpośredni lub pośredni wpływ. Ich identyfikacja pozwala określić oczekiwania wobec systemu, zakres odpowiedzialności oraz potencjalne konflikty interesów.

Do najważniejszych interesariuszy należą kierowcy, organizatorzy wydarzeń, administrator platformy, jej właściciel lub operator oraz podmioty współpracujące z organizatorami, w szczególności właściciele obiektów motorsportowych.

Z punktu widzenia **kierowcy** najistotniejszym wymaganiem jest zapewnienie aktualnych



Rysunek 4.1: Podział interesariuszy projektu — funkcjonalności dostępne dla ról użytkowników (materiał koncepcyjny).

i wiarygodnych informacji o wydarzeniach. Szczególne znaczenie mają czytelne warunki uczestnictwa, możliwość sprawdzenia statusu zgłoszenia oraz dostęp do informacji o ewentualnych zmianach.

Organizator oczekuje przede wszystkim narzędzia ograniczającego liczbę czynności administracyjnych. Istotne jest dla niego zarządzanie wydarzeniami z jednego miejsca, możliwość weryfikowania zgłoszeń oraz komunikowanie decyzji uczestnikom.

Administrator odpowiada za utrzymanie odpowiedniej jakości publikowanych treści i kontrolę przyznawanych uprawnień. Jego zadaniem jest również ograniczanie ryzyka publikacji nieprawdziwych, nieaktualnych lub niezgodnych z przeznaczeniem platformy informacji.

Operator platformy odpowiada za długoterminowe utrzymanie i rozwój systemu. Z jego perspektywy najważniejsze są koszty funkcjonowania usługi, możliwość pozyskiwania użytkowników, bezpieczeństwo informacji oraz skalowalność przyjętego rozwiązania.

Właściciele obiektów motorsportowych mogą być zarówno organizatorami wydarzeń, jak i podmiotami współpracującymi z zewnętrznymi organizatorami. Platforma może stanowić dla nich dodatkowy kanał promocji obiektów i organizowanych na ich terenie imprez.

Wskazane grupy różnią się zakresem oczekiwań, jednak ich interesy są wzajemnie powiązane. Wzrost liczby organizatorów i publikowanych wydarzeń zwiększa użyteczność platformy

dla kierowców, natomiast większa liczba aktywnych uczestników może zachęcać kolejnych organizatorów do korzystania z systemu.

Poniżej przedstawiono karty kluczowych udziałowców w formacie środowiska `stakeholder` szablonu PJATK. Karty uszczegóławiają oczekiwania, ograniczenia oraz powiązania z wymaganiami i projektem systemu.

KARTA UDZIAŁOWCA	
Identyfikator:	U-00
Nazwa:	Gość / kibic (użytkownik anonimowy)
Opis:	Odbiorca informacji przeglądający publiczny katalog wydarzeń bez zakładania konta.
Typ udziałowca:	Użytkownik końcowy / odbiorca treści
Punkt widzenia:	Chce szybko znaleźć aktualne wydarzenia, lokalizacje i terminy bez bariery rejestracji.
Ograniczenia:	Bez konta nie może złożyć zgłoszenia ani prowadzić garażu; zależny od kompletności i aktualności katalogu publikowanego przez organizatorów.
Wymagania:	Publiczny katalog i filtry — rozdz. 7; widoki lista/mapa/kalendarz — rozdz. 11.

Tabela 4.1: Karta udziałowca: Gość / kibic

KARTA UDZIAŁOWCA	
Identyfikator:	U-01
Nazwa:	Kierowca
Opis:	Użytkownik z kontem, który utrzymuje garaż pojazdów i składa zgłoszenia na wydarzenia motorsportowe.
Typ udziałowca:	Użytkownik końcowy / klient platformy
Punkt widzenia:	Chce szybko znaleźć starty, dopasować pojazd do wymagań wydarzenia i śledzić status zgłoszenia w jednym miejscu.
Ograniczenia:	Korzystanie z garażu i zapisów wymaga rejestracji oraz podania danych niezbędnych do obsługi zgłoszenia; skuteczność zapisu zależy od odpowiedzi organizatora i kompletności wymagań wydarzenia.
Wymagania:	Wymagania funkcjonalne katalogu, garażu i zgłoszeń — rozdz. 7; panel użytkownika — rozdz. 11.

Tabela 4.2: Karta udziałowca: Kierowca

4.3 Propozycja wartości systemu

Propozycja wartości RacePortal opiera się na centralizacji informacji o wydarzeniach motorsportowych oraz uproszczeniu procesu wyszukiwania i rejestracji uczestników.

KARTA UDZIAŁOWCA	
Identyfikator:	U-02
Nazwa:	Organizator
Opis:	Podmiot publikujący wydarzenia i zarządzający listą zgłoszeń uczestników.
Typ udziałowca:	Dostawca treści / partner biznesowy
Punkt widzenia:	Potrzebuje skutecznego kanału dotarcia do uczestników oraz narzędzia do zarządzania wydarzeniami, weryfikacji zgłoszeń i komunikacji statusów.
Ograniczenia:	Publikacja wymaga pozytywnej weryfikacji uprawnień; rozliczanie wpisowego odbywa się poza platformą, więc organizator nadal prowadzi proces płatności we własnym zakresie; wartość panelu zależy od liczby aktywnych kierowców.
Wymagania:	Panel organizatora i kontrola dostępu — rozdz. 7, 8; realizacja — rozdz. 10.

Tabela 4.3: Karta udziałowca: Organizator

KARTA UDZIAŁOWCA	
Identyfikator:	U-03
Nazwa:	Administrator (rola operacyjna)
Opis:	Operator platformy moderujący wydarzenia, role kont oraz wnioski o uprawnienia organizatora.
Typ udziałowca:	Właściciel / operator systemu
Punkt widzenia:	Zapewnia jakość katalogu, spójność uprawnień i bezpieczeństwo działania platformy.
Ograniczenia:	Skuteczność moderacji zależy od dostępności operatora i jakości zgłoszeń od użytkowników; przy wzroście skali rośnie nakład pracy na weryfikację treści i ról.
Wymagania:	Panel administracyjny oraz bezpieczeństwo dostępu — rozdz. 13; testy kontroli dostępu — rozdz. 14.

Tabela 4.4: Karta udziałowca: Administrator

W tradycyjnym modelu informacje o imprezach mogą być publikowane w wielu niezależnych miejscach, między innymi na stronach internetowych organizatorów, w mediach społecznościowych, na stronach obiektów sportowych oraz w komunikatorach. Powoduje to konieczność samodzielnego wyszukiwania i porównywania informacji pochodzących z różnych źródeł.

RacePortal proponuje ujednolicony sposób prezentacji wydarzeń poprzez wspólny katalog, który umożliwia ich przeglądanie w formie listy, kalendarza oraz mapy. Zastosowanie mechanizmów wyszukiwania i filtrowania pozwala użytkownikom zawęzić wyniki zgodnie z wybranymi kryteriami.

Dla kierowców szczególną wartością jest połączenie procesu wyszukiwania wydarzenia z możliwością zgłoszenia udziału. Użytkownik może przejść od zapoznania się z ofertą wydarzenia do przesłania zgłoszenia i późniejszego monitorowania jego statusu w ramach jednego

KARTA UDZIAŁOWCA	
Identyfikator:	U-04
Nazwa:	Właściciel toru / partner obiektu
Opis:	Podmiot dysponujący infrastrukturą (tor, obiekt) wykorzystywaną przy organizacji wydarzeń publikowanych na platformie.
Typ udziałowca:	Partner pośredni / interesariusz otoczenia
Punkt widzenia:	Jest zainteresowany widocznością obiektu oraz poprawnością informacji o lokalizacji wydarzeń odbywających się na jego terenie.
Ograniczenia:	Nie zarządza bezpośrednio zapisami uczestników w MVP; wpływ na treść katalogu jest pośredni (poprzez organizatorów lub kontakt z operatorem); nie posiada dedykowanego panelu partnerskiego.
Wymagania:	Katalog lokalizacji i dane wydarzeń — rozdz. 7, 8.

Tabela 4.5: Karta udziałowca: Właściciel toru / partner obiektu

systemu.

Dodatkowym elementem jest funkcjonalność wirtualnego garażu. Przechowywanie danych pojazdów umożliwia ich ponowne wykorzystywanie podczas zgłaszania się na kolejne wydarzenia, bez konieczności każdorazowego wprowadzania tych samych informacji. Mechanizm dopasowania pojazdu stanowi wsparcie informacyjne, nie zastępując ostatecznej weryfikacji wymagań technicznych przez organizatora.

Z punktu widzenia organizatorów główną wartością jest uporządkowanie procesu obsługi wydarzeń. Możliwość zarządzania zgłoszeniami oraz komunikacji statusów może zmniejszyć liczbę czynności wykonywanych ręcznie i ograniczyć ryzyko pomyłek wynikających z prowadzenia zapisów w wielu niezależnych narzędziach.

Dla kibiców i pasjonatów wartością jest natomiast bezpłatny dostęp do uporządkowanych informacji o wydarzeniach, bez konieczności zakładania konta.

Najważniejszym wyróżnikiem RacePortal jest zatem połączenie funkcji informacyjnych i organizacyjnych. Platforma nie ogranicza się wyłącznie do prezentowania kalendarza imprez, lecz obejmuje również podstawowy proces zarządzania udziałem w wydarzeniu.

Tabela 4.6 przedstawia porównanie modelu działania grup społecznościowych z modelem RacePortal w ujęciu koncepcyjnym.

4.4 Kanały komunikacji i dystrybucji

Kanały komunikacji i dystrybucji określają sposób udostępniania platformy odbiorcom, pozyskiwania nowych użytkowników oraz utrzymywania relacji pomiędzy uczestnikami ekosystemu RacePortal.

Tabela 4.6: Porównanie modelu grup społecznościowych z modelem RacePortal (ujęcie koncepcyjne).

Kryterium	**Grupy społecznościowe**	**RacePortal (model docelowy)**
Wyszukiwanie wydarzeń	Zależne od postów i wyszukiwarki grupy	Ustrukturyzowane filtry i wspólny katalog
Porównanie wydarzeń	Ręczne, między rozproszonymi źródłami	Wspólny katalog z ujednoliconym opisem
Dopasowanie pojazdu	Weryfikacja po stronie uczestnika	Wsparcie dopasowania względem wymagań wydarzenia
Zgłoszenie udziału	Zależne od praktyki danego organizatora	Ujednolicony proces zgłoszenia
Status zgłoszenia	Różne kanały komunikacji	Centralny podgląd statusu i powiadomienia

Podstawowym kanałem dystrybucji jest aplikacja internetowa dostępna pod adresem `https://raceportal.pl`. Za jej pośrednictwem użytkownicy mogą przeglądać wydarzenia, rejestrować konta, zarządzać pojazdami, zgłaszać udział oraz korzystać z funkcji przeznaczonych dla organizatorów i administratorów.

Aplikacja internetowa została również dostosowana do korzystania na urządzeniach mobilnych. Zastosowanie rozwiązania typu Progressive Web App umożliwia dodanie skrótu do platformy na ekranie głównym urządzenia.

Kanałem uzupełniającym jest aplikacja mobilna umożliwiająca przeglądanie katalogu wydarzeń oraz ich szczegółów. W obecnym zakresie pełna obsługa konta i zgłoszeń realizowana jest przez aplikację internetową, do której użytkownik może zostać przekierowany.

Istotną rolę w komunikacji z zarejestrowanymi użytkownikami odgrywa poczta elektroniczna. Jest ona wykorzystywana do przekazywania wiadomości związanych z obsługą konta, potwierdzeniami zgłoszeń oraz zmianami ich statusów. Taki sposób komunikacji umożliwia informowanie użytkowników bez konieczności ciągłego monitorowania platformy.

W kontekście pozyskiwania nowych odbiorców można wyróżnić również potencjalne kanały marketingowe. Należą do nich media społecznościowe związane z motorsportem, współpraca z klubami i organizatorami, promocja podczas wydarzeń oraz działania zwiększające widoczność witryny w wyszukiwarkach internetowych.

Wymienione działania marketingowe należy traktować jako proponowany kierunek rozwoju, wymagający osobnej strategii i oceny skuteczności.

Z perspektywy biznesowej szczególne znaczenie ma współpraca z organizatorami. Udostępnianie przez nich odnośników do własnych wydarzeń publikowanych w RacePortal mogłoby jednocześnie promować platformę i zwiększać jej rozpoznawalność wśród kierowców.

4.5 Kluczowe zasoby i działania

Funkcjonowanie RacePortal wymaga odpowiednich zasobów technologicznych, ludzkich, informacyjnych i organizacyjnych.

Do podstawowych zasobów technologicznych należy kod źródłowy aplikacji internetowej, serwera aplikacyjnego i aplikacji mobilnej, baza danych oraz infrastruktura umożliwiająca udostępnianie systemu użytkownikom. W projekcie wykorzystano rozwiązania oparte na Spring Boot, React, MySQL i Docker, co pozwala rozwijać poszczególne części systemu oraz uruchamiać je w powtarzalnym środowisku.

Istotnym zasobem ludzkim jest pięcioosobowy zespół projektowy odpowiadający za analizę wymagań, projektowanie, implementację, testowanie i dokumentację systemu. W przypadku dalszego rozwoju komercyjnego konieczne byłoby również określenie odpowiedzialności za utrzymanie infrastruktury, kontakt z organizatorami oraz obsługę zgłoszeń użytkowników.

Ważnym zasobem informacyjnym jest baza wydarzeń motorsportowych, obejmująca ich terminy, lokalizacje, kategorie i warunki uczestnictwa. Wartość tego zasobu zależy nie tylko od liczby zgromadzonych wydarzeń, ale również od ich aktualności, kompletności i wiarygodności.

Do kluczowych działań związanych z funkcjonowaniem platformy należą utrzymanie systemu, rozwijanie funkcjonalności, weryfikacja organizatorów, moderowanie wydarzeń, zapewnianie bezpieczeństwa danych oraz obsługa zgłaszanych problemów.

Z biznesowego punktu widzenia szczególnie ważnym działaniem jest pozyskiwanie organizatorów i budowanie relacji z podmiotami działającymi na rynku motorsportowym. Bez wystarczającej liczby aktualnych wydarzeń nawet poprawnie działająca platforma może mieć ograniczoną wartość dla użytkowników.

W dłuższej perspektywie konieczne byłoby również monitorowanie sposobu korzystania z systemu, identyfikowanie najczęściej wykorzystywanych funkcji oraz analizowanie potrzeb użytkowników. Pozwoliłoby to podejmować decyzje rozwojowe na podstawie rzeczywistych danych, a nie wyłącznie początkowych założeń projektowych.

4.6 Model działania platformy

RacePortal można zaklasyfikować jako platformę dwustronną, której funkcjonowanie opiera się na współpracy dwóch podstawowych grup: organizatorów wydarzeń oraz potencjalnych uczestników.

Organizatorzy odpowiadają za dostarczanie informacji o wydarzeniach i obsługę zgłoszeń, natomiast kierowcy korzystają z tych informacji oraz generują zainteresowanie uczestnictwem w imprezach.

W przyjętym modelu organizator uzyskuje odpowiednie uprawnienia, a następnie przygotowuje i publikuje informacje o wydarzeniu zgodnie z procedurą obowiązującą w systemie. Kierowca wyszukuje interesującą go imprezę, zapoznaje się z wymaganiami i przesyła zgłoszenie. Organizator może następnie przeprowadzić jego weryfikację oraz przekazać informację o podjętej decyzji.

Taki model umożliwia uporządkowanie relacji pomiędzy organizatorem a uczestnikiem i ograniczenie zależności od indywidualnych sposobów prowadzenia zapisów.

W aktualnym zakresie funkcjonalnym RacePortal nie realizuje automatycznego pobierania opłat za uczestnictwo. W przypadku wydarzeń płatnych rozliczenia odbywają się poza platformą, a system może służyć do przekazania informacji związanych z opłatą i jej potwierdzeniem.

Na etapie opracowanego rozwiązania przyjęto model bezpłatnego dostępu do podstawowych funkcjonalności zarówno dla kierowców, jak i organizatorów. Głównym celem jest weryfikacja użyteczności platformy oraz stworzenie podstaw do rozwoju społeczności użytkowników.

W przyszłości możliwe byłoby wprowadzenie modelu mieszanego, obejmującego bezpłatne korzystanie z podstawowych funkcji oraz dodatkowe usługi komercyjne.

Potencjalne źródła przychodów mogłyby obejmować odpłatne promowanie wydarzeń, oznaczone treści sponsorowane, abonament dla organizatorów korzystających z rozszerzonych narzędzi oraz prowizję od obsługi zapisów po wdrożeniu odpowiedniego mechanizmu płatności.

O wyborze konkretnego modelu powinny decydować wyniki analizy zainteresowania organizatorów, częstotliwość korzystania z platformy oraz gotowość użytkowników biznesowych do ponoszenia opłat.

Działalność platformy wiązałaby się jednocześnie z kosztami utrzymania infrastruktury serwerowej, domeny, usług poczty elektronicznej, przechowywania danych, aktualizacji oprogramowania i obsługi użytkowników. W przypadku zwiększania skali działalności istotnymi kategoriami kosztów mogłyby stać się również promocja oraz praca osób odpowiedzialnych za moderację.

Na obecnym etapie nie ma wystarczających podstaw do określenia rentowności komercyjnego przedsięwzięcia. Wymagałoby to oszacowania liczby aktywnych użytkowników, potencjalnych przychodów i rzeczywistych kosztów operacyjnych.

4.7 Realistyczność wdrożenia i dalsze perspektywy

Ocena możliwości wdrożenia RacePortal powinna uwzględniać zarówno aspekty technologiczne, jak i organizacyjne oraz ekonomiczne.

Pod względem technologicznym opracowano działający system obejmujący katalog wydarzeń, obsługę kont użytkowników, zarządzanie pojazdami i zgłoszeniami oraz funkcje admini-

stracyjne. Aplikacja została również udostępniona w środowisku internetowym, co potwierdza techniczną możliwość uruchomienia rozwiązania poza lokalnym środowiskiem programistycznym.

Nie oznacza to jednak automatycznego potwierdzenia gotowości platformy do działalności komercyjnej na dużą skalę. W takim przypadku niezbędne byłyby dalsze działania dotyczące monitorowania systemu, utrzymania niezawodności, bezpieczeństwa, obsługi użytkowników i zarządzania kosztami infrastruktury.

Istotnym wyzwaniem biznesowym jest pozyskanie odpowiedniej liczby organizatorów wydarzeń. Wartość RacePortal dla kierowców zależy przede wszystkim od dostępności atrakcyjnej i aktualnej oferty imprez. Jednocześnie zainteresowanie organizatorów może być uzależnione od liczby użytkowników aktywnie korzystających z platformy.

Z tego względu uzasadnionym kierunkiem byłoby rozpoczęcie od ograniczonego wdrożenia pilotażowego, obejmującego wybraną grupę organizatorów oraz wydarzenia z określonych kategorii motorsportu. Pozwoliłoby to zweryfikować sposób korzystania z systemu, zidentyfikować problemy organizacyjne oraz zebrać informacje o potrzebach odbiorców.

Skuteczność takiego pilotażu można byłoby mierzyć za pomocą liczby aktywnych organizatorów, liczby opublikowanych rzeczywistych wydarzeń, liczby zakończonych zgłoszeń, odsetka użytkowników powracających do platformy oraz opinii uczestników i organizatorów.

Kolejnym etapem mogłoby być rozszerzenie zakresu funkcjonalnego systemu. Do potencjalnych kierunków rozwoju należą integracja z systemami płatności elektronicznych, dodatkowe mechanizmy powiadomień, możliwość obserwowania wydarzeń, eksport informacji do zewnętrznych kalendarzy oraz dalszy rozwój aplikacji mobilnej.

Dalsze inwestycje powinny być jednak uzależnione od wyników pilotażu i potwierdzonych potrzeb użytkowników. Pozwoliłoby to ograniczyć ryzyko przeznaczania zasobów na funkcje, które nie przynoszą istotnej wartości biznesowej.

Realistyczność rozwoju RacePortal zależy zatem nie tylko od możliwości technologicznych zespołu, ale również od pozyskania partnerów, zbudowania aktualnej bazy wydarzeń i zapewnienia długoterminowego utrzymania usługi.

4.8 Kontekst prawny i etyczny

Funkcjonowanie internetowej platformy obsługującej konta użytkowników i zgłoszenia udziału w wydarzeniach motorsportowych wymaga uwzględnienia zagadnień związanych z ochroną danych osobowych, odpowiedzialnością za publikowane treści oraz bezpieczeństwem korzystania z usługi.

Jednym z podstawowych obszarów jest ochrona danych osobowych. RacePortal przetwarza informacje związane z kontami użytkowników, danymi kontaktowymi, pojazdami oraz zgło-

szeniami na wydarzenia. W związku z tym podmiot odpowiedzialny za działanie platformy powinien zapewnić zgodność przetwarzania danych z rozporządzeniem RODO.

Szczególne znaczenie mają zasady minimalizacji danych, przejrzystości przetwarzania, ograniczenia celu oraz zapewnienia odpowiednich środków bezpieczeństwa. Użytkownicy powinni otrzymywać informacje o celach i podstawach prawnych przetwarzania danych, okresie ich przechowywania oraz przysługujących im prawach.

W systemie przewidziano udostępnienie polityki prywatności i regulaminu korzystania z platformy. Istotnym elementem dalszego rozwoju organizacyjnego jest również określenie procedur realizacji praw użytkowników, zarządzania okresem przechowywania danych oraz reagowania na ewentualne naruszenia ochrony danych osobowych.

Należy podkreślić, że obowiązki wynikające z RODO nie są uzależnione wyłącznie od komercyjnego charakteru przedsięwzięcia lub liczby użytkowników. Ich zakres powinien odpowiadać rzeczywistemu sposobowi i skali przetwarzania informacji.

Kolejnym zagadnieniem jest odpowiedzialność za treści publikowane przez organizatorów. Informacje dotyczące wydarzeń powinny być rzetelne, aktualne oraz zgodne z obowiązującymi przepisami. Organizatorzy odpowiadają za prawidłowe przygotowanie i przeprowadzenie swoich imprez, natomiast operator platformy powinien określić zasady publikowania treści oraz reagowania na zgłoszenia dotyczące informacji niezgodnych z prawem lub regulaminem.

Zależnie od kwalifikacji prawnej świadczonej usługi zastosowanie mogą mieć również przepisy regulujące usługi pośrednie w internecie, w szczególności akt o usługach cyfrowych (*Digital Services Act*).

W kontekście wydarzeń motorsportowych szczególną uwagę należy zwrócić na bezpieczeństwo uczestników i osób postronnych. Platforma nie powinna służyć do promowania nielegalnych wyścigów ulicznych ani innych działań stwarzających zagrożenie dla bezpieczeństwa publicznego. Ograniczaniu tego ryzyka mają służyć mechanizmy weryfikacji organizatorów oraz moderacji publikowanych wydarzeń.

Istotna jest również ochrona praw autorskich i własności intelektualnej. Organizatorzy powinni publikować wyłącznie materiały, do których posiadają odpowiednie prawa, w tym fotografie, plakaty, regulaminy i elementy identyfikacji wizualnej.

W wymiarze etycznym ważne jest zapewnienie przejrzystych zasad działania systemu, równego traktowania użytkowników oraz oddzielenia informacji redakcyjnych od potencjalnych treści reklamowych. W przypadku wprowadzenia płatnej promocji wydarzeń takie materiały powinny być odpowiednio oznaczone.

Uwzględnienie opisanych uwarunkowań ma znaczenie nie tylko dla zgodności prawnej systemu, ale również dla budowania zaufania użytkowników i organizatorów, które stanowi jeden z podstawowych warunków długoterminowego rozwoju platformy RacePortal.

Rozdział 5

Analiza SWOT

Analiza SWOT (ang. *Strengths*, *Weaknesses*, *Opportunities*, *Threats*) służy w tej pracy do uporządkowanej oceny RacePortal jako produktu MVP: co stanowi przewagę rozwiązania względem rozproszonego rynku informacji o startach, gdzie leżą ograniczenia obecnej wersji, jakie kierunki rozwoju są realne oraz jakie czynniki zewnętrzne mogą utrudnić utrzymanie lub skalowanie systemu. Ocena odnosi się do **stanu zaimplementowanego** (web, API, mobile, wdrożenie produkcyjne), a nie wyłącznie do założeń koncepcyjnych.

5.1 Mocne strony (Strengths)

Mocne strony wynikają przede wszystkim z domknięcia spójnej podstawowej ścieżki dla wszystkich ról oraz z możliwości uruchomienia środowiska demonstracyjnego i produkcyjnego.

- Pełny przepływ ról: gość, kierowca (`USER`), organizator (`ORGANIZER`) i administrator (`ADMIN`), z kontrolą dostępu RBAC oraz uwierzytelnianiem JWT (w tym opcjonalne logowanie Google).
- Publiczny katalog wydarzeń z trzema widokami na wspólnych filtrach (lista / mapa / kalendarz), detal wydarzenia, wyznaczanie trasy (OSRM) oraz archiwum.
- Panel organizatora jako miejsce zarządzania wydarzeniem: zgłoszenia (statusy, klasy zawodów, eksport CSV), wyniki z publikacją, komunikacja masowa oraz KPI wyświetleń.
- Widok publiczny listy startowej i opublikowanych wyników na stronie wydarzenia.
- Dedykowany backoffice administratora (`bo.raceportal.pl`) z moderacją i eksportami, oddzielony od apexu prezentacyjnego.
- Konteneryzacja (Docker Compose), seed danych demo, powiadomienia e-mail (Mailbux) oraz wdrożenie produkcyjne (Oracle Always Free + Cloudflare Tunnel, TLS po stronie Cloudflare).

- Warstwa web jako PWA; wspólne typy API (`packages/api-types`) ograniczające niezgodności w danych wymienianych przez klienta webowego i mobilnego.
- Testy automatyczne (integracyjne API, E2E Playwright, unit mobile / api-types) oraz pomiary obciążenia k6 na VM produkcyjnej (lista wydarzeń API z MySQL wyraźnie poniżej wyniku health i nginx).

5.2 Słabe strony (Weaknesses)

Słabe strony to świadome lub nierozwiązane ograniczenia MVP, które obniżają kompletność produktową albo utrudniają odbiór jakościowy bez dodatkowych prac.

- Brak natywnej bramki płatności (PSP) — obowiązuje model „zapłać później" z dowodem przelewu (upload pliku / URL), a nie checkout online; upload jest w webie/API, nie w mobilce.
- Aplikacja mobilna Expo pozostaje wizytówką katalogu: bez logowania, garażu i pełnej równoważności funkcji z webem; brak publikacji w sklepach App Store / Google Play.
- Zależność od publicznych serwisów mapy i routingu (CARTO / OSRM; opcjonalnie Google Maps przy kluczu API) — awaria lub limity zewnętrzne pogarszają UX lokalizacji.
- Częściowe rozjazdy względem wewnętrznej Specyfikacji Formularzy (udokumentowane w rejestrze luk).
- Nie domknięto pierwotnego kryterium jakości „ok. 10 000 rekordów w katalogu" jako osobnego scenariusza obciążeniowego; pomiary k6 dotyczyły przepustowości RPS na istniejących danych, a nie zapełnienia katalogu do progu 10k.
- Ograniczenia hostingu Always Free (m.in. współdzielone zasoby CPU przy testach obciążeniowych) oraz dalszy hardening produkcyjny (np. pełne domknięcie kontroli dostępu do zasobów uploadów) pozostają w zakresie prac jakościowych, nie jako osobna funkcja produktowa MVP.

5.3 Szanse (Opportunities)

Szanse wynikają z rozpoznawalnego problemu rynkowego (rozproszone informacje o startach) oraz z już istniejącego rdzenia platformy, który można rozszerzać przyrostowo.

- Integracja PSP (np. Przelewy24 / Stripe) oraz fakturowanie / zwroty — domknięcie ścieżki płatnej bez ręcznej weryfikacji dowodu.

- Waitlista miejsc, watchlista wydarzeń oraz powiadomienia push / iCal — zwiększenie retencji kierowców i redukcja kanałów społecznościowych jako jedynego źródła aktualności.
- Rozszerzenie eksportów i odprawy (PDF listy startowej, kod QR zgłoszenia) na bazie już działającego CSV i publicznej listy startowej.
- Budowa społeczności wokół torów i organizatorów regionalnych (featured events, partnerzy, współorganizatorzy).
- Stopniowe zbliżanie mobile do weba (konto, garaż) albo świadome utrzymanie wizytówki z mocniejszym przekierowaniem do PWA / deep-link.

5.4 Zagrożenia (Threats)

Zagrożenia mają charakter zewnętrzny: konkurencja zachowań użytkowników, zależność od infrastruktury i dostawców oraz obowiązki prawne przy skali produkcyjnej.

- Konkurencja ze strony grup społecznościowych, komunikatorów oraz dedykowanych systemów zapisów organizatorów — niska bariera „zostania przy Facebooku / Discordzie".
- Zmiany limitów, cenników lub dostępności publicznych API mapy i routingu (oraz polityk OAuth Google).
- Wymogi RODO i odpowiedzialność za dane osobowe przy wzroście skali (retencja, DPIA, obsługa żądań podmiotów danych) — UI zawiera politykę prywatności i notę RODO, ale pełna zgodność procesowa wykracza poza demonstracyjne MVP.
- Ryzyko nadużyć powierzchni ataku typowej dla serwisów publicznych (spam kont / zgłoszeń, scraping katalogu, niewłaściwe użycie uploadów) mimo wdrożonych kontroli JWT/RBAC i auditów bezpieczeństwa.
- Uzależnienie od limitu darmowej / niskokosztowej infrastruktury chmurowej (Oracle Always Free, Cloudflare Tunnel) przy niespodziewanym wzroście ruchu.

5.5 Podsumowanie analizy SWOT

RacePortal ma silny rdzeń funkcjonalny i techniczny jak na MVP akademickie: spójne role, katalog, zapisy, narzędzia organizatora i administratora, testy oraz działające wdrożenie. Główne słabości dotyczą świadomie odłożonych elementów produktowych (PSP, pełne mobile) oraz domknięcia wybranych kryteriów jakościowych (scenariusz 10k rekordów). Szanse rozwoju

są naturalną kontynuacją istniejącego API i paneli; zagrożenia wymagają przede wszystkim dyscypliny bezpieczeństwa, prywatności i niezależności od pojedynczych usług zewnętrznych. Wnioski te są spójne z ograniczeniami i perspektywami opisanymi w rozdziale 18.

Rozdział 6

Planowanie projektu

Ten rozdział opisuje planowanie RacePortal: cele, zakres MVP, sposób pracy, zespół, harmonogram, infrastrukturę oraz ryzyka. W rozdziale oddzielono dokumenty formalne — Kartę projektu (KP), Dokument Założeń Wstępnych (DZW), Specyfikację Wymagań Systemowych (SWS) oraz historyczny Harmonogram przedmiotu PRO — od tego, co faktycznie powstało w kodzie, Docker Compose, plikach `docs/MVP.md`, chronologii `docs/changes.md`, decyzjach wdrożeniowych i pomiarach k6. Daty publikacji dokumentów i daty wpisów w chronologii to dwie różne warstwy: przebieg prac nie zastępuje oficjalnych kamieni milowych karty projektu, a karty formalne nie zastępują rzeczywistego stanu systemu.

6.1 Cele projektu

Głównym celem RacePortal, zgodnie z KP (data ustalenia tematu 01.10.2025), jest zbudowanie jednego miejsca z informacjami o wydarzeniach motorsportowych w Polsce — oficjalnych i amatorskich (m.in. drift, time attack, track day, KJS) — oraz narzędzi dla organizatorów do publikacji imprez i obsługi zapisów. Opiekunem (promotorem) projektu w KP i w pracy dyplomowej jest Marek Bednarczyk. KP rozwija cel główny w czterech skutkach: zebranie danych w jednym miejscu (odejście od chaosu grup społecznościowych i nieaktualnych stron), wsparcie mniejszych organizatorów, udogodnienia dla kierowców (profil, wirtualny garaż, szybkie zgłoszenia) oraz popularyzacja motorsportu przez kalendarz i mapę.

DZW — którego historia wersji w nagłówku sięga m.in. v1.4 z 26.05.2026 — doprecyzowuje cele systemu: katalog z listą i kalendarzem, filtrowanie (kategoria, tor, województwo, data, nazwa), szczegółowy podgląd wydarzeń, panel organizatora, konto kierowcy z garażem i zgłoszeniami oraz fundament pod dalszy rozwój. SWS formalizuje te cele jako podstawę implementacji, testów i oceny produktu akademickiego. W warstwie problemowej DZW wskazuje rozproszenie informacji o zawodach oraz brak jednego kanału dla organizatorów do publikacji i zarządzania wydarzeniem — to nadal punkt wyjścia przy planowaniu zakresu.

W pracy inżynierskiej celem praktycznym jest działające MVP demonstracyjne: katalog

i zapisy z rolami użytkowników, mapa i trasa, powiadomienia e-mail, ścieżka mobilna, konteneryzacja oraz dokumentacja i testy umożliwiające odbiór lub prezentację. Miary sukcesu z KP obejmują m.in. stabilne uruchomienie w Dockerze, cele wydajnościowe API dla żądań listujących, zgodność z OWASP ASVS Level 1 i RODO, pozytywne testy manualne kluczowych punktów zakresu oraz przygotowanie systemu na obsługę rzędu 1–5 tys. użytkowników miesięcznie (skala pilotowa z DZW/KP). Te miary wyznaczają kierunek planowania jakości; ich sprawdzenie w praktyce należy do rozdziałów realizacji i testowania, a nie zastępuje planu.

6.2 Zakres projektu

Zakres MVP jest spójny między KP, DZW i `docs/MVP.md` [9] jako lista piętnastu funkcji: kalendarz i lista wydarzeń; filtry oraz wyszukiwarka; szczegóły wydarzenia; panel administratora; konto kierowcy; garaż; system zgłoszeń; konto organizatora; narzędzia organizatora; baza danych wydarzeń; powiadomienia mailowe; mapa wydarzeń; integracja mapowa trasy (w KP i DZW: Google Maps API); archiwum; aplikacja mobilna (w KP: PWA / mobile-first).

Poza zakresem MVP — świadomie — pozostają m.in. integracje z kalendarzami zewnętrznymi, powiadomienia push poza mailami, automatyczna integracja z mediami społecznościowymi oraz live timing / pomiary czasów. KP wymienia je jako możliwości dalszej rozbudowy; w planowaniu nie traktuje się ich jako zobowiązań odbiorczych.

Warunki odbioru przyjęte w DZW i powtórzone w `docs/MVP.md` wymagają, by funkcje 1–11 były dostępne i przetestowane (manualnie oraz podstawowymi testami e2e), by UX zachowywał podejście mobile-first oraz by zakres był spójny z pitchiem i prezentacją. Jakość planowana obejmuje codzienne kopie zapasowe i kontrole stanu (health checks), Docker, paginację, indeksy i cache, czas odpowiedzi P95 nie większy niż 500–700 ms dla list do 50 rekordów, test obciążenia przy ok. 10 000 rekordów i 50 RPS, OWASP ASVS Level 1, RBAC, HTTPS z nagłówkami bezpieczeństwa, minimalizację danych (RODO) oraz przenośność środowiska Docker.

Według synchronizacji `docs/MVP.md` z 28.10.2026 funkcje 1–12 oraz 14–15 są domknięte funkcjonalnie (z lukami względem Specyfikacji Formularzy przy koncie kierowcy), a funkcja 13 — częściowo: trasa działa przez publiczne OSRM, a Google Maps pozostaje jedynie opcją konfiguracyjną. Ścieżkę mobilną zrealizowano jako wizytówkę Expo oraz webową PWA, a nie jako pełną wersję natywną ani publikację w sklepach — to świadoma adaptacja wobec sformułowania KP „PWA / Mobile-first”. Upload plików (obrazy JPG/PNG/WebP oraz PDF) istnieje w API (`/api/uploads/...`) i jest odnotowany w MVP jako domknięty 23.10.2026; nadal pozostają ograniczenia (m.in. brak pola zdjęcia w mobilce oraz niekompletna zgodność formularzy ze Specyfikacją). Elementy zrealizowane ponad pierwotną listę piętnastu funkcji (m.in. motywy, i18n, regulaminy PDF, tagi opon) odróżnia się w opisie od pierwotnego MVP,

żeby nie zacierać granicy zakresu.

Tabela 6.1: Zakres MVP — funkcje z KP/DZW oraz stan według `docs/MVP.md` (28.10.2026).

Nr	Funkcja	Stan realizacji
1	Kalendarz / lista wydarzeń	Zrealizowane: lista z API, paginacja, UI `/wydarzenia`
2	Filtry i wyszukiwarka	Zrealizowane: m.in. `q`, kategoria, miasto, filtr płatności
3	Szczegóły wydarzenia	Zrealizowane: detal, zapis, trasa, regulaminy PDF
4	Panel administratora	Zrealizowane: użytkownicy, eventy, wnioski org., błędy
5	Konto kierowcy	Częściowo: JWT, dashboard, weryfikacja e-mail; luki vs Spec
6	Garaż	Zrealizowane: CRUD pojazdów; zdjęcie przy dodaniu; tagi opon
7	System zgłoszeń	Zrealizowane: statusy, maile; orientacyjny termin na dowód wpłaty
8	Konto organizatora	Zrealizowane: wniosek o rolę + akceptacja admina
9	Narzędzia organizatora	Zrealizowane: tworzenie/edycja eventów, plakat, PDF
10	Baza danych wydarzeń	Zrealizowane: MySQL + JPA; seed torów PL
11	Powiadomienia mailowe	Zrealizowane: SMTP Mailbux
12	Mapa wydarzeń	Zrealizowane: Leaflet + OpenStreetMap
13	Trasa (plan: Google Maps)	Częściowo: OSRM domyślnie; Google Maps tylko opcjonalnie
14	Archiwum	Zrealizowane: archiwum + auto-archiwizacja
15	Aplikacja mobilna	Zrealizowane: Expo jako wizytówka + web PWA

6.3 Metodyka pracy

W historycznej dokumentacji planistycznej przedmiotu PRO (*Harmonogram* v02, folder „stare wersje”; sporządzenie 04–06.11.2025) przyjęto cykl przyrostowy z prototypami: każdy etap obejmuje planowanie, implementację i test wybranego modułu (np. kalendarz, panel administratora, mapy). Etapy nazwane w Harmonogramie to: analiza i planowanie, projektowanie, implementacja backendu, implementacja frontendu i integracje, testy oraz dokumentacja, a następnie prezentacja i ewaluacja MVP. Jako narzędzia planowania wymieniono m.in. GitHub Projects oraz — w części zasobowej — JIRA lub Monday.com; dokumentacja nie potwierdza jednak formalnych sprintów Scrum ani Kanban z datami. Harmonogram PRO traktuje się wyłącznie jako materiał historyczny przedmiotowy, nie jako obowiązującą kartę dyplomu; opiekun przedmiotu PRO w tym dokumencie (Marta Łabuda) nie zastępuje promotora pracy dyplomowej (Marek Bednarczyk).

Projekt rozwijano stopniowo we wspólnym repozytorium: wspólny kontrakt API, bieżąca dokumentacja zmian (układ „było → jest → dlaczego” w `docs/changes.md`), synchronizacja stanu względem MVP oraz decyzje infrastrukturalne (`docs/decyzje-wdrozeniowe.md`). Przegląd przed merge do `main` uzupełniają testy JUnit/MockMvc, Vitest i Playwright. Prototyp UI w Figma Make („Advanced UI/UX for RACEPORTAL”) był punktem startu frontendu; kolejne fale sprawdzeń UI i poprawek dokumentowano jako prace jakościowe, a nie jako for-

malny wykres Gantta. Zarządzanie jakością planowane w Harmonogramie PRO (code review przed `main`, testy manualne i automatyczne, inspekcja dokumentacji) pozostaje spójne z praktyką repozytorium, choć konkretne narzędzia się zmieniały (m.in. Postman z planu historycznego ustąpił miejsca testom JUnit/MockMvc i Playwright).

6.4 Organizacja zespołu

Zespół jest pięcioosobowy (specjalizacja cyberbezpieczeństwo, tryb stacjonarny — KP): Natalia Otrombke, Oliwier Kasprowicz, Michał Gutowski, Miłosz Parkitny i Wojciech Wronisz. Opiekunem projektu w KP oraz w pracy dyplomowej jest Marek Bednarczyk.

Współpraca opiera się o wspólne repozytorium GitHub, gałąź `main` oraz dokumentację w `docs/` i `Dokumentacja/`. Jakość planowana obejmuje przegląd przed merge, testy manualne i automatyczne oraz bieżącą aktualizację dokumentów porównawczych (MVP, zgodność ze Specyfikacją Formularzy). Historyczny Harmonogram PRO przewidywał m.in. cotygodniowe statusy na Discordzie oraz prezentacje postępów zgodne z planem zajęć — te zapisy to kontekst organizacyjny przedmiotu, a nie formalny regulamin dyplomu.

Za formalny podział ról w tym rozdziale przyjmuje się zapis z DZW: kierownik projektu — Wojciech Wronisz; frontend — Natalia Otrombke i Oliwier Kasprowicz; backend — Miłosz Parkitny; fullstack / marketing — Michał Gutowski. Późniejsza praktyka opisana w `README.md` repozytorium różni się w szczegółach (kierownik: Michał Gutowski; backend: Michał Gutowski i Miłosz Parkitny; fullstack, testy i bezpieczeństwo: Wojciech Wronisz) — odnotowuje się ją jako zmianę organizacyjną w czasie, bez przypisywania tu autorstwa poszczególnych funkcji. Opis wkładu merytorycznego pozostaje w osobnym rozdziale pracy.

Tabela 6.2: Podział ról według DZW (formalny dla tego rozdziału) oraz opis w `README.md`.

Osoba	**Rola w DZW**	**Opis w** `README.md`
Wojciech Wronisz	Kierownik projektu	Fullstack, testy i bezpieczeństwo
Natalia Otrombke	Frontend	Frontend
Oliwier Kasprowicz	Frontend	Frontend
Miłosz Parkitny	Backend	Backend
Michał Gutowski	Fullstack / marketing	Kierownik; backend
Marek Bednarczyk	Opiekun (promotor)	Opiekun projektu

6.5 Harmonogram pracy

W KP data 31.01.2026 figuruje jako ograniczenie czasowe karty projektu. W tej pracy datę tę traktuje się jako termin dokumentacyjny karty (ograniczenie planistyczne KP), a **nie** jako kalendarzowy termin domknięcia gotowego i przetestowanego systemu. Przebieg implementacji

— udokumentowany w `docs/changes.md` — wykracza poza tę datę karty i obejmuje m.in. wdrożenie produkcyjne oraz domknięcia funkcji w październiku 2026; nie wprowadza się tu konkurencyjnej „daty ukończenia" spoza potwierdzonej interpretacji karty.

Planowanie etapowe z Harmonogramu PRO (analiza, projekt, backend, frontend z integracjami, testy, prezentacja MVP) stanowiło plan prac na zajęciach przedmiotu, a nie zamknięty formalny Gantt dyplomu. W Harmonogramie PRO podano również okno zajęć (ok. 15 tygodni od 6.10.2025 do 19.01.2026) oraz szacunek nakładu — te zapisy pozostają historią przedmiotową i nie są tu przepisywane jako obowiązujący harmonogram dyplomu z wymyślonymi datami kamieni milowych.

Przebieg prac — w odróżnieniu od kamieni milowych karty — dokumentuje plik `docs/changes.md`. Chronologia obejmuje m.in. przeniesienie prototypu z Figma Make do lokalnego monorepo, szkielet projektu Vite i Dockera, migrację API na Spring Boot z MySQL, budowę aplikacji mobilnej Expo jako wizytówki, iteracje testów i formularzy, a w październiku 2026 przenosiny produkcyjne na Oracle Always Free z Cloudflare Tunnel, SMTP Mailbux i webową PWA, następnie domknięcie uploadu plików oraz pomiary k6 na VM. Daty wpisów w `changes.md` służą jako punkty odniesienia przebiegu; nie zastępują oficjalnego terminu karty KP i nie tworzą formalnego wykresu Gantta.

Tabela 6.3: Etapy prac — plan z Harmonogramu PRO wobec charakteru przebiegu w repozytorium.

lp.	Etap (plan PRO)	Produkty planowane	Charakter w przebiegu
1	Analiza i planowanie	DZW, plan projektu	KP/DZW/SWS jako ramy; aktualizacje DZW w historii wersji
2	Projektowanie	SWS, makiety UI	Prototyp Figma Make; model danych i API w implementacji
3	Implementacja backendu	Moduły REST, baza	Spring Boot + MySQL (zamiast historycznego Postgres z Harmonogramu)
4	Frontend i integracje	Widoki React, mapy	Vite/React; Leaflet/OSRM; Expo wizytówka; PWA
5	Testy i dokumentacja	Raporty, README	JUnit/MockMvc, Vitest, Playwright; `docs/`
6	Prezentacja MVP	Demo odbiorcze	Compose lokalnie + prod Oracle/Tunnel; scenariusz prezentacji

6.6 Infrastruktura i środowisko pracy

Wizja konstrukcyjna DZW zakładała frontend React, backend Java Spring Boot (REST), bazę MySQL, integracje mapowe, GitHub, Docker, Maven oraz GitHub Projects. W realizacji lo-

kalnej i produkcyjnej źródłem prawdy uruchomieniowej jest Docker Compose: kontenery `web` (nginx z buildem Vite/React), `api` (Spring Boot), `mysql` (MySQL 8.4) oraz `backup` (cykliczny `mysqldump`). Aplikacja mobilna Expo działa poza Compose. Maile systemowe idą SMTP przez Mailbux (hasło wyłącznie w `.env`).

Wdrożenie produkcyjne zrealizowano na maszynie Oracle Always Free (ARM Ampere) z pełnym Compose. Publiczny dostęp zapewnia Cloudflare Tunnel do nginx na porcie `:8081`; domena `raceportal.pl` korzysta z TLS, HSTS i DNS po stronie Cloudflare. Role warstw są rozdzielone: frontend (statyczny SPA za nginx z proxy `/api`), API (Spring Boot) oraz baza (MySQL). Cloudflare pełni rolę tunelu i bramy TLS/DNS, a nie hostingu frontu na Pages — wcześniejsze warianty GCP/Pages pozostają wyłącznie historią decyzji, opisaną w `docs/decyzje-wdrozeniowe.md`.

Stack mapowy w kodzie to Leaflet z kafelkami OpenStreetMap po stronie weba oraz wyznaczanie trasy przez publiczne OSRM (proxy `/api/maps/route`). Jest to świadoma realizacja względem planu KP/DZW opartego o Google Maps API; klucz Google Maps może figurować w konfiguracji, lecz aktywna ścieżka routingu w serwisie map korzysta z OSRM.

Środowisko pracy obejmuje Docker Desktop, Node.js (web i mobile), JDK/Maven (API w kontenerze), Expo z symulatorem lub emulatorem, Playwright, GitHub oraz `cloudflared` na VM Oracle. Upload plików jest częścią API (katalog wolumenowy `UPLOAD_DIR` w Compose), z ograniczeniami odnotowanymi w MVP i Specyfikacji Formularzy — nie zastępuje pełnej zgodności ze wszystkimi polami formularzy ani równoważności funkcji w aplikacji mobilnej.

6.7 Analiza ryzyka projektowego

6.7.1 Analiza zagrożeń

Poniżej zebrano ryzyka istotne dla planowania i odbioru MVP, wraz ze sposobami ograniczenia ryzyka wynikającymi z dokumentacji i stanu systemu. Część zagrożeń planowano już w historycznym Harmonogramie PRO (opóźnienia, spójność kodu, integracja map); poniżej ujęto je łącznie z ryzykami ujawnionymi w przebiegu implementacji i porównaniu plan–stan.

W zakresie wydajności empiryczne pomiary Grafana k6 na Oracle VM (25.10.2026; raport `docs/testy/wyniki/k6-vm-rps.md` [10]) dotyczyły m.in. paginowanej listy wydarzeń (`GET /api/events`). Podczas testów uzyskano około 150–200 żądań na sekundę — wyraźnie mniej niż dla lekkiego health API i statycznego frontu (ok. 2000 żądań na sekundę). Raport porównuje te warstwy i wskazuje, że ograniczenie dotyczy endpointu listy wydarzeń (API z MySQL), a nie nginx ani samego health. Pomiar na współdzielonej VM (k6 lokalnie ze stackiem) zaniża wynik względem zewnętrznego generatora i nie zastępuje formalnego testu 10 000 rekordów / 50 RPS; szczegółowa analiza należy do rozdziału testowania.

Planowanie RacePortal łączy formalne ramy KP/DZW z iteracyjnym przebiegiem imple-

Tabela 6.4: Analiza ryzyka RacePortal — wybrane zagrożenia i sposoby ich ograniczenia.

Ryzyko	**Skutek / objaw**	**Sposób ograniczenia ryzyka**
Niespójność Specyfikacji Formularzy względem implementacji	Luki odbiorcze (m.in. wymagalność danych osobowych)	Rejestr luk w `docs/spec-conformity.md`; rozróżnienie „działa funkcjonalnie” / „pełna zgodność ze Spec”
Awaria lub limity zewnętrznych serwisów map i routingu	Brak mapy lub trasy	Leaflet/OSM bez klucza; OSRM z komunikatami błędu; możliwa wymiana dostawcy
Niezgodności w danych wymienianych przez aplikacje (web $\leftrightarrow$ mobile)	Błędne typy lub adresy API	Wspólny pakiet `packages/api-types`
Rozrost zakresu ponad MVP	Opóźnienie odbioru MVP	Świadome odłożenie PSP, push i store; elementy ponad MVP nazywane osobno
Ograniczenia zasobowe i czasowe zespołu	Niewykonanie planu przy budżecie zerowym	Oracle Always Free; jeden origin przez Tunnel; termin karty KP 31.01.2026 jako ograniczenie planistyczne (nie termin domknięcia systemu)
Niekompletność bazy wydarzeń na starcie	Słaba wartość katalogu bez społeczności	Dane demo na start, katalog torów PL, podstawowe ścieżki ról; kompletność rynkowa poza czystym zakresem kodu
Brak formalnego testu 10k/50 RPS	Niespełnienie pierwotnego kryterium odbiorczego KP/DZW	Paginacja, indeksy, cache; pomiary k6 na VM (`docs/testy/wyniki/k6-vm-rps.md` [10]; rozdz. 14)
Zależność od dostarczalności poczty	Rejestracja / statusy „nie działają” dla użytkownika	SMTP Mailbux zamiast Mailpita; diagnostyka SMTP w health (warstwa admin)
Utrata spójności kodu / regresje integracji	Błędy na styku warstw	Code review przed `main`; testy automatyczne API/web/mobile

mentacji. Zakres MVP pozostaje czytelny dzięki porównaniu „miało być / jest” w dokumencie `MVP.md`, a infrastruktura produkcyjna — dzięki rozdzieleniu ról web, API i bazy oraz świadomym decyzjom o Oracle i Cloudflare Tunnel. Ryzyka utrzymuje się w rejestrze jakościowym (Spec, wydajność, zależności zewnętrzne), ze sposobami ograniczenia ryzyka osadzonymi w architekturze i dokumentacji porównawczej.

Rozdział 7

Analiza wymagań

7.1 Źródła i sposób pozyskania wymagań

Wymagania dla platformy RacePortal zostały zebrane i sformułowane w oparciu o wielotorowy proces analityczny, ukierunkowany na rozwiązanie realnych problemów rynku motorsportowego. Głównymi źródłami pozyskania wymagań były:

- **Dokument Założeń Wstępnych (DZW) oraz Specyfikacja Wymagań Systemowych (SWS)**, które zdefiniowały biznesowy cel platformy i zakres funkcjonalności dla wersji MVP.
- **Obserwacja polskiego środowiska wydarzeń motorsportowych**, charakteryzującego się dotychczas znacznym rozproszeniem informacji, brakiem spójnych standardów zapisów oraz chaosem komunikacyjnym w mediach społecznościowych.
- **Bezpośrednie rozmowy z kierowcami oraz organizatorami imprez**, które pozwoliły zidentyfikować kluczowe problemy operacyjne, takie jak uciążliwa i powtarzalna weryfikacja techniczna pojazdów czy trudności w efektywnym zarządzaniu listą startową.
- **Osobiste doświadczenia od strony aktywnego kierowcy**, zdobyte przez członka zespołu, Michała Gutowskiego, które dostarczyły praktycznej wiedzy na temat problemów uczestników i oczekiwań wobec systemów rejestracji na zawody.

7.2 Wymagania ogólne i dziedzinowe

KARTA WYMAGANIA			
Identyfikator:	W01	Priorytet:	Wysoki
Nazwa:	Zintegrowany katalog wydarzeń motorsportowych		
Opis:	System musi zbierać dane o imprezach w jednym miejscu, udostępniając je użytkownikom w sposób przejrzysty poprzez klasyczną listę, czytelny kalendarz oraz interaktywną mapę.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	Brak		

Tabela 7.1: Zintegrowany katalog wydarzeń motorsportowych

KARTA WYMAGANIA			
Identyfikator:	W02	Priorytet:	Wysoki
Nazwa:	Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów		
Opis:	Platforma ma automatyzować proces zapisów, weryfikując systemowo czy pojazd zawodnika wybrany z jego wirtualnego garażu spełnia regulaminowe wymogi danej kategorii wydarzenia. Odciąża to organizatorów i niweluje błędy ludzkie.		
Udziałowiec:	Kierowca, Organizator		
Wymagania powiązane:	F04, F05		

Tabela 7.2: Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów

KARTA WYMAGANIA			
Identyfikator:	W03	Priorytet:	Wysoki
Nazwa:	Przejrzysty podział ról w systemie		
Opis:	System opiera się na wyraźnym podziale uprawnień na: użytkowników (szukających startów), organizatorów (zarządzających swoimi imprezami) oraz administratorów (dbających o jakość, wsparcie i bezpieczeństwo platformy).		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	SEC02		

Tabela 7.3: Przejrzysty podział ról w systemie

7.3 Wymagania funkcjonalne

KARTA WYMAGANIA			
Identyfikator:	F01	Priorytet:	Wysoki
Nazwa:	Publiczny katalog i filtrowanie		
Opis:	Każdy użytkownik platformy ma dostęp do bazy wydarzeń z możliwością filtrowania po kategorii zawodów, dacie oraz województwie.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	W01		

Tabela 7.4: Publiczny katalog i filtrowanie

KARTA WYMAGANIA			
Identyfikator:	F02	Priorytet:	Wysoki
Nazwa:	Szczegóły wydarzenia i wyznaczanie trasy		
Opis:	Platforma udostępnia szczegółowe informacje o imprezie, w tym mapę ułatwiającą zawodnikom wyznaczenie trasy dojazdu na obiekt.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	W01		

Tabela 7.5: Szczegóły wydarzenia i wyznaczanie trasy

KARTA WYMAGANIA			
Identyfikator:	F03	Priorytet:	Wysoki
Nazwa:	Zarządzanie profilem i kontem		
Opis:	Zapewnienie prostego procesu zakładania konta, logowania oraz edycji profilu, umożliwiającego kierowcom sprawne wprowadzanie niezbędnych danych kontaktowych i licencyjnych.		
Udziałowiec:	Kierowca, Organizator		
Wymagania powiązane:	Brak		

Tabela 7.6: Zarządzanie profilem i kontem

KARTA WYMAGANIA			
Identyfikator:	F04	Priorytet:	Wysoki
Nazwa:	Wirtualny Garaż Kierowcy		
Opis:	Cyfrowa ewidencja pojazdów przypisana do profilu. Pozwala na jednorazowe wprowadzenie danych technicznych auta, co ułatwia i przyspiesza kolejne zapisy na różne zawody.		
Udziałowiec:	Kierowca		
Wymagania powiązane:	W02		

Tabela 7.7: Wirtualny Garaż Kierowcy

KARTA WYMAGANIA			
Identyfikator:	F05	Priorytet:	Wysoki
Nazwa:	Szybkie aplikowanie na wydarzenia		
Opis:	Kierowca może zgłosić chęć udziału w wybranych zawodach poprzez wskazanie odpowiedniego pojazdu z garażu. Poprawne zgłoszenie natychmiast trafia do organizatora ze statusem oczekującym na akceptację.		
Udziałowiec:	Kierowca		
Wymagania powiązane:	W02, F04		

Tabela 7.8: Szybkie aplikowanie na wydarzenia

KARTA WYMAGANIA			
Identyfikator:	F06	Priorytet:	Średni
Nazwa:	Skuteczny system powiadomień		
Opis:	Wdrożenie powiadomień systemowych wewnątrz aplikacji oraz wiadomości e-mail, które na bieżąco informują użytkowników o istotnych zmianach, takich jak akceptacja lub odrzucenie wniosku o start.		
Udziałowiec:	Zalogowany Użytkownik		
Wymagania powiązane:	Brak		

Tabela 7.9: Skuteczny system powiadomień

KARTA WYMAGANIA			
Identyfikator:	F07	Priorytet:	Wysoki
Nazwa:	Panel Zarządzania dla Organizatora		
Opis:	Panel dla organizatorów imprez, pozwalający na publikację nowych wydarzeń oraz zarządzanie listą zgłoszonych kierowców i ich opłatami.		
Udziałowiec:	Organizator		
Wymagania powiązane:	W03		

Tabela 7.10: Panel Zarządzania dla Organizatora

KARTA WYMAGANIA			
Identyfikator:	F08	Priorytet:	Wysoki
Nazwa:	Panel Administratora i wsparcie użytkowników		
Opis:	Centralny panel zarządzania, umożliwiający weryfikację publikowanych treści, nadawanie uprawnień nowym organizatorom oraz obsługę zgłoszeń i problemów zgłaszanych przez użytkowników.		
Udziałowiec:	Administrator		
Wymagania powiązane:	W03		

Tabela 7.11: Panel Administratora i wsparcie użytkowników

KARTA WYMAGANIA			
Identyfikator:	F09	Priorytet:	Średni
Nazwa:	Mobilna aplikacja wizytówkowa		
Opis:	Aplikacja na smartfony do szybkiego przeglądania katalogu imprez, z możliwością przekierowania do głównej platformy webowej w celu dokończenia zapisu.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	Brak		

Tabela 7.12: Mobilna aplikacja wizytówkowa

7.4 Wymagania niefunkcjonalne

KARTA WYMAGANIA			
Identyfikator:	NF01	Priorytet:	Wysoki
Nazwa:	Wydajność i responsywność platformy		
Opis:	System musi działać płynnie, z krótkim czasem ładowania katalogu oraz stabilną obsługą przy wzmożonym ruchu.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.13: Wydajność i responsywność platformy

KARTA WYMAGANIA			
Identyfikator:	NF02	Priorytet:	Wysoki
Nazwa:	Dostępność wieloplatformowa (RWD)		
Opis:	Interfejs aplikacji musi dostosowywać się do rozmiaru ekranu użytkownika, tak aby był czytelny na smartfonach, tabletach oraz komputerach stacjonarnych.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.14: Dostępność wieloplatformowa (RWD)

KARTA WYMAGANIA			
Identyfikator:	NF03	Priorytet:	Średni
Nazwa:	Niezawodność i łatwość wdrażania		
Opis:	Platforma powinna działać stabilnie oraz pozwalać na wdrażanie nowych funkcji bez długotrwałych przerw w dostępności usług dla kierowców.		
Udziałowiec:	Administrator		
Wymagania powiązane:	Brak		

Tabela 7.15: Niezawodność i łatwość wdrażania

7.5 Wymagania bezpieczeństwa

KARTA WYMAGANIA			
Identyfikator:	SEC01	Priorytet:	Wysoki
Nazwa:	Bezpieczeństwo danych osobowych i kont		
Opis:	Ochrona kont użytkowników, bezpieczne uwierzytelnianie oraz haszowanie haseł jako elementy ochrony danych osobowych; pełna zgodność procesowa z RODO wykracza poza samą warstwę techniczną.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.16: Bezpieczeństwo danych osobowych i kont

KARTA WYMAGANIA			
Identyfikator:	SEC02	Priorytet:	Wysoki
Nazwa:	Ścisła kontrola dostępu do zasobów		
Opis:	Gwarancja systemowa, że kierowca widzi i zarządza wyłącznie własnymi pojazdami oraz zgłoszeniami, a organizator posiada uprawnienia do wglądu i edycji tylko swoich wydarzeń.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	W03		

Tabela 7.17: Ścisła kontrola dostępu do zasobów

KARTA WYMAGANIA			
Identyfikator:	SEC03	Priorytet:	Średni
Nazwa:	Odporność na nadużycia platformy		
Opis:	Implementacja mechanizmów chroniących platformę przed zautomatyzowanym spamem, atakami masowymi oraz próbami podszywania się pod oficjalną obsługę platformy w celu wyłudzenia informacji.		
Udziałowiec:	Administrator		
Wymagania powiązane:	Brak		

Tabela 7.18: Odporność na nadużycia platformy

7.6 Wymagania interfejsu użytkownika

KARTA WYMAGANIA			
Identyfikator:	UI01	Priorytet:	Wysoki
Nazwa:	Nowoczesny design branżowy		
Opis:	Spójny interfejs oparty na ciemnym motywie z kontrastującymi akcentami, dopasowany do estetyki branży motorsportowej.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.19: Nowoczesny design branżowy

KARTA WYMAGANIA			
Identyfikator:	UI02	Priorytet:	Wysoki
Nazwa:	Intuicyjność i czytelność procesów		
Opis:	Jasne komunikaty dla użytkownika, widoczne wskaźniki ładowania oraz zrozumiałe formularze ograniczające pomyłki podczas rejestracji na zawody.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.20: Intuicyjność i czytelność procesów

7.7 Diagramy UML

7.7.1 Diagram przypadków użycia

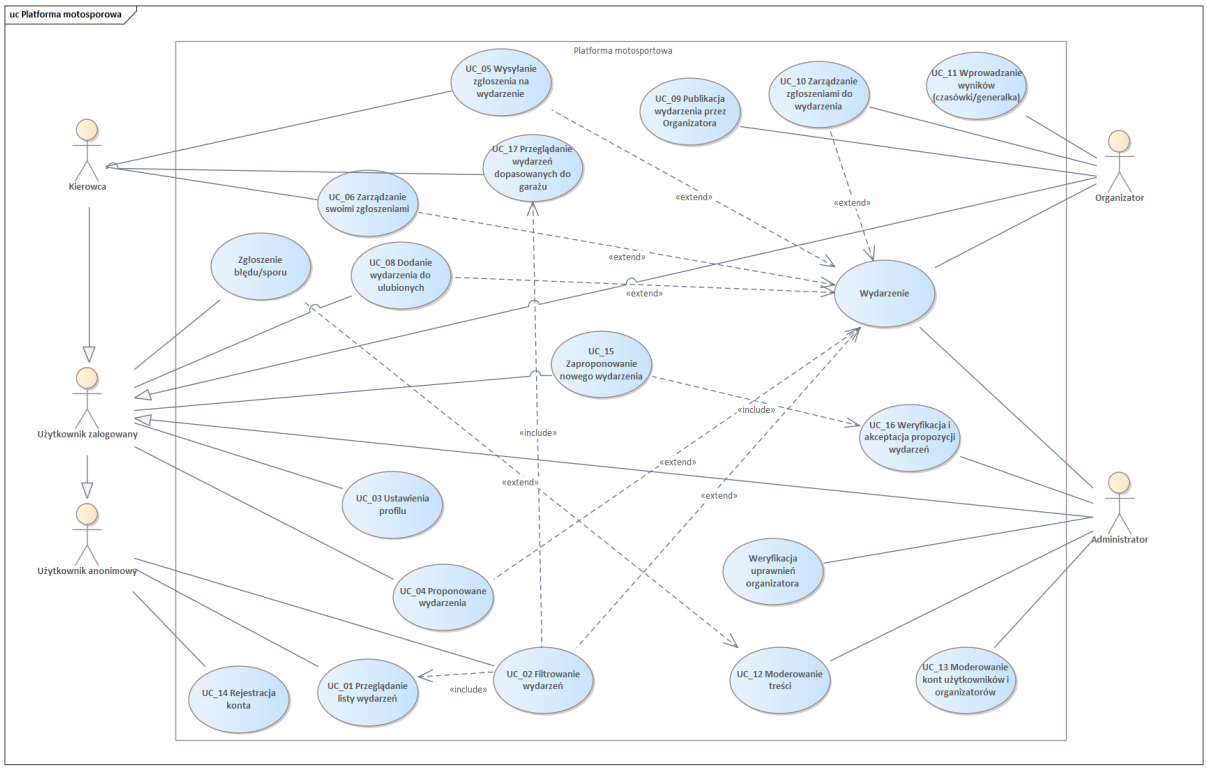
Wczesny diagram przypadków użycia (materiały projektowe) przedstawia rys. 7.1.

7.7.2 Diagram sekwencji: rejestracja i logowanie

7.7.3 Diagram sekwencji: zapis na wydarzenie

7.7.4 Diagram sekwencji: zarządzanie garażem

Uwaga: diagram klas / model danych przeniesiono do projektu systemu — podrozdział 8.4.1.



Rysunek 7.1: Diagram przypadków użycia — Platforma motorsportowa (wersja koncepcyjna).

Rozdział 8

Projekt systemu

Rozdział przedstawia projekt architektury RacePortal: założenia warstwowe, podział logiczny, wdrożenie fizyczne, model danych, kontrakt REST oraz projekt aplikacji mobilnej i mechanizmów bezpieczeństwa. Projekt warstwy webowej (w tym prototyp Figma) omówiono osobno w podrozdziale 8.6. Celem jest pokazanie, *jak* system został zaprojektowany przed opisem realizacji i szczegółów implementacji.

8.1 Założenia architektoniczne

Przyjęto klasyczną architekturę trójwarstwową z jednym kontraktem API dla klientów:

- **warstwa prezentacji** — aplikacja webowa (React/Vite, serwowana statycznie) oraz aplikacja mobilna Expo (wizytówka katalogu);
- **warstwa aplikacji** — API REST (Spring Boot) z logiką domenową, walidacją i kontrolą dostępu;
- **warstwa danych** — relacyjna baza MySQL (JPA / Spring Data).

Kluczowe założenia projektowe:

- jeden origin produkcyjny (`https://raceportal.pl`): nginx serwuje SPA i przekazuje `/api` do backendu — bez osobnego hostingu frontu z osobnym URL API w buildzie;
- wspólny kontrakt JSON oraz pakiet typów `packages/api-types` ograniczający niezgodności w danych wymienianych przez aplikacje webową i mobilną;
- konteneryzacja Docker Compose jako referencyjne środowisko uruchomieniowe (lokalnie i na VM);
- świadomy podział odpowiedzialności: pełny produkt na webie (w tym PWA), aplikacja mobilna jako kanał przeglądania katalogu z CTA zapisu na web;

- bezpieczeństwo bazowe wbudowane w projekt API (JWT, RBAC, walidacja wejścia), a TLS na brzegu sieci (Cloudflare).

8.2 Architektura logiczna systemu

Logicznie system dzieli się na moduły domenowe współpracujące przez warstwę serwisów backendu (bez bezpośredniego dostępu klientów do bazy):

- **użytkownicy i auth** — rejestracja, weryfikacja e-mail, logowanie hasłem / Google, odświeżanie sesji, profil;
- **wydarzenia** — katalog, detal, statusy cyklu życia, archiwizacja, licznik wyświetleń;
- **zgłoszenia** — flow statusów, dowód płatności, przypisanie klasy;
- **garaż** — pojazdy użytkownika i dopasowanie do wymagań wydarzenia (`carMatch`);
- **organizator** — workspace wydarzenia: klasy, wyniki, komunikacja, eksporty;
- **administracja** — moderacja wydarzeń i ról oraz backoffice na osobnym hoście;
- **mapy / trasa** — markery katalogu i wyznaczanie dojazdu (OSRM);
- **powiadomienia i poczta** — powiadomienia in-app oraz maile transakcyjne (SMTP);
- **pliki** — upload obrazów i PDF (m.in. dowody wpłaty, regulaminy) z odczytem przez API.

Podział odpowiada granicom uprawnień RBAC: gość korzysta z odczytu publicznego, kierowca ze zgłoszeń i garażu, organizator z własnych wydarzeń, administrator z moderacji całej platformy.

8.3 Architektura fizyczna i wdrożeniowa

Środowisko uruchomieniowe oparto o Docker Compose. Lokalnie (i analogicznie na VM) pracują kontenery:

- `web` — nginx ze zbudowanym SPA oraz reverse proxy `/api`;
- `api` — Spring Boot (JVM);
- `mysql` — baza trwała (volume);
- `backup` — periodyczne kopie zapasowe bazy.

Aplikacja mobilna Expo działa poza Compose (Expo Go / emulator). Poczta wychodząca korzysta z zewnętrznego SMTP Mailbux (poza Compose); dane dostępowe SMTP są trzymane w `.env`, poza repozytorium.

Wdrożenie produkcyjne: maszyna Oracle Always Free (ARM) z tym samym stackiem Compose. Publiczny dostęp zapewnia **Cloudflare Tunnel** [11] (`cloudflared`) do nginx [12] na porcie `:8081`. Apex `https://raceportal.pl` oraz host backoffice są w koncie Cloudflare — TLS/HSTS i DNS po stronie Cloudflare, bez hostowania frontu na Cloudflare Pages. Dzięki temu deploy sprowadza się do aktualizacji kodu na VM i przebudowy kontenerów, a klienci wołają API względną ścieżką `/api`.

8.4 Projekt bazy danych

Model danych zaprojektowano wokół wydarzenia motorsportowego i procesu zapisu. Główne encje implementacyjne obejmują m.in.:

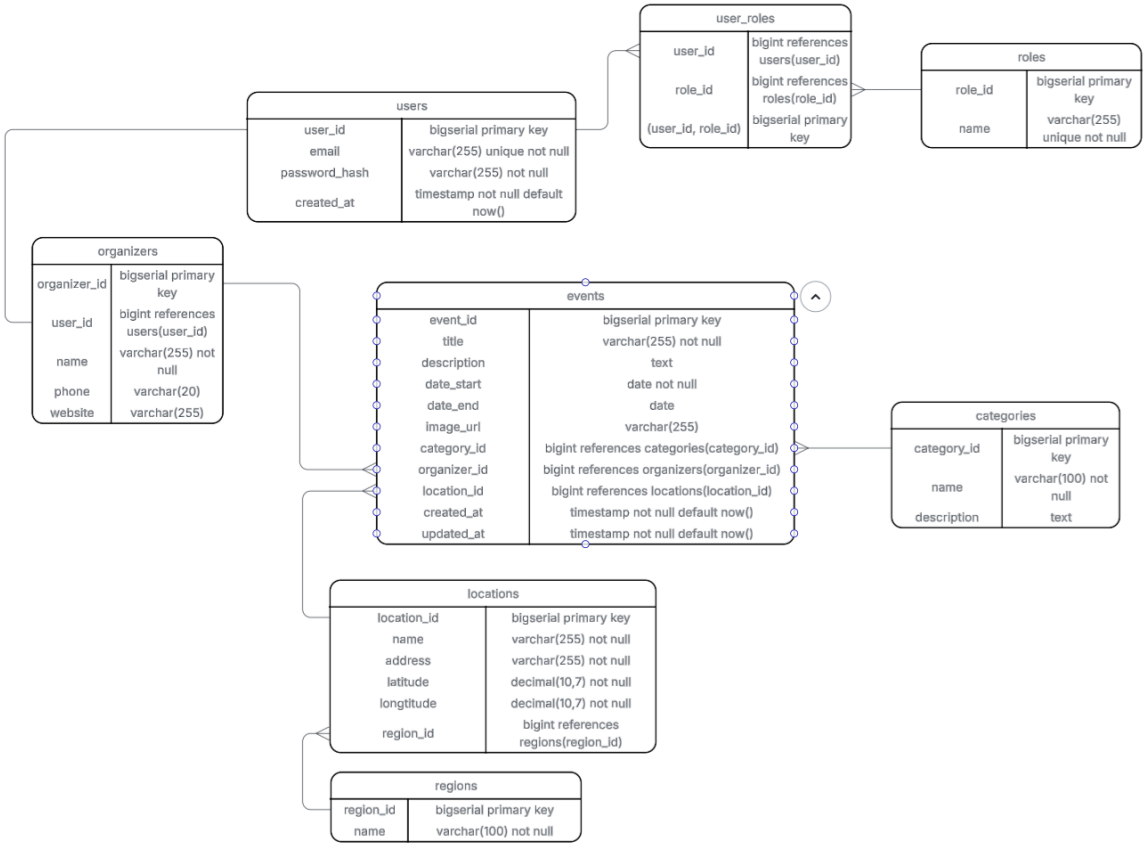
- użytkownika (rola jako enum: `USER` / `ORGANIZER` / `ADMIN`);
- pojazd w garażu;
- wydarzenie (lokalizacja i współrzędne, status cyklu życia, opłaty, regulaminy, flaga publikacji wyników, licznik wyświetleń);
- zgłoszenie (status, dowód płatności, klasa zawodów);
- tor z katalogu PL (seed przy starcie API; odczyt listy torów przez API);
- klasy zawodów, wyniki i reguły punktacji powiązane z wydarzeniem;
- wniosek o rolę organizatora, powiadomienia, refresh tokeny oraz zgłoszenia błędów.

Wydarzenie przechodzi przez kolejne stany: od szkicu i moderacji, przez publikację w katalogu, aż do archiwum albo anulowania. Zgłoszenie kierowcy ma osobne stany: najpierw czeka na decyzję organizatora (`PENDING`), potem może zostać wstępnie przyjęte (`ACCEPTED`) albo od razu potwierdzone (`CONFIRMED`); anulowanie oznacza `CANCELED`. Przy wydarzeniach płatnych po `ACCEPTED` kierowca dokłada dowód przelewu (model „zapłać później”), a organizator dopiero wtedy potwierdza miejsce. Listy torów w formularzach web i mobile korzystają ze wspólnego zestawu nazw, żeby obie aplikacje pokazywały te same lokalizacje.

8.4.1 Diagram klas

Rysunek 8.1 przedstawia wczesny model bazy danych (ERD) stanowiący punkt wyjścia projektu klas / tabel domenowych. W implementacji Spring Data JPA model został uproszczony

i dostosowany do MVP (m.in. role jako enum użytkownika, lokalizacja osadzona w wydarzeniu). Kolejne encje operacyjne (klasy, wyniki, uploady jako URL-e w polach domenowych) rozbudowano iteracyjnie bez zmiany ogólnego wzorca: jedna baza MySQL, relacje JPA, statusy jako enumy.



Rysunek 8.1: Diagram klas / model danych (ERD) — wersja koncepcyjna projektu.

8.5 Projekt interfejsu REST API

Interfejs API zaprojektowano w stylu REST nad protokołem HTTP, z wymianą danych w formacie JSON. Prefiks zasobów to `/api/...`. Kontrakt odpowiedzi jest wspólny dla klienta webowego i mobilnego; typy współdzielone utrzymywane są w pakiecie `packages/api-types`, co ogranicza rozjazd interpretacji pól pomiędzy warstwami.

Zasoby podzielono według odpowiedzialności domenowych. **Uwierzytelnianie i profil** (`/api/auth`) obejmują rejestrację, weryfikację e-mail, logowanie, odświeżanie i unieważnianie sesji, opcjonalne logowanie Google oraz operacje na profilu (`/me`). **Katalog wydarzeń** (`/api/events`) udostępnia listę z paginacją i filtrami, markery mapy, metadane kategorii, szczegóły wydarzenia oraz operacje tworzenia, edycji, anulowania i usuwania dla uprawnionych ról. Uzupełniają je zasoby klas startowych, listy startowej, wyników oraz komunikacji organizatora do uczestni-

ków konkretnego wydarzenia.

Zgłoszenia (`/api/registrations`) umożliwiają utworzenie zgłoszenia, podgląd własnych zapisów, listę zgłoszeń wydarzenia, zmianę statusu, anulowanie, dołączenie dowodu płatności oraz przypisanie klasy. **Garaż** (`/api/garage`) obsługuje CRUD pojazdów użytkownika. **Organizator** (`/api/organizer`) obejmuje wniosek o rolę oraz listę zarządzanych wydarzeń, a **administrator** (`/api/admin`) — statystyki, użytkowników, moderację wydarzeń i rozstrzyganie wniosków organizatorskich.

Pozostałe zasoby wspierające to: katalog torów (`/api/tracks`), wyznaczanie trasy (`/api/maps/route`), upload plików (`/api/uploads`), powiadomienia in-app (`/api/notifications`), publiczne statystyki (`/api/stats`), zgłoszenia błędów (`/api/bug-reports`) oraz monitoring (`/api/health`).

Z punktu widzenia dostępu projekt rozróżnia zasoby publiczne (m.in. odczyt wydarzeń i torów, health, trasa, rejestracja/logowanie) oraz zasoby wymagające uwierzytelnienia i odpowiedniej roli. Błędy zwracane są w ujednoliconym formacie JSON (walidacja pól, kody HTTP 4xx/5xx), co upraszcza obsługę po stronie klientów. Szczegóły implementacji endpointów i przepływów opisano w rozdziale 10.

8.6 Projekt warstwy webowej

Warstwa webowa RacePortal [1] to pełny klient transakcyjny systemu: obsługuje katalog wydarzeń, konta użytkowników, wirtualny garaż, zgłoszenia udziału oraz panele organizatora i administratora. Stack aplikacji to **Vite 6** [2] z **React 18** i **TypeScript** [3], stylowanie **Tailwind CSS v4** [13] oraz zestaw komponentów zbliżonych do **shadcn/ui** (Radix UI). Routing realizuje **React Router v7** (`createBrowserRouter`); komunikacja z backendem — moduł komunikacji z API w `web/src/app/lib/api.ts`, z bazą URL z `VITE_API_URL` albo ścieżkami względnymi `/api/...` (proxy Vite w dev, proxy nginx w Dockerze). Sesja opiera się o **httpOnly cookies** (`rp_at` / `rp_rt`) z `credentials: "include"`; starszy JWT w `localStorage` jest usuwany przy starcie. Szczegóły implementacji frontendu są rozwinięte w rozdziale 11; poniżej opisano projekt warstwy i kluczowe decyzje.

Architektura frontendu

Korzeń aplikacji (`App.tsx`) owija drzewo tras w `AuthProvider`, wczytuje ustawienia UI (akcent / język) oraz renderuje `RouterProvider`. Strona główna ładuje się od razu (LCP); pozostałe strony — przez `React.lazy`, z obsługą błędu po aktualizacji (jednorazowe odświeżenie strony). Wspólny układ zapewnia `RootLayout` z `<Outlet />`. Trasy chronione opakowano w `AuthGate`: podczas wczytywania sesji (`GET /api/auth/me`) wyświetlany jest spinner; brak sesji przekierowuje na `/login`; opcjonalna biała lista ról (`ADMIN`, `ORGANIZER`) chroni panele. W Dockerze nginx stosuje `try_files $uri /index.html`, aby bezpośrednie odnośniki SPA

nie kończyły się kodem 404.

Szkic kluczowych tras (fragment `routes.tsx`):

```tsx
export const router = createBrowserRouter([
  {
    path: "/",
    Component: RootLayout,
    children: [
      { index: true, Component: HomePage },
      { path: "login", Component: LoginPage },
      { path: "wydarzenia", Component: EventsPage },
      { path: "wydarzenia/:id", Component: EventDetailPage },
      { path: "dashboard", Component: ProtectedDashboard },
      { path: "garaz", Component: ProtectedGarage },
      { path: "organizer", Component: ProtectedOrganizer },
      { path: "admin", Component: ProtectedAdmin },
      // ...
    ],
  },
]);
```

Fragment kodu 8.1: Fragment drzewa tras React Router (katalog, auth, panele).

Komunikacja z API i stan sesji

Moduł `api` zbiera metody `get`/`post`/`patch`/`delete`, mapuje błędy na `ApiError` oraz przy kodzie 401 wykonuje jedną próbę odświeżenia sesji (`POST /api/auth/refresh`) przed wylogowaniem. Kontekst `AuthContext` utrzymuje `user`, flagę `isLoading` oraz funkcje konta (logowanie, rejestracja z weryfikacją e-mail, OAuth Google, profil, zmiana hasła, zamknięcie konta). Wczytywanie sesji jest odroczone do idle (`deferUntilIdle`), a przy powrocie karty przeglądarki odświeżany jest token — bez przechowywania JWT w JavaScript.

```tsx
useEffect(() => {
  clearLegacyToken();
  return deferUntilIdle(() => {
    void (async () => {
      try {
        const me = await api.get<User>("/api/auth/me");
        persistAuth(me);
      } catch {
        persistAuth(null);
      } finally {
        setIsLoading(false);
      }
    })();
  }, 2000);
}, []);
```

Fragment kodu 8.2: Wczytywanie sesji w `AuthContext` (httpOnly cookies).

Wspólny kontrakt typów z backendem wspiera pakiet `packages/api-types` (m.in. presety

torów `TRACK_PRESETS`). Mapa wydarzeń korzysta z **Leaflet** + OpenStreetMap; wyznaczanie trasy — z publicznego OSRM (Google Maps pozostaje opcją konfiguracyjną poza domyślną ścieżką MVP).

Komponenty i użyteczność

Struktura katalogów oddziela strony (`pages/`), kontekst auth, moduł komunikacji z API oraz bibliotekę UI (`components/ui/` — przyciski, dialogi, formularze, karty itd.). Tokeny marki (`--race-accent`, `--race-fg`, `--race-muted`) spójnie sterują motywem. Do nagłówków użyto fontu **Oxanium** (zastąpiono Orbitron ze względu na brak *latin-ext* dla polskich znaków), a do treści — Montserrat. Interfejs jest **mobile-first**: na telefonie działa dolna nawigacja oraz PWA (manifest + service worker, instalacja skrótu na ekranie początkowym). Stany puste, błędy tras oraz powiadomienia toast (`sonner`) informują użytkownika o wyniku działania. Panele ról nie dublują katalogu publicznego — organizator zarządza własnymi wydarzeniami, administrator moderuje publikacje i wnioski o rolę.

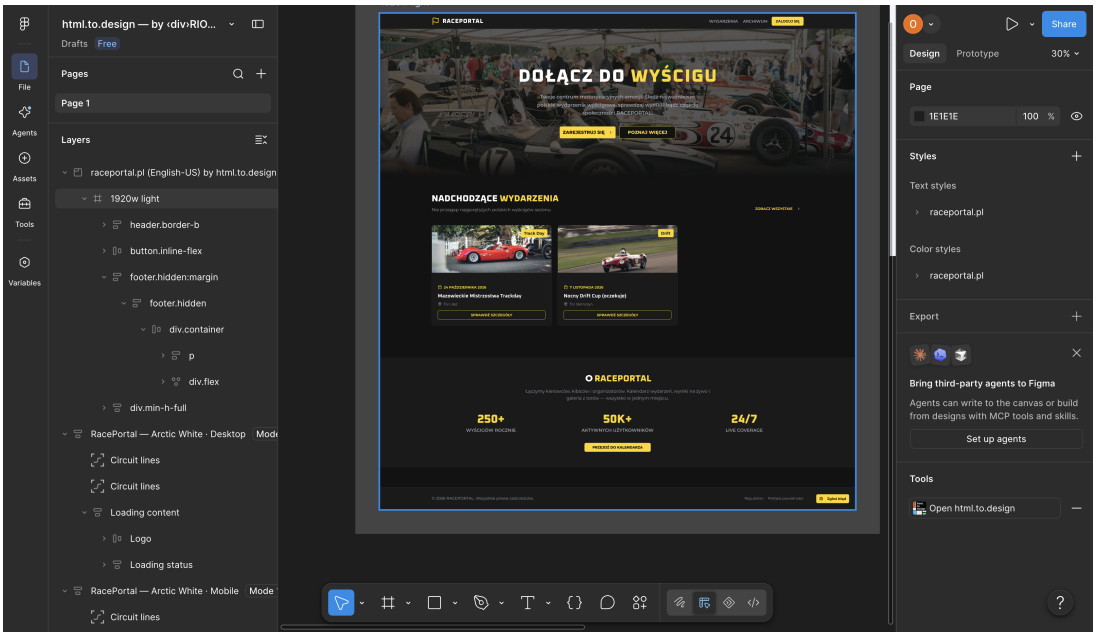
Użyteczność projektowano wokół krótkiej ścieżki: gość przegląda katalog → rejestracja / logowanie → garaż → zapis na wydarzenie; organizator publikuje imprezę po akceptacji administratora. CTA na stronie głównej i w kartach wydarzeń prowadzą do konkretnych tras SPA, a nie do martwych placeholderów prototypu.

8.6.1 Prototyp UI w Figma Make

Wyjściowym projektem interfejsu był plik Figma Make **Advanced UI/UX for RACEPORTAL** (ciemny motyw motorsportowy, akcent marki w kolorze złotym / żółtym, wyraziste fonty nagłówków, komponenty zbliżone do shadcn/ui). Prototyp obejmował m.in. stronę główną z hero i sekcją nadchodzących wydarzeń, katalog, logowanie/rejestrację, dashboard oraz szkice paneli organizatora i administratora. Warstwy w Figmie nosiły nazwy zbliżone do klas narzędziowych CSS (`header.border-b`, `button.inline-flex`), co ułatwiło przeniesienie układu do React/Tailwind.

Rysunek 8.2 przedstawia zrzut ekranu prototypu strony głównej (stan roboczy z Figmy, dokumentacja zespołu z 23.10.2026): hero z hasłem „Dołącz do wyścigu”, karty wydarzeń oraz blok informacyjny z wezwaniem do kalendarza.

Kod frontendu został przeniesiony z Figma Make do lokalnego monorepo (`web/`), a następnie podpięty pod realne API Spring Boot zamiast mocków i `localStorage`. Wczesna wizja produktu i ról — jeszcze przed pełną implementacją — jest też widoczna na diagramach koncepcyjnych (rys. 7.1, 4.1). W toku realizacji UI dopracowano pod kątem responsywności, stanów pustych/błędów, PWA oraz spójności z tokenami marki (`-race-accent`); font Orbitron zastąpiono Oxaniumem ze względu na polskie znaki diakrytyczne. Atrybucje komponentów shadcn/ui oraz zdjęć Unsplash udokumentowano w `docs/ATTRIBUTIONS.md`.



Rysunek 8.2: Prototyp UI RacePortal w Figma Make — strona główna (ciemny motyw motorsportowy, akcent marki).

8.7 Projekt aplikacji mobilnej

Aplikację mobilną zaprojektowano świadomie jako **wizytówkę katalogu**, a nie pełny klon weba. Założenia:

- stack Expo (React Native) z nawigacją typu stack wokół listy i detalu wydarzeń;
- publiczny klient HTTP do tych samych endpointów katalogu co web;
- konfiguracja adresów API i weba przez zmienne środowiskowe Expo (`EXPO_PUBLIC_API_URL`, `EXPO_PUBLIC_WEB_URL`);
- konwersja zapisu: przycisk CTA otwiera detal wydarzenia w przeglądarce / PWA (`Linking` → ścieżka wydarzenia na webie).

Poza zakresem projektu mobilnego pozostają: logowanie, garaż, panele organizatora/admina oraz publikacja w sklepach. Aplikacja mobilna służy więc głównie do przeglądania katalogu; sam zapis na wydarzenie użytkownik dokańcza w wersji webowej.

8.8 Projekt mechanizmów bezpieczeństwa

Bezpieczeństwo zaprojektowano warstwowo, z naciskiem na kontrolę dostępu API:

- uwierzytelnianie JWT — sesja webowa w ciasteczkach httpOnly (`rp_at`/`rp_rt`); API akceptuje też nagłówek Bearer (np. testy) oraz opcjonalne OAuth Google;

- autoryzacja RBAC na endpointach i w serwisach (rozróżnienie właściciela wydarzenia vs administrator);
- hasła przechowywane hashowane (BCrypt); osobne refresh tokeny;
- walidacja DTO po stronie API oraz odrzucanie niebezpiecznych URL-i tam, gdzie przyjmowane są odnośniki;
- CORS ograniczony do skonfigurowanych originów; na brzegu — TLS i nagłówki bezpieczeństwa nginx (m.in. CSP);
- rozdzielenie apexu i backoffice administracyjnego na osobnym hoście, z twardym wymaganiem roli `ADMIN`.

Szczegółową analizę zagrożeń, klasyfikację danych i ograniczenia przyjętych zabezpieczeń rozwinięto w rozdziale 13.

Rozdział 9

Realizacja

Rozdział opisuje, jak projekt z rozdziału 8 przełożono na działający system: organizację prac implementacyjnych, podział na komponenty, sposób ich integracji oraz uruchamianie środowiska (lokalnie i produkcyjnie). Szczegóły kodu backendu, frontendu i mobile rozwijają kolejne rozdziały implementacyjne.

9.1 Organizacja implementacji

Prace prowadzono iteracyjnie we wspólnym monorepo (web, API, mobile, pakiet typów, dokumentacja). Każda istotniejsza zmiana była weryfikowana uruchomieniem Compose oraz — w miarę możliwości — testami automatycznymi. Środowisko Docker Compose traktowano jako referencyjne środowisko uruchomieniowe: to, co działa w kontenerach lokalnie, jest tym samym stackiem co na VM produkcyjnej.

Organizacja pracy zakładała:

- dostarczanie kolejnych ścieżek użytkownika (katalog → konto → garaż/zapisy → panele ról), a nie budowę wszystkich warstw „na zapas”;
- bieżące dokumentowanie decyzji i luk (m.in. rejestr zgodności ze Specyfikacją Formularzy);
- świadome odkładanie elementów poza MVP (m.in. natywne PSP, pełna równoważność funkcji w aplikacji mobilnej, sklepy aplikacji);
- utrzymanie seedu danych demo, aby prezentacja i testy E2E miały powtarzalny punkt startu.

9.2 Podział systemu na komponenty

System zrealizowano jako trzy główne komponenty kliencko-serwerowe plus warstwę danych. Backend i frontend webowy omówiono skrótowo poniżej (szczegóły — rozdz. 10 i 11); aplika-

cję mobilną — skrótowo poniżej, a szczegółowo w rozdz. 12.

9.2.1 Backend

Komponent backendowy stanowi warstwę serwerową RacePortal i odpowiada za udostępnienie API REST, trwałe przechowywanie danych oraz egzekwowanie reguł biznesowych i uprawnień. Został zrealizowany w technologii Spring Boot (wersja 3.3.5) na platformie Java 21, z relacyjną bazą MySQL oraz mapowaniem obiektowo-relacyjnym Spring Data JPA. Budowa i uruchamianie odbywają się w kontenerze (obraz Maven do kompilacji, obraz JRE do runtime), co zapewnia powtarzalność środowiska lokalnego i produkcyjnego.

Struktura kodu w pakiecie `pl.raceportal` odzwierciedla klasyczny podział warstwowy. Warstwa kontrolerów eksponuje zasoby HTTP (m.in. uwierzytelnianie, wydarzenia, zgłoszenia, garaż, tory, mapy, uploady, panele organizatora i administratora oraz health-check). Warstwa serwisów zawiera logikę dziedzinową — m.in. rejestrację i logowanie, zarządzanie wydarzeniami, przepływ statusów zgłoszeń, dopasowanie pojazdu do wymagań wydarzenia, powiadomienia e-mail, moderację oraz archiwizację. Warstwa repozytoriów zapewnia dostęp do encji domenowych (użytkownik, pojazd, wydarzenie, zgłoszenie, tor i encje powiązane). Obiekty DTO oraz walidacja Bean Validation oddzielają kontrakt API od modelu persystencji, a globalna obsługa wyjątków ujednolica odpowiedzi błędów.

Bezpieczeństwo dostępu oparto o Spring Security [5] oraz tokeny JWT [14]: część zasobów katalogu pozostaje publiczna (przeglądanie wydarzeń, torów, trasy), natomiast operacje konta, garażu, zgłoszeń i moderacji wymagają uwierzytelnienia oraz odpowiednich ról. Komponent backendowy realizuje również zadania wspierające demonstrację i utrzymanie systemu: inicjalizację danych startowych (`DataInitializer`), katalog torów, cache wybranych list, a także cykliczną archiwizację minionych wydarzeń (`ArchiveService`, uruchamiana przy starcie aplikacji oraz codziennie według harmonogramu).

W podziale odpowiedzialności backend jest jedynym źródłem prawdy dla stanu domeny — klienci webowy i mobilny nie utrzymują własnej bazy wydarzeń ani zgłoszeń, lecz konsumują wspólny kontrakt JSON. Szczegółowy opis encji, endpointów i przepływów implementacyjnych przedstawiono w rozdziale 10.

9.2.2 Frontend webowy

Frontend webowy RacePortal został przygotowany w React z wykorzystaniem TypeScript i narzędzia Vite. Aplikacja składa się z osobnych stron oraz komponentów, a z backendem komunikuje się przez REST API. Użytkownik może przeglądać katalog wydarzeń i sprawdzać szczegóły wybranej imprezy. Po zalogowaniu kierowca ma również dostęp do swojego profilu oraz garażu, w którym może dodawać i edytować pojazdy.

Dostęp do poszczególnych funkcji zależy od roli użytkownika. Organizator korzysta z osobnego panelu do zarządzania wydarzeniami i zgłoszeniami, a administrator posiada panel do moderowania treści i zarządzania uprawnieniami. Aplikacja sprawdza, czy użytkownik jest zalogowany i czy ma odpowiednie uprawnienia do otwarcia danej strony. Dzięki temu jedna aplikacja łączy publiczny katalog wydarzeń z funkcjami dostępnymi po zalogowaniu.

9.2.3 Aplikacja mobilna

Komponent mobilny zrealizowano w Expo jako publiczną wizytówkę katalogu:

- lista i detal wydarzeń na tych samych danych API co web;
- brak logowania, garażu i paneli organizatora/admina;
- CTA „Zapisz się na stronie" przekierowujące do detalu wydarzenia w wersji webowej.

Aplikacja mobilna służy więc głównie do przeglądania katalogu; sam zapis na wydarzenie użytkownik dokańcza w wersji webowej.

9.3 Integracja komponentów

Integracja opiera się na wspólnym kontrakcie REST/JSON. Pakiet typów TypeScript (`packages/api-types`) ogranicza niezgodności w danych wymienianych przez aplikacje webową i mobilną. W środowisku Compose nginx w kontenerze `web` serwuje SPA i przekazuje żądania `/api` do kontenera `api`, dzięki czemu przeglądarka korzysta z jednego originu (bez osobnego URL API w buildzie produkcyjnym).

Przepływ typowy: klient (web lub mobile) woła endpoint API $\rightarrow$ Spring realizuje reguły domenowe i RBAC $\rightarrow$ MySQL utrwala stan $\rightarrow$ opcjonalnie wychodzi mail transakcyjny (SMTP Mailbux). Pliki (obrazy, PDF) trafiają do volume uploadów i są odczytywane przez API.

9.4 Konteneryzacja i uruchamianie środowiska

Uruchomienie lokalne sprowadza się do zbudowania i startu stacku Compose, a następnie sprawdzenia zdrowia API. Mobilkę uruchamia się osobno w katalogu Expo. W skrócie:

- start stacku: `docker compose up -build -d` [15];
- weryfikacja: endpoint health API zwraca stan OK;
- mobile: start Expo w katalogu aplikacji mobilnej;
- produkcja: ten sam Compose na Oracle VM, a ruch publiczny przez Cloudflare Tunnel do nginx na porcie `:8081`.

Dzięki zastosowaniu tego samego stacku kontenerów w środowisku deweloperskim i produkcyjnym wdrożenie nowej wersji polega na aktualizacji kodu na maszynie wirtualnej oraz przebudowie usług Docker Compose. Aplikacja webowa i API są publikowane łącznie, bez niezależnego procesu wdrożenia frontendu.

Rozdział 10

Szczegóły implementacji backendu

Rozdział niniejszy uszczegóławia realizację warstwy serwerowej RacePortal, której ogólny zarys przedstawiono w podrozdziale 9.2.1. Opis oparto o faktyczną strukturę pakietu `pl.raceportal` oraz zachowanie API REST.

10.1 Struktura projektu backendowego

Backend jest osobnym modułem Maven (`raceportal-backend`) w monorepozytorium. Rodzic projektu to `spring-boot-starter-parent` w wersji 3.3.5; język implementacji to Java 21. Do kluczowych zależności należą Spring Web, Spring Security, Spring Data JPA, Bean Validation, Spring Mail, sterownik MySQL, cache Caffeine oraz biblioteka JJWT. Opcjonalne logowanie Google wykorzystuje klienta Google API.

Kod aplikacji zorganizowano w pakietach odpowiadających odpowiedzialnościom:

- `controller` — ekspozycja zasobów HTTP;
- `service` — logika biznesowa i integracje (m.in. poczta, mapy, upload);
- `repository` — dostęp do danych przez Spring Data JPA;
- `domain` — encje i enumy statusów/ról;
- `dto` — kontrakty żądań i odpowiedzi z adnotacjami walidacji;
- `security` — JWT, ciasteczka sesji, limity prób, konfiguracja Spring Security;
- `web` — wyjątki domenowe i globalna obsługa błędów;
- `config`, `i18n` — konfiguracja serializacji oraz teksty wiadomości e-mail.

Klasa startowa `RacePortalApplication` włącza harmonogram zadań (`@EnableScheduling`), wykorzystywany m.in. do archiwizacji wydarzeń.

10.2 Model domenowy i encje

Model persystencji odzwierciedla główne pojęcia katalogu i zapisów. Encja `User` reprezentuje konto: unikalny e-mail i nazwę użytkownika, skrót hasła, rolę (`USER`, `ORGANIZER`, `ADMIN`), dane profilu, flagi weryfikacji e-mail oraz pola związane z opcjonalnym kontem Google. Encja `Car` opisuje pojazd w garażu (marka, model, klasa, napęd, parametry techniczne, opony, wymagane dokumenty/wyposażenie). Encja `Event` przechowuje opis wydarzenia, lokalizację (w tym współrzędne), kategorię, status publikacji, warunki płatności oraz wymagania wobec uczestnika i pojazdu; wydarzenie jest powiązane z organizatorem (`User`).

Encja `Registration` modeluje zgłoszenie udziału: powiązanie użytkownika, wydarzenia i pojazdu, status, komentarze, dowód płatności oraz termin opłaty. Statusy zgłoszenia obejmują m.in. `PENDING`, `ACCEPTED`, `CONFIRMED` i `CANCELED`. Statusy wydarzenia obejmują m.in. `DRAFT`, `PENDING`, `APPROVED`, `REJECTED`, `ARCHIVED` i `CANCELLED`.

Dodatkowo w modelu występują: katalog torów (`Track`), wnioski o rolę organizatora (`OrganizerApplica`
odświeżające tokeny sesji (`RefreshToken`), klasy zawodów powiązane z wydarzeniem, wyniki, reguły punktacji, powiadomienia in-app oraz zgłoszenia błędów. Katalog torów nie jest kluczem obcym wydarzenia — lokalizacja wydarzenia jest przechowywana w polach wydarzenia, a katalog służy jako słownik wspomagający formularze i filtry. Fragment encji `User` (listing 10.1) ilustruje kluczowy model tożsamości i ról.

```
1  @Entity
2  @Table(name = "users")
3  public class User {
4      @Id
5      @GeneratedValue(strategy = GenerationType.UUID)
6      private String id;
7
8      @Column(nullable = false, unique = true, length = 190)
9      private String email;
10
11     @Column(name = "password_hash", nullable = false, length = 100)
12     private String passwordHash;
13
14     @Enumerated(EnumType.STRING)
15     @Column(nullable = false, length = 20)
16     private Role role = Role.USER;
17     // ... profil, weryfikacja e-mail, tokenVersion
18 }
```

Fragment kodu 10.1: Kluczowe pola encji `User`

10.3 Rejestracja użytkowników i logowanie

Uwierzytelnianie zrealizowano w `AuthController` (`/api/auth`) oraz `AuthService`. Rejestracja kierowcy tworzy konto z rolą `USER` w stanie niezweryfikowanym i wysyła wiadomość

aktywacyjną z linkiem `/verify-email/{token}` (czas ważności tokenu ograniczony). Do momentu weryfikacji system nie wydaje sesji użytkownika. Dostępna jest także ścieżka rejestracji organizatora, która równolegle zakłada wniosek o uprawnienia.

Logowanie hasłem weryfikuje poświadczenia i wystawia sesję: krótkotrwały token dostępu JWT oraz rotowany token odświeżający przechowywany w bazie w postaci skrótu. Token dostępu niesie m.in. identyfikator, e-mail, nazwę użytkownika, rolę oraz wersję tokenu (`tokenVersion`), umożliwiając unieważnienie sesji przy wylogowaniu lub zmianie poświadczeń. Odświeżanie sesji odbywa się endpointem `/refresh`; wylogowanie unieważnia refresh i podnosi wersję tokenu. Opcjonalnie wspierane jest logowanie Google (weryfikacja `idToken` przy skonfigurowanym identyfikatorze klienta).

Ochronę przed nadużyciami zapewnia `AttemptRateLimiter` (m.in. limity prób logowania i weryfikacji). Konfiguracja Spring Security utrzymuje sesję bezstanową po stronie serwera i udostępnia publicznie m.in. endpointy rejestracji, logowania, weryfikacji, odświeżania oraz odczyt katalogu wydarzeń (listing 10.2). Wydanie sesji po udanym logowaniu lub weryfikacji e-mail realizuje `issueSession` (listing 10.3).

```
http
  .csrf(csrf -> csrf.disable())
  .sessionManagement(sm ->
      sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
  .authorizeHttpRequests(auth -> auth
      .requestMatchers(HttpMethod.POST,
          "/api/auth/login", "/api/auth/register",
          "/api/auth/verify-email", "/api/auth/refresh").permitAll()
      .requestMatchers(HttpMethod.GET, "/api/events/**").permitAll()
      .anyRequest().authenticated())
  .addFilterBefore(jwtAuthFilter,
      UsernamePasswordAuthenticationFilter.class);
```

Fragment kodu 10.2: Fragment `SecurityFilterChain` — sesja bezstanowa i ścieżki publiczne

```
private AuthResponse issueSession(User user) {
    String access = jwtService.generateToken(user);
    String refresh = refreshTokenService.issue(user);
    return new AuthResponse(access, refresh, toUserDto(user));
}
```

Fragment kodu 10.3: Wydanie sesji: JWT access + rotowany refresh

10.4 Zarządzanie profilami użytkowników

Profil zalogowanego użytkownika jest obsługiwany w ramach zasobów `/api/auth` (brak osobnego kontrolera profilu). Endpoint `GET /me` zwraca bieżące dane konta, a `PATCH /me` umożliwia aktualizację nazwy użytkownika, avatara, imienia i nazwiska, telefonu, adresu, odnośnika

do Instagrama, deklaracji prawa jazdy kategorii B i licencji PZM oraz preferencji lokalizacji językowej i motywu interfejsu.

Zmiana hasła (`POST /me/password`) jest dostępna dla kont lokalnych i zablokowana dla kont utworzonych wyłącznie przez Google. Zmiana adresu e-mail przebiega dwuetapowo (żądanie + potwierdzenie), z powiadomieniem na dotychczasowy adres. Użytkownik może zamknąć własne konto po podaniu wymaganej frazy potwierdzającej. Takie podejście łączy zarządzanie tożsamością z kontem w jednym module autentycznym, upraszczając powierzchnię API (listing 10.4).

```
@GetMapping("/me")
@PreAuthorize("isAuthenticated()")
public ResponseEntity<UserDto> me(
        @AuthenticationPrincipal UserPrincipal currentUser) {
    return ResponseEntity.ok(authService.me(currentUser.getId()));
}

@PatchMapping("/me")
@PreAuthorize("isAuthenticated()")
public ResponseEntity<UserDto> updateMe(
        @AuthenticationPrincipal UserPrincipal currentUser,
        @Valid @RequestBody UpdateMeRequest request) {
    return ResponseEntity.ok(
        authService.updateMe(currentUser.getId(), request));
}
```

Fragment kodu 10.4: Endpointy profilu w `AuthController`

10.5 Obsługa wydarzeń

Zasób `/api/events` (`EventController`, `EventService`) realizuje publiczny katalog oraz zarządzanie wydarzeniami przez uprawnione role. Lista wydarzeń obsługuje paginację oraz filtry (m.in. tekst, kategoria, miasto, województwo, tor, zakres dat, status, płatność, dopasowanie do pojazdu z garażu). Publicznie domyślnie widoczne są wydarzenia zatwierdzone z datą bieżącą lub przyszłą. Osobny endpoint markerów dostarcza dane do widoku mapy.

Tworzenie i edycja wydarzeń wymagają roli organizatora lub administratora. Nowe wydarzenie organizatora trafia do statusu oczekującego na moderację (z powiadomieniem administratorów); administrator może publikować od razu jako zatwierdzone (listing 10.5). Edycja przez organizatora ponownie kieruje treść do moderacji i może uruchomić powiadomienia do osób już zgłoszonych. Anulowanie wydarzenia ustawia odpowiedni status, anuluje otwarte zgłoszenia i informuje uczestników. Uzupełnieniem cyklu życia jest `ArchiveService`, który przy starcie aplikacji oraz codziennie przenosi minione, zatwierdzone wydarzenia do archiwum.

```
@Transactional
@CacheEvict(cacheNames = "events", allEntries = true)
public EventResponse create(EventCreateRequest request,
```

```
                                UserPrincipal currentUser) {
    Event event = new Event();
    // ... mapowanie pol z requestu (nazwa, data, lokalizacja, oplaty)
    event.setStatus(currentUser.getRole() == Role.ADMIN
            ? EventStatus.APPROVED
            : EventStatus.PENDING);
    event.setOrganizer(organizer);
    event = eventRepository.save(event);
    if (event.getStatus() == EventStatus.PENDING) {
        notificationService.notifyAdminsPendingEvent(event);
    }
    return serialize(event);
}
```

Fragment kodu 10.5: Tworzenie wydarzenia — status zależny od roli

10.6 Obsługa zgłoszeń

Zgłoszenia obsługuje `RegistrationController` (`/api/registrations`) przy wymaganiu uwierzytelnienia. Utworzenie zgłoszenia wiąże kierowcę, zatwierdzone wydarzenie przyjmujące zapisy oraz — opcjonalnie — pojazd z garażu; system może przypisać klasę startową według reguł wydarzenia. Początkowy status to `PENDING`.

Zmiany statusów podlegają walidacji przejść zależnej od charakteru wydarzenia. Dla imprez płatnych typowy przebieg obejmuje akceptację (`ACCEPTED`) z wyznaczeniem terminu opłaty, dołączenie dowodu płatności przez kierowcę oraz potwierdzenie (`CONFIRMED`). Dla imprez bezpłatnych możliwe jest przejście z `PENDING` bezpośrednio do `CONFIRMED`. Kierowca może anulować własne zgłoszenie w dopuszczalnych stanach. Operacje listy zgłoszeń wydarzenia oraz zmiany statusów są ograniczone do organizatora-właściciela lub administratora. Kluczowe zmiany generują transakcyjne wiadomości e-mail (utworzenie, status, anulowanie). Maszynę stanów ilustruje listing 10.6.

```
private void validateTransition(RegistrationStatus from,
        RegistrationStatus to, Registration registration, Event event) {
    if (from == RegistrationStatus.CANCELED
            || from == RegistrationStatus.CONFIRMED) {
        throw ApiException.badRequest(
            "Nie mozna zmienic statusu zgloszenia zakonczonego");
    }
    boolean valid = switch (from) {
        case PENDING -> event.isPaid()
            ? (to == RegistrationStatus.ACCEPTED
                || to == RegistrationStatus.CANCELED)
            : (to == RegistrationStatus.CONFIRMED
                || to == RegistrationStatus.CANCELED);
        case ACCEPTED -> to == RegistrationStatus.CONFIRMED
                || to == RegistrationStatus.CANCELED;
        default -> false;
    };
    if (!valid) {
```

```
19          throw ApiException.badRequest(
20              "Nieprawidlowe przejscie statusu zgloszenia");
21      }
22      if (to == RegistrationStatus.CONFIRMED
23              && from == RegistrationStatus.ACCEPTED
24              && (registration.getPaymentProofUrl() == null
25                  || registration.getPaymentProofUrl().isBlank())) {
26          throw ApiException.badRequest(
27              "Brak potwierdzenia przelewu");
28      }
29  }
```

Fragment kodu 10.6: Walidacja przejść statusu zgłoszenia

10.7 Obsługa garażu

Garaż udostępnia `GarageController` (`/api/garage`) wyłącznie użytkownikom zalogowanym. Serwis umożliwia listowanie, dodawanie, edycję i usuwanie pojazdów należących do bieżącego konta. Przy tworzeniu wymagane są co najmniej marka i model; pozostałe atrybuty techniczne i deklaracje (np. OC, przegląd, klatka) służą późniejszemu dopasowaniu do wymagań wydarzeń.

Ograniczenia spójności chronią otwarte procesy zapisów: przy aktywnym zgłoszeniu zablokowana jest zmiana kluczowych cech pojazdu, a usunięcie pojazdu nie jest możliwe, gdy bierze on udział w zgłoszeniach w stanach uniemożliwiających bezpieczne odpięcie (listing 10.7). Dzięki temu garaż pozostaje spójny z modułem zgłoszeń i regułami *carMatch*.

```
1   @Transactional
2   public void delete(String userId, String carId) {
3       Car car = carRepository.findByIdAndUser_Id(carId, userId)
4           .orElseThrow(() -> ApiException.notFound("Nie znaleziono auta"));
5
6       if (registrationRepository.existsByCar_IdAndStatusIn(
7               carId, OPEN_STATUSES)) {
8           throw ApiException.badRequest(
9               "Brak mozliwosci usuniecia auta ze wzgledu na otwarte zgloszenie");
10      }
11      registrationRepository.clearCarId(carId);
12      carRepository.delete(car);
13  }
```

Fragment kodu 10.7: Usuwanie pojazdu z blokadą przy otwartym zgłoszeniu

10.8 Panel administratora i organizatora

Panel administracyjny eksponuje `AdminController` pod prefiksem `/api/admin` z wymaganiem roli `ADMIN`. Udostępnia m.in. statystyki platformy, listę użytkowników, zmianę ról (z za-

bezpieczeniami przed usunięciem ostatniego administratora), kolejki wydarzeń do moderacji, zmianę statusów wydarzeń oraz rozstrzyganie wniosków o rolę organizatora (akceptacja skutkuje nadaniem roli i powiadomieniem e-mail).

Ścieżki organizatora obejmują `OrganizerController` (`/api/organizer`): podgląd własnego wniosku, złożenie wniosku o rolę oraz listę wydarzeń zarządzanych przez organizatora (administrator widzi pełny zbiór). Operacje tworzenia i edycji wydarzeń oraz zarządzania zgłoszeniami uczestników są realizowane przez zasoby wydarzeń i zgłoszeń z kontrolą dostępu opartą o role i własność zasobu. Taki podział utrzymuje czytelne granice RBAC bez dublowania całej logiki w jednym kontrolerze. Klasę panelu administratora przedstawia listing 10.8.

```
1  @RestController
2  @RequestMapping("/api/admin")
3  @PreAuthorize("hasRole('ADMIN')")
4  public class AdminController {
5
6      @GetMapping("/events/pending")
7      public ResponseEntity<List<EventResponse>> pendingEvents() {
8          return ResponseEntity.ok(adminService.pendingEvents());
9      }
10
11     @PatchMapping("/organizer-applications/{id}")
12     public ResponseEntity<OrganizerApplicationResponse>
13             updateOrganizerApplication(@PathVariable String id,
14                 @Valid @RequestBody
15                 OrganizerApplicationStatusUpdateRequest request) {
16         return ResponseEntity.ok(
17             adminService.updateOrganizerApplicationStatus(
18                 id, request.status()));
19     }
20 }
```

Fragment kodu 10.8: Panel administratora — RBAC na poziomie klasy

10.9 Integracja z bazą danych

Trwałość danych zapewnia MySQL w połączeniu ze Spring Data JPA. Konfiguracja połączenia pochodzi ze zmiennych środowiskowych (w środowisku Compose hostem jest usługa kontenerowa bazy). Aplikacja działa z `spring.jpa.open-in-view=false`. Schemat jest utrzymywany trybem `ddl-auto` (domyślnie `update`); w projekcie nie zastosowano Flyway ani Liquibase — jest to świadome uproszczenie MVP, akceptowalne przy kontrolowanym środowisku demonstracyjnym, lecz wymagające ostrożności przy dalszym rozwoju produkcyjnym.

Dla list wydarzeń zastosowano cache Caffeine (krótki czas życia), invalidowany m.in. przy archiwizacji. Seed danych startowych (`DataInitializer`) zapewnia konta demonstracyjne, katalog torów oraz przykładowe treści ułatwiające prezentację ról. Kluczowe ustawienia JPA i cache pokazuje listing 10.9.

```yaml
spring:
  datasource:
    url: ${DATABASE_URL:jdbc:mysql://localhost:3306/raceportal}
    username: ${DATABASE_USERNAME:raceportal}
    password: ${DATABASE_PASSWORD:raceportal}
  jpa:
    hibernate:
      ddl-auto: ${JPA_DDL_AUTO:update}
    open-in-view: false
  cache:
    type: caffeine
    cache-names: events
    caffeine:
      spec: maximumSize=500,expireAfterWrite=20s
```

Fragment kodu 10.9: Konfiguracja MySQL/JPA i cache Caffeine (`application.yml`)

10.10 Obsługa wyjątków i walidacja

Wejście API jest walidowane adnotacjami Bean Validation na rekordach DTO (`@Valid` w kontrolerach). Błędy reguł biznesowych i autoryzacji są reprezentowane przez `ApiException` z odpowiednim kodem HTTP (m.in. 400, 401, 403, 404, 409, 429).

`GlobalExceptionHandler` (`@RestControllerAdvice`) mapuje wyjątki na jednolite odpowiedzi JSON, w tym błędy walidacji pól, naruszenia integralności danych, odmowy dostępu oraz limity liczby prób (z nagłówkiem `Retry-After` przy 429). Dzięki temu klienci webowy i mobilny otrzymują przewidywalny kontrakt błędów, a szczegóły techniczne wyjątków nie są niekontrolowanie ujawniane użytkownikowi końcowemu. Fabryki `ApiException` oraz handler walidacji przedstawiają listingi 10.10 i 10.11.

```
public static ApiException badRequest(String message) {
    return new ApiException(HttpStatus.BAD_REQUEST, message);
}
public static ApiException unauthorized(String message) {
    return new ApiException(HttpStatus.UNAUTHORIZED, message);
}
public static ApiException forbidden(String message) {
    return new ApiException(HttpStatus.FORBIDDEN, message);
}
public static ApiException notFound(String message) {
    return new ApiException(HttpStatus.NOT_FOUND, message);
}
public static ApiException conflict(String message) {
    return new ApiException(HttpStatus.CONFLICT, message);
}
public static ApiException tooManyRequests(String message) {
    return new ApiException(HttpStatus.TOO_MANY_REQUESTS, message);
}
```

Fragment kodu 10.10: Fabryki wyjątków biznesowych API

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ApiException.class)
    public ResponseEntity<ErrorResponse> handleApiException(ApiException ex) {
        return ResponseEntity.status(ex.getStatus())
            .body(new ErrorResponse(ex.getMessage(), ex.getDetails()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(fe ->
            fieldErrors.put(fe.getField(), fe.getDefaultMessage()));
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse("Nieprawidlowe dane", fieldErrors));
    }
}
```

Fragment kodu 10.11: Globalna obsługa `ApiException` i błędów `@Valid`

Rozdział 11

Szczegóły implementacji frontendu webowego

11.1 Struktura aplikacji webowej

Warstwa klientowska systemu RacePortal została zaimplementowana jako aplikacja jednostronicowa (SPA) oparta o bibliotekę **React 18** oraz język **TypeScript**, z bundlerem **Vite** oraz stylowaniem **Tailwind CSS** i komponentami Radix UI. Katalogi w module `web/src/app` podzielono modularnie, co ułatwia czytanie i utrzymanie kodu:

- `pages/` — komponenty widoków ekranowych reprezentujące poszczególne podstrony aplikacji.
- `components/` — reużywalne elementy interfejsu (formularze, karty, modale, elementy nawigacyjne).
- `context/` — zarządzanie stanem globalnym, w tym główny kontekst autoryzacji (`AuthContext`) kontrolujący sesję użytkownika.
- `lib/` — warstwa logiki sieciowej (`api.ts`), typowania oraz konwersji danych zgodna ze wspólnym pakietem typów API.

Poniższy kod definiuje drzewo tras aplikacji przy użyciu `createBrowserRouter` z React Router v7: układ bazowy (`RootLayout`) oraz chronione trasy dla poszczególnych ról systemowych.

```
export const router = createBrowserRouter([
  {
    path: "/",
    Component: RootLayout,
    children: [
      { index: true, Component: HomePage },
      { path: "login", Component: LoginPage },
      { path: "wydarzenia", Component: EventsPage },
      { path: "wydarzenia/:id", Component: EventDetailPage },
      { path: "dashboard", Component: ProtectedDashboard },
```

```
11        { path: "garaz", Component: ProtectedGarage },
12        { path: "organizer", Component: ProtectedOrganizer },
13        { path: "admin", Component: ProtectedAdmin },
14      ],
15    },
16  ]);
```

Fragment kodu 11.1: Fragment drzewa tras React Router (katalog, auth, panele) — warstwa webowa.

11.2 Strona główna i katalog wydarzeń

Strona główna integruje główny katalog wydarzeń motorsportowych, oferując użytkownikom trzy spójne widoki z zachowaniem tych samych filtrów: **listę chronologiczną**, **interaktywną mapę** oraz **kalendarz**.

- W widoku listy zastosowano *debounce* filtrów, żeby nie wysyłać zbędnych zapytań HTTP do backendu podczas wpisywania fraz wyszukiwania.
- Użytkownik ma do dyspozycji funkcję „Wyczyść filtry” oraz paginację listy zsynchronizowaną z markerami przeglądowymi na mapie.

11.3 Widok szczegółów wydarzenia

Podstrona szczegółów dostarcza użytkownikowi pełnych informacji o wybranej imprezie motorsportowej:

- Wyświetla dokładny opis, dane organizatora oraz wymagania techniczne stawiane pojazdom i uczestnikom.
- Integruje moduł szybkiego zapisu na wydarzenie, który weryfikuje posiadane przez kierowcę auta w garażu pod kątem reguł dopasowania kategorii (`carMatch`).
- Zawiera mapę z wyznaczaniem trasy dojazdu do lokalizacji obiektu z wykorzystaniem routingu OSRM [16].

11.4 Logowanie i rejestracja

Moduł autoryzacji opiera się na formularzach logowania, rejestracji oraz odzyskiwania hasła. Sesję utrzymują ciasteczka httpOnly (`rp_at`, `rp_rt`) z flagą `credentials: "include"`, dzięki czemu tokeny JWT nie są trzymane w `localStorage`.

Poniższy kod implementuje asynchroniczne pobieranie danych o zalogowanym użytkowniku przy użyciu mechanizmu optymalizacyjnego `deferUntilIdle`, dbając o to, by zapytanie nie blokowało renderowania głównego wątku aplikacji.

```
useEffect(() => {
  clearLegacyToken();
  return deferUntilIdle(() => {
    void (async () => {
      try {
        const me = await api.get<User>("/api/auth/me");
        persistAuth(me);
      } catch {
        persistAuth(null);
      } finally {
        setIsLoading(false);
      }
    })();
  }, 2000);
}, []);
```

Fragment kodu 11.2: Wczytywanie sesji w `AuthContext` (httpOnly cookies).

11.5 Dashboard użytkownika

Panel użytkownika (Dashboard) to miejsce, w którym zalogowany kierowca zarządza swoją aktywnością w systemie. Pokazuje podsumowanie aktualnych zgłoszeń na nadchodzące wydarzenia motorsportowe, statusy (oczekujące, zatwierdzone, anulowane) oraz dostęp do szczegółów rezerwacji.

11.6 Widok garażu

Moduł wirtualnego garażu pozwala kierowcom zarządzać własnymi pojazdami startowymi.

- Formularz dodawania pojazdu sprawdza pola wymagane (m.in. marka, model, specyfikacja techniczna, masa, dopasowanie opon).
- Można edytować dane oraz usuwać auta z profilu; system blokuje m.in. usunięcie pojazdu powiązanego z aktywnym, zatwierdzonym zgłoszeniem na zawody.

11.7 Panel organizatora

Panel organizatora służy do publikacji imprez motorsportowych. Umożliwia:

- Tworzenie oraz edycję wydarzeń w jednym formularzu, z presetami torów.

- Podgląd zgłoszeń kierowców oraz zarządzanie ich statusami (np. akceptacja, odrzucenie lub zmiana statusu płatności).

11.8 Panel administratora

Panel administratora daje operatorom platformy narzędzia do nadzoru nad treścią i uprawnieniami. Obejmuje:

- Moderację nadesłanych wydarzeń oraz rozpatrywanie wniosków o uprawnienia organizatora (formularz pod adresem `/zostan-organizatorem`).
- Zarządzanie rolami kont oraz podgląd systemowych zgłoszeń błędów.

11.9 Mapa i elementy lokalizacyjne

Warstwa mapowa opiera się na integracji biblioteki Leaflet [17] z kafelkami CARTO / OpenStreetMap. Umożliwia ona interaktywną prezentację punktów zainteresowania, selekcję współrzędnych geograficznych w formularzach wydarzeń oraz płatną lub opartą o OSRM [16] integrację routingu trasy dojazdu dla kierowców (proxy `/api/maps/route`).

11.10 Responsywność i doświadczenie użytkownika

Interfejs aplikacji zaprojektowano w koncepcji *mobile-first*. Układy siatek (grid, flexbox) w Tailwind CSS dostosowują się do ekranów smartfonów, tabletów oraz monitorów stacjonarnych. Spójność wizualną daje ciemny motyw motorsportowy z akcentem `-race-accent`, dobrany pod czytelność danych.

Rozdział 12

Szczegóły implementacji aplikacji mobilnej

Rozdział opisuje implementację klienta mobilnego RacePortal. Aplikacja powstała w technologii Expo (React Native) i pełni rolę publicznej wizytówki katalogu: umożliwia przeglądanie wydarzeń na telefonie, natomiast zapis udziału oraz funkcje konta realizowane są w wersji webowej.

12.1 Założenia aplikacji mobilnej

Przyjęto świadome ograniczenie zakresu względem klienta webowego. Aplikacja mobilna ma przede wszystkim umożliwić przeglądanie nadchodzących wydarzeń motorsportowych — w postaci listy, mapy, kalendarza oraz ekranu szczegółów — a nie stanowić drugiej, pełnej aplikacji z logowaniem i garażem.

Założenia wynikające z tego podziału:

- brak uwierzytelniania i przechowywania tokenów JWT w aplikacji mobilnej;
- wyłącznie publiczne operacje odczytu katalogu;
- konwersja zapisu przez otwarcie odpowiedniej strony wydarzenia w przeglądarce / PWA;
- wspólny kontrakt danych z API (te same wydarzenia i filtry co na webie);
- brak publikacji w sklepach App Store / Google Play w ramach MVP (uruchomienie przez Expo / emulator).

We wcześniejszej fazie projektu rozważano szerszą równoważność funkcji z webem; ostatecznie zakres zawężono do wizytówki, aby uniknąć dublowania logiki zapisów i utrzymać jeden główny kanał zapisów.

12.2 Architektura rozwiązania mobilnego

Rozwiązanie oparto o Expo SDK oraz React Native. Warstwa prezentacji korzysta z React Navigation (native stack) ograniczonego do przepływu wydarzeń. Motyw wizualny (ciemne tło, akcent marki) oraz lokalizacja interfejsu (PL/EN) są dostarczane przez konteksty aplikacji.

Klient HTTP ograniczono do minimum potrzebnego wizytówce: wykonuje wyłącznie publiczne żądania `GET` do API RacePortal, bez nagłówka autoryzacji. Adres API i adres wersji webowej (używany przy CTA zapisu) konfigurowane są zmiennymi środowiskowymi Expo, z domyślnymi wartościami dopasowanymi do środowiska deweloperskiego (m.in. inny host dla emulatora Androida niż dla symulatora iOS).

Aplikacja mobilna **nie** wchodzi w skład Docker Compose — uruchamiana jest osobno obok kontenerów web/API/MySQL.

12.3 Lista wydarzeń i widok szczegółów

Ekran listy pobiera wydarzenia z publicznego endpointu katalogu i udostępnia trzy tryby prezentacji: lista, kalendarz oraz mapa (markery). Filtry publiczne obejmują m.in. wyszukiwanie tekstowe, kategorię, województwo, miasto, tor, zakres dat oraz oznaczenie wydarzeń płatnych / bezpłatnych. Zestawy lokalizacji (województwa, tory) są spójne z listami używanymi po stronie weba, aby obie aplikacje operowały na tych samych słownikach.

Dobór zakresu dat korzysta z natywnego komponentu wyboru daty. Wygląd i zachowanie różnią się między platformami: na iOS stosowany jest tryb `inline`, na Androidzie — systemowy dialog (`default`). Po wyborze daty na Androidzie selektor jest zamykany od razu (listing 12.1).

```
const onDatePicked = (event, selectedDate) => {
  if (Platform.OS === "android") setDatePickerTarget(null);
  if (event.type !== "set" || !selectedDate || !datePickerTarget) return;
  const value = formatDate(selectedDate);
  if (datePickerTarget === "from") setDateFrom(value);
  if (datePickerTarget === "to") setDateTo(value);
};

{datePickerTarget && (Platform.OS === "android" || Platform.OS === "ios") ? (
  <DateTimePicker
    value={currentPickerDate}
    mode="date"
    display={Platform.OS === "ios" ? "inline" : "default"}
    onChange={onDatePicked}
  />
) : null}
```

Fragment kodu 12.1: Wybór daty filtra z rozróżnieniem iOS / Android

Widok szczegółów pobiera pojedyncze wydarzenie po identyfikatorze i prezentuje podstawowe metadane (nazwa, kategoria, data, lokalizacja, opis, multimedia). Z poziomu detalu użytkownik może otworzyć lokalizację w zewnętrznej aplikacji map oraz przejść do zapisu na stronie webowej. Otwarcie map realizuje systemowe `Linking` z zapytaniem zawierającym tor, miasto i współrzędne (listing 12.2).

```
const openMaps = () => {
  if (!event?.lat || !event?.lng) return;
  const q = encodeURIComponent(
    `${event.track}, ${event.city} ${event.lat},${event.lng}`
  );
  void Linking.openURL(
    `https://www.google.com/maps/search/?api=1&query=${q}`
  );
};
```

Fragment kodu 12.2: Otwarcie lokalizacji wydarzenia w zewnętrznej aplikacji map

12.4 Nawigacja i interfejs użytkownika

Nawigacja ogranicza się do jednego stosu ekranów: lista wydarzeń oraz detal wybranego wydarzenia. Nie zastosowano dolnych kart (tabów) typowych dla pełnego klienta, ponieważ poza katalogiem nie ma w mobilce osobnych obszarów konta, garażu ani paneli ról. Ilustruje to listing 12.3: zarejestrowane są wyłącznie ekrany katalogu.

```
function EventsNavigator() {
  const { accentColor } = useTheme();
  const { t } = useLocale();
  return (
    <EventsStack.Navigator
      screenOptions={{
        headerStyle: { backgroundColor: colors.card },
        headerTintColor: accentColor,
        headerTitleStyle: { color: colors.text, fontWeight: "800" },
      }}
    >
      <EventsStack.Screen
        name="EventsList"
        component={EventsScreen}
        options={{ headerShown: false }}
      />
      <EventsStack.Screen
        name="EventDetail"
        component={EventDetailScreen}
        options={{ title: t("events.details") }}
      />
    </EventsStack.Navigator>
  );
}
```

Fragment kodu 12.3: Stos nawigacji ograniczony do listy i detalu wydarzenia

Interfejs utrzymuje ciemny motyw motorsportowy spójny z identyfikacją RacePortal. Elementy wspólne (nagłówki, przyciski, stany ładowania i braku danych) wydzielono do prostych komponentów UI, aby zachować jednolity wygląd obu ekranów.

12.5 Integracja z API

Integracja obejmuje wyłącznie publiczne odczyty potrzebne katalogowi, w szczególności:

- listę wydarzeń (z filtrami i podziałem na strony),
- markery mapy,
- detal wydarzenia,
- opcjonalnie sprawdzenie dostępności API (health) w scenariuszach diagnostycznych.

Ścieżki względne multimediów (np. uploady) są uzupełniane o adres bazowy API, ponieważ klient natywny — w odróżnieniu od aplikacji webowej serwowanej z tego samego hosta co API — nie rozwiązuje automatycznie ścieżek względnych zasobów.

Konfiguracja hosta uwzględnia typowe różnice środowisk: lokalny host na symulatorze iOS, specjalny adres bramy emulatora Androida (`10.0.2.2`) oraz możliwość wskazania adresu LAN / produkcji przez zmienne Expo. Fragment z listingu 12.4 pokazuje domyślne wyznaczanie adresu API (analogiczna logika dotyczy adresu wersji webowej używanego przy CTA).

```
export const API_URL = (() => {
  if (process.env.EXPO_PUBLIC_API_URL) {
    return process.env.EXPO_PUBLIC_API_URL.replace(/\/$/, "");
  }
  const expoGoHost = resolveExpoGoHost();
  if (expoGoHost && expoGoHost !== "localhost"
      && expoGoHost !== "127.0.0.1") {
    return `http://${expoGoHost}:4000`;
  }
  return Platform.OS === "android"
    ? "http://10.0.2.2:4000"
    : "http://127.0.0.1:4000";
})();
```

Fragment kodu 12.4: Wyznaczanie bazowego adresu API dla klienta mobilnego

12.6 Przekierowanie do wersji webowej

Zapis na wydarzenie nie jest realizowany w aplikacji mobilnej. Na ekranie detalu umieszczono przycisk CTA, który buduje adres wersji webowej wydarzenia i otwiera go systemowym mechanizmem łączy zewnętrznych (`Linking`). Użytkownik trafia tym samym do pełnego flow zapisu na webie (konto, garaż, statusy zgłoszenia, ewentualny dowód płatności).

Takie rozwiązanie utrzymuje jedną implementację reguł biznesowych zapisów i zmniejsza ryzyko rozjazdu mobile $\leftrightarrow$ web.

12.7 Ograniczenia wersji mobilnej

Wersja mobilna MVP nie obejmuje:

- logowania, rejestracji ani zarządzania profilem;
- garażu pojazdów i składania zgłoszeń bezpośrednio z aplikacji;
- paneli organizatora i administratora;
- archiwum, wyników zawodów ani funkcji społecznościowych;
- powiadomień push oraz publikacji w sklepach aplikacji.

Ograniczeniem wynikającym z przyjętego modelu CTA jest zależność od dostępności wersji webowej w momencie zapisu. Jednocześnie ograniczenie to jest zamierzone: mobilka pozostaje lekkim kanałem przeglądania katalogu, a kompletny produkt transakcyjny skupia się na webie (w tym PWA).

Rozdział 13

Bezpieczeństwo systemu

Rozdział opisuje założenia ochrony RacePortal, klasyfikację przetwarzanych danych oraz wybrane aspekty ochrony zasobów, komunikacji, analizy zagrożeń i ryzyka. Uwzględniono także wyniki testów penetracyjnych powierzchni HTTP(S) produkcji oraz wdrożone remedacje. Szczegóły mechanizmów uwierzytelniania, RBAC, JWT i walidacji wejścia opisano w sąsiednich podrozdziałach.

13.1 Założenia bezpieczeństwa

Przyjęto poziom ochrony adekwatny do akademickiego MVP demonstracyjnego (orientacyjnie ASVS L1): ochrona kont, egzekwowanie ról oraz ograniczenie dostępu do zasobów prywatnych użytkownika, przy świadomym odłożeniu pełnego hardeningu enterprise (WAF, 2FA, menedżer sekretów).

Założenia projektowe:

- bezpieczeństwo egzekwowane przede wszystkim po stronie API (Spring Security [5]), a nie wyłącznie w UI;
- zasada najmniejszych uprawnień: gość ma odczyt publiczny, kierowca — własne zasoby, organizator — własne wydarzenia, administrator — moderację platformy;
- obrona wielowarstwowa: Cloudflare (HTTPS/HSTS/Tunnel) $\rightarrow$ nginx (nagłówki bezpieczeństwa, proxy) $\rightarrow$ Spring Boot (JWT, rate limit, walidacja) $\rightarrow$ MySQL;
- sekrety i hasła nie powinny trafiać do repozytorium; w produkcji TLS kończy się na brzegu sieci (Cloudflare);
- decyzje konfiguracyjne (seed kont demo, domyślne sekrety) są dopuszczalne lokalnie, lecz niedopuszczalne na trwałej ekspozycji publicznej bez dodatkowej kontroli;

- cykl poprawy: odkrycie w teście penetracyjnym → odtworzenie → naprawa w kodzie → weryfikacja (testy automatyczne oraz krótka kontrola działania kluczowych funkcji na środowisku produkcyjnym).

13.2 Klasyfikacja danych

Dane przetwarzane przez RacePortal podzielono według wrażliwości i widoczności:

- **publiczne** — zatwierdzone treści wydarzeń (nazwa, opis, termin, lokalizacja, kategoria), publiczna lista startowa / opublikowane wyniki, katalog torów;
- **konto i profil** — e-mail, hasło (skrót BCrypt), rola, dane profilu, preferencje; dostęp tylko dla właściciela konta lub administratora w zakresie moderacji;
- **garaż** — pojazdy użytkownika (w tym opcjonalne multimedia); dostęp właściciela;
- **zgłoszenia** — statusy, notatki, dowody płatności; dostęp właściciela zgłoszenia oraz organizatora / administratora w ramach obsługi wydarzenia;
- **operacyjne** — refresh tokeny (skróty), logi / health, zgłoszenia błędów; ograniczone do mechanizmów systemowych i ról uprzywilejowanych.

Klasyfikacja uzasadnia, dlaczego część endpointów jest publiczna (katalog), a pozostałe wymagają sesji i kontroli właściciela lub roli.

13.3 Uwierzytelnianie użytkowników

Uwierzytelnianie RacePortal zrealizowano w module `AuthController` / `AuthService` bez klasycznej sesji serwerowej w pamięci. Tożsamość użytkownika opiera się na dwóch warstwach poświadczeń: krótkotrwałym tokenie dostępu JWT oraz rotowanym tokenie odświeżającym przechowywanym w bazie w postaci skrótu. Po stronie klienta webowego sesja jest utrzymywana w ciasteczkach HTTP-only (`rp_at`, `rp_rt`), ustawianych przez `AuthCookies` z parametrami `Secure` / `SameSite` zależnymi od środowiska.

Rejestracja lokalna tworzy konto z rolą `USER` w stanie niezweryfikowanym i wysyła link aktywacyjny e-mail. Do momentu weryfikacji system nie wydaje sesji — logowanie hasłem kończy się odmową z komunikatem o konieczności aktywacji. Hasła lokalne są hashowane algorytmem BCrypt [18] (`PasswordEncoder` w `SecurityConfig`). Polityka siły hasła wymaga co najmniej 8 znaków, wielkiej litery, cyfry i znaku specjalnego (wzorzec w `AuthDtos`). Ścieżka rejestracji organizatora równolegle zakłada wniosek o rolę, lecz sama nie nadaje uprawnień `ORGANIZER` — awans następuje dopiero po akceptacji administratora.

Logowanie hasłem stosuje ochronę czasową przed enumeracją kont: zarówno przy braku użytkownika, jak i przy złym haśle wykonywane jest porównanie BCrypt (w tym wobec hashu zastępczego), a komunikat błędu pozostaje jednolity. Dodatkowo `AttemptRateLimiter` ogranicza liczbę prób w oknie 15 minut — osobno per e-mail (5 prób) i per adres IP (20 prób); przekroczenie limitu zwraca HTTP 429 z nagłówkiem `Retry-After`. Opcjonalnie wspierane jest logowanie Google: weryfikacja `idToken` przy skonfigurowanym identyfikatorze klienta, po której system wydaje własną sesję RacePortal. Wylogowanie unieważnia refresh tokeny użytkownika, podnosi `tokenVersion` konta i czyści ciasteczka, co natychmiast unieważnia wcześniejsze access JWT.

13.4 Autoryzacja i kontrola dostępu RBAC

Kontrola dostępu opiera się o role domenowe `USER`, `ORGANIZER` i `ADMIN`, mapowane na authorities Spring Security w formie `ROLE_*` w `UserPrincipal`. Konfiguracja `@EnableMethodSecurity` umożliwia egzekwowanie reguł deklaratywnie przez `@PreAuthorize` na kontrolerach. Publicznie (bez logowania) dostępne są m.in. health-check, odczyt katalogu wydarzeń i torów, wyznaczanie trasy oraz wybrane endpointy auth (rejestracja, logowanie, weryfikacja, odświeżenie). Pozostałe żądania wymagają uwierzytelnienia.

Podział uprawnień odzwierciedla model biznesowy MVP. Zasób garażu (`/api/garage`) jest dostępny wyłącznie dla zalogowanego użytkownika i operuje wyłącznie na pojazdach należących do bieżącego konta. Zgłoszenia (`/api/registrations`) wymagają uwierzytelnienia; listowanie i zmiana statusów po stronie wydarzenia — roli organizatora lub administratora, z dodatkową weryfikacją własności wydarzenia w warstwie serwisów. Tworzenie i edycja wydarzeń wymaga `ORGANIZER` lub `ADMIN`. Panel `/api/admin` jest chroniony na poziomie całej klasy adnotacją `hasRole('ADMIN')` i obejmuje m.in. moderację wydarzeń, zarządzanie rolami oraz wnioskami o organizatora. W UI webowym bramki routingu i warunkowe renderowanie paneli stanowią drugą, uzupełniającą warstwę — ostatecznym źródłem prawdy pozostaje API.

13.5 Wykorzystanie tokenów JWT

Access token jest wystawiany przez `JwtService` (biblioteka JJWT) z podpisem HMAC-SHA opartym o sekret aplikacji (klucz wyprowadzany przez SHA-256 z `app.jwt.secret`). Domyślny czas życia access tokenu wynosi rzędu 15 minut i jest konfigurowalny. Token niesie m.in. identyfikator użytkownika, e-mail, nazwę użytkownika, rolę oraz claim `tv` (`tokenVersion`). Filtr `JwtAuthFilter` odczytuje token z nagłówka `Authorization: Bearer` albo z ciasteczka `rp_at`, weryfikuje podpis i termin ważności, a następnie ładuje użytkownika z bazy i porównuje `tv` z bieżącą wersją konta. Niezgodność wersji (np. po wylogowaniu lub zmianie hasła) czyści kontekst bezpieczeństwa — token staje się bezużyteczny mimo ważnego podpisu.

Refresh token jest osobnym artefaktem: surowa wartość trafia do ciasteczka `rp_rt` (ścieżka ograniczona do `/api/auth`), a w bazie przechowywany jest wyłącznie skrót. Przy odświeżeniu sesji następuje rotacja — stary refresh jest unieważniany, a wydawany jest nowy access i nowy refresh. Ponowne użycie już zrotowanego refresh tokenu skutkuje unieważnieniem wszystkich sesji użytkownika (ochrona przed kradzieżą tokenu). W warstwie webowej access JWT nie jest zapisywany w `localStorage`; starsze wartości w pamięci przeglądarki są usuwane przy starcie aplikacji. Dzięki temu typowy wektor XSS ma istotnie utrudniony dostęp do poświadczeń sesyjnych względem modelu opartego o `localStorage`.

```
return Jwts.builder()
    .subject(user.getId())
    .claim("id", user.getId())
    .claim("email", user.getEmail())
    .claim("username", user.getUsername())
    .claim("role", user.getRole().name())
    .claim("tv", user.getTokenVersion())
    .issuedAt(Date.from(now))
    .expiration(Date.from(now.plus(accessTtl)))
    .signWith(key)
    .compact();
```

Fragment kodu 13.1: Claimy access JWT wystawiane przez `JwtService`

13.6 Walidacja danych wejściowych

Wejście API jest walidowane dwustopniowo: deklaratywnie na rekordach DTO (Jakarta Bean Validation [19]) oraz regułami biznesowymi w serwisach. Kontrolery stosują `@Valid` na ciele żądania; naruszenia ograniczeń są mapowane przez `GlobalExceptionHandler` na odpowiedź HTTP 400 z mapą pole → komunikat w polu `details`, co pozwala frontowi wyróżnić konkretne błędy formularza.

W module auth wymagane są m.in. niepusty, unikalny e-mail (`@Email`), ograniczenia długości nazwy użytkownika oraz wzorzec silnego hasła. Pola profilu (imię, nazwisko, telefon, Instagram) mają osobne wzorce formatu. Analogiczne adnotacje stosują DTO garażu, wydarzeń, zgłoszeń i panelu administratora (statusy, role). Poza Bean Validation serwisy egzekwują reguły dziedzinowe — np. dozwolone przejścia statusów zgłoszenia, blokadę edycji pojazdu przy otwartym zgłoszeniu, unikalność telefonu czy bezpieczną normalizację URL mediów (`SafeHttpUrls` / `ImageReferences`). Upload plików dodatkowo weryfikuje rozmiar oraz typ po sygnaturze bajtowej (JPG/PNG/WebP, PDF), a nie wyłącznie po nagłówku przeglądarki. Dzięki temu walidacja obejmuje zarówno kontrakt składniowy żądania, jak i spójność stanu domenowego.

13.7 Ochrona zasobów użytkowników

Zasoby prywatne są wiązane z tożsamością użytkownika lub z własnością wydarzenia. W szczególności:

- garaż i „moje zgłoszenia” są dostępne wyłącznie dla właściciela konta;
- organizator zarządza zgłoszeniami i ustawieniami tylko swoich wydarzeń (nie cudzych);
- administrator ma uprawnienia moderacyjne na poziomie platformy, ale backoffice wymaga twardej roli `ADMIN`;
- upload plików (obrazy, PDF) wymaga uwierzytelnienia przy zapisie; odczyt ścieżek publicznych uploadów pozostaje kompromisem MVP i jest wskazywany jako ograniczenie do dalszego domknięcia.

Celem tej warstwy jest ograniczenie klasycznych błędów IDOR: użytkownik nie powinien odczytać ani zmodyfikować cudzego garażu, zgłoszenia ani panelu organizatora poprzez manipulację identyfikatorami w API.

13.8 Bezpieczeństwo komunikacji

W środowisku deweloperskim komunikacja lokalna odbywa się zwykle po HTTP wewnątrz Dockera / sieci LAN. W środowisku produkcyjnym ruch publiczny przechodzi przez Cloudflare Tunnel do nginx, a TLS (oraz HSTS) jest terminowany po stronie Cloudflare — przeglądarka łączy się z apexem i backoffice po HTTPS. W wyniku testów penetracyjnych wzmocniono spójność transportu: po stronie Cloudflare włączono Always Use HTTPS oraz HSTS, a w nginx wyłączono problematyczne absolutne przekierowania i ustawiono długi `max-age` HSTS.

Dodatkowo na brzegu aplikacji stosowane są nagłówki bezpieczeństwa nginx (m.in. CSP, `X-Frame-Options`). Po remedacjach CSP usunięto z produkcji odwołania do `localhost`, dodano m.in. `object-src`, `base-uri` i `form-action`, a inline boot przeniesiono do plików statycznych. CORS jest konfigurowany listą dozwolonych originów (z credentials dla ciasteczek sesji). Komunikacja SMTP do Mailbux odbywa się poza przeglądarką, po stronie backendu, z danymi dostępowymi trzymanymi w zmiennych środowiskowych.

13.9 Analiza zagrożeń

Dla RacePortal rozważono typowe zagrożenia aplikacji webowej z rolami i publicznym katalogiem, a następnie zweryfikowano je testami penetracyjnymi powierzchni HTTP(S) produkcji (konsultant: Paweł Maryszczuk, Senior SecOps). Kluczowe obszary zagrożeń:

- **uwierzytelnianie** — stuffing haseł, brute-force logowania, enumeracja kont po czasie odpowiedzi, przejęcie konta przy słabym sekrecie JWT lub aktywnym seedzie demo;
- **sesje** — kradzież długotrwałego tokenu z `localStorage`/JS, brak unieważnienia po wylogowaniu / zmianie hasła, reuse refresh tokenu;
- **autoryzacja / IDOR** — dostęp do cudzego garażu, zgłoszenia lub wydarzenia organizatora; podszywanie się pod zaufaną nazwę wyświetlaną;
- **XSS** — m.in. schemat `javascript:` w polach URL (wydarzenie, garaż, dowód płatności);
- **CSRF** — przy sesji cookie nadal temat uwagi przy dalszym hardeningu (`SameSite`, model API);
- **nadużycie zasobów** — automatyczne próby logowania/resetu, spam publicznych endpointów, nadmierne użycie uploadów;
- **ujawnienie informacji** — zbyt szczegółowy publiczny health, `robots.txt` mapujący panele, wersje serwera, tokeny w logach dostępowych.

Zagrożenia mapowano orientacyjnie na kategorie OWASP Top 10 [20] (m.in. A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A05 Security Misconfiguration, A07 Identification and Authentication Failures) oraz na wymagania ASVS [21].

13.10 Analiza ryzyka

Ryzyko oceniono jakościowo (prawdopodobieństwo $\times$ skutek) oraz praktycznie — przez odtworzenie znalezisk pentestu i pomiar, czy remedacja je zamyka:

- **wysokie skutki** — fałszowanie sesji administratora, masowy stuffing kont, stored XSS w treściach z URL; wymagają twardego sekretu JWT, wyłączenia seeda na ekspozycji publicznej, httpOnly cookies, walidacji URL oraz rate limitu;
- **średnie skutki** — enumeracja kont, zmiana e-maila bez re-autentykacji, luźna CSP, wyciek informacji z health/`robots.txt`; po remedacjach ryzyko spadło, ale wymaga utrzymania konfiguracji;
- **niższe / świadomie akceptowane w MVP** — brak 2FA i WAF, zależność od publicznych API map, publiczny odczyt części uploadów, model „zapłać później” bez PSP.

Dla środowiska prezentacyjnego ryzyko uznano za akceptowalne przy kontrolowanym użyciu kont demo. Dla trwałej ekspozycji publicznej kluczowe pozostaje utrzymanie bezpiecznej konfiguracji operatorskiej (sekret, seed, cookies `Secure`) — sam kod z remedacjami nie wystarczy, jeśli Compose nadpisze ustawienia niebezpiecznymi domyślnymi wartościami.

13.11 Ograniczenia przyjętych zabezpieczeń

Mimo przeprowadzonych testów penetracyjnych i wdrożonych poprawek model nie obejmuje pełnego hardeningu enterprise. Poza zakresem MVP pozostają m.in.:

- dwuskładnikowe uwierzytelnianie (2FA / WebAuthn) oraz step-up przy wrażliwych operacjach kont tylko-Google;
- dedykowany WAF / CAPTCHA oraz rozproszony rate limit (np. Redis / reguły Cloudflare);
- menedżer sekretów i formalna rotacja kluczy w stylu produkcyjnym;
- podpisane, czasowo ograniczone URL-e do prywatnych uploadów;
- pełny SAST oraz testy penetracyjne infrastruktury Oracle (poza powierzchnią HTTP aplikacji);
- gwarancje SLA publicznych serwisów map / routingu (OSRM).

Ograniczenia te nie negują wartości przeprowadzonego pentestu aplikacji: celem było domknięcie krytycznych luk warstwy web/API w cyklu odkrycie–naprawa–weryfikacja, a nie zbudowanie kompletnego SOC.

13.12 Testy penetracyjne i remedacje

Na potrzeby pracy wykonano testy penetracyjne produkcyjnej powierzchni HTTP(S) RacePortal (`https://raceportal.pl`). Konsultantem SecOps był Paweł Maryszczuk. Zakres obejmował transport, sesje, uwierzytelnianie, walidację wejścia oraz ograniczenie nadużyć; poza zakresem pozostały m.in. pełny SAST, testy infrastruktury VM oraz custom WAF.

Najważniejsze znaleziska wysokiej istotności oraz remedacje (październik 2026):

- **Transport / HSTS** — niespójne HTTPS/HSTS za Tunelem; remedacja: Always Use HTTPS + HSTS w Cloudflare oraz poprawki nginx (`absolute_redirect off`, długi HSTS).
- **Reset hasła** — tymczasowe hasło mogło zablokować konto; remedacja: jednorazowy token z TTL i zmiana hasła dopiero po linku z maila.

- **Sesje JWT** — ryzyko długotrwałego tokenu w JS; remedacja: krótki access, refresh w bazie (hash), cookies httpOnly, rotacja refresh, unieważnianie przy logout/zmianie hasła (`tokenVersion`).
- **Stored XSS przez URL** — akceptacja schematu `javascript:`; remedacja: serwerowa walidacja `SafeHttpUrls` (tylko http/https lub ścieżki uploadu).
- **Brute-force logowania** — brak limitu prób; remedacja: `AttemptRateLimiter` (limit per e-mail i IP, HTTP 429 + `Retry-After`).
- **Enumeracja kont po czasie** — różny czas odpowiedzi dla istniejącego/nieistniejącego e-maila; remedacja: wyrównanie kosztu (m.in. zawsze BCrypt, decoy mail przy forgot-password).

Znaleziska średnie i niższe objęły m.in. rezerwację/unikalność nazw użytkowników (ochrona przed podszywaniem), zaostrzenie CSP, wymóg hasła przy zmianie e-maila, ograniczenie publicznego `/api/health`, odchudzenie `robots.txt`, poprawę kodów błędów HTTP oraz ograniczenie wycieków informacyjnych (np. tokeny poza access logami ścieżek weryfikacji). Świadomie potwierdzono też model biznesowy „zapłać później" bez PSP — nie jako lukę implementacyjną, lecz decyzję zakresu MVP.

Weryfikacja remedacji łączyła testy automatyczne API (MockMvc) z krótką kontrolą działania kluczowych funkcji na środowisku produkcyjnym (nagłówki bezpieczeństwa, limity odpowiedzi 429, zachowanie resetu hasła i sesji). Szczegółowy rejestr znalezisk i commitów utrzymywany jest w dokumentacji projektowej [22].

Rozdział 14

Testowanie i weryfikacja systemu

14.1 Strategia testowania

Strategia testowania RacePortal opiera się na piramidzie testów i obejmuje kolejne warstwy architektury:

- **Testy jednostkowe (unit)** — wspólne typy (`packages/api-types`) oraz logika kliencka aplikacji mobilnej (adresy API/WEB).
- **Testy integracyjne backendu** — serwer Spring Boot, logika biznesowa oraz baza MySQL przy użyciu JUnit i MockMvc.
- **Testy End-to-End (E2E)** — zautomatyzowane scenariusze interfejsu webowego oraz podglądu mobilnego w Playwright [23].
- **Testy wydajnościowe (load)** — pomiary RPS narzędziem Grafana k6 [24] w środowisku chmurowym.

Szczegółowa metodyka i katalog przypadków testowych są opisane w `docs/testy/TESTY.md`.

14.2 Środowisko testowe

Środowisko testowe i demonstracyjne opiera się na konteneryzacji Docker Compose, grupującej serwer WWW (nginx), backend API (Spring Boot) oraz bazę MySQL 8.4. Opcjonalnie uruchamiany jest podgląd Expo na porcie `:8082`. Do pomiarów obciążenia wykorzystano maszynę produkcyjną Oracle Always Free (`VM.Standard.A1.Flex`, Frankfurt: 2 OCPU, 11 GiB RAM + 2 GiB swap), uruchamiając k6 lokalnie na węźle — tak, by oszacować wydajność *originu* przed tunelem Cloudflare [10].

14.3 Testy jednostkowe

Testy jednostkowe weryfikują kontrakty danych i niezmienniki. Biblioteką Vitest przetestowano pakiet `packages/api-types` (udokumentowany przebieg: 599 PASS / 0 FAIL w `docs/testy/wyniki/unit-500-wyniki.md`) oraz unit aplikacji mobilnej (3 przypadki: adresy API/WEB / host Expo). Dzięki temu niespójności typów wychwytywane są jeszcze przed integracją warstw.

14.4 Testy integracyjne backendu

Weryfikację warstwy biznesowej Spring Boot przeprowadzono przy użyciu JUnit oraz MockMvc. Scenariusze obejmują m.in.:

- procedury uwierzytelniania, mechanizmy tokenów JWT oraz ochronę przed nadmierną liczbą prób (*rate limiter*);
- operacje CRUD na zasobach wydarzeń motorsportowych;
- integralność relacji w wirtualnym garażu oraz mechanizm `carMatch`;
- obieg zgłoszeń i generowanie powiadomień e-mail;
- egzekwowanie RBAC dla ról USER, ORGANIZER oraz ADMIN.

W historii wyników odnotowano m.in. przebieg API 24/24 PASS. Liczba przypadków w `ApiIntegrationTest` mogła później wzrosnąć — aktualny werdykt należy odczytywać z `docs/testy/wyniki/` po lokalnym uruchomieniu.

14.5 Testy End-to-End aplikacji webowej

Automatyzacja testów interfejsu webowego realizowana jest frameworkiem Playwright [23]. Skrypty symulują ścieżki użytkownika (viewporty desktop i mobile), weryfikując m.in. ładowanie strony głównej, filtry katalogu, autoryzację, garaż oraz restrykcje dostępu do paneli ról.

Poniższy fragment przedstawia funkcję pomocniczą logowania oraz test `TC-WEB-07` (wejście kierowcy na dashboard po autoryzacji). Placeholder hasła w kodzie to maska ośmiu kropek (w listingu: `********`).

```
async function loginAs(page: Page, email: string, password: string) {
  await page.goto("/login");
  await page.getByPlaceholder("twoj@email.com").fill(email);
  await page.getByPlaceholder("********").fill(password);
  await Promise.all([
    page.waitForURL((url) => !url.pathname.endsWith("/login"),
      { timeout: 20000 }),
    page.locator('main button[type="submit"]').click(),
  ]);
```

```
10 }
11
12 test.describe("Web --- auth i panel kierowcy", () => {
13   test("TC-WEB-07: logowanie kierowcy i wejście na dashboard",
14     async ({ page }) => {
15     await loginAs(page, "test@wp.pl", "test123");
16     await expect(page).toHaveURL(/\/dashboard/);
17     await expect(page.locator("main")
18       .getByText(/test|KIEROWCA|dashboard|profil|garaż/i)
19       .first()).toBeVisible();
20   });
21 });
```

Fragment kodu 14.1: Funkcja `loginAs` oraz test `TC-WEB-07` (Playwright).

14.6 Testy aplikacji mobilnej

Testy aplikacji mobilnej (Expo) obejmują unit weryfikujący adresy URL API oraz WEB, a także scenariusze E2E w trybie podglądu webowego. Sprawdzane są m.in. pobieranie listy wydarzeń z API, przełączanie widoków (Lista / Kalendarz / Mapa) oraz przekierowanie do pełnej wersji webowej (CTA).

14.7 Testy mechanizmów bezpieczeństwa

W ramach weryfikacji odporności systemu przetestowano m.in.:

- odporność endpointów administracyjnych na nieautoryzowany dostęp przez użytkownika z rolą kierowcy (USER);
- walidację DTO po stronie backendu (odrzucanie niepoprawnego formatu danych wejściowych).

14.8 Automatyzacja uruchamiania testów

Testy automatyczne są uruchamiane skryptami npm oraz bash w katalogu `scripts/` (m.in. `test:unit`, `test:api`, `test:e2e`, `test:mobile-unit`). Raporty zapisuje się w `docs/testy/wyniki/`; sposób odbioru opisuje `docs/testy/TESTY.md`.

14.9 Analiza wyników testów i testy wydajnościowe k6

Werdykty przebiegów funkcjonalnych i jednostkowych są odnotowywane w dokumentacji wyników (`docs/testy/wyniki/`); regresie wychwytywane są przed prezentacją. Ostatni zapisany skrót sesji z 14.09.2026 obejmuje m.in. PASS dla web build, mobile unit (3/3) oraz api-types

(599/599); API i E2E były wtedy zablokowane brakiem Dockera — nie cytuje się tu niepotwierdzonych liczników E2E.

Uzupełnieniem weryfikacji jakościowej były pomiary przepustowości z 25.10.2026 narzędziem Grafana k6 [24] bezpośrednio na produkcyjnej VM Oracle Cloud. Cel: oszacować maksymalną przepustowość (RPS) stacku przy ruchu na `localhost` (origin za Cloudflare Tunnel, bez narzutu publicznej bramki na czas pomiaru). Metodyka: *ramping arrival-rate* oraz *constant arrival-rate* (ok. 30 s) dla kluczowych zasobów. Pełne tabele i interpretacja: `docs/testy/wyniki/k6-vm-rps.md` [10].

Tabela 14.1: Skrót wyników k6 na Oracle VM (25.10.2026) — komfort vs sufit.

Warstwa	**Endpoint**	**Komfort RPS**	**Sufit**
API lekkie	`GET /api/health`	∼2000 (p95 ∼116 ms)	∼2200–2260
Front static	`GET /` (nginx :8081)	∼2000 (p95 ∼23 ms)	∼3250
API + MySQL	`GET /api/events`	∼150–200 (p95 ∼4–22 ms)	∼250–300

W porównaniu warstw: dla `health` i statycznego frontu uzyskano ok. 2000 żądań na sekundę; dla paginowanej listy wydarzeń (`GET /api/events?page=0&size=10`) — około 150–200 żądań na sekundę (przy 50 RPS: 0% błędów, p95 ok. 4,5 ms). Przy forsowaniu listy powyżej ok. 300 RPS pojawia się saturacja i degradacja. Raport porównuje te wyniki i wskazuje, że w warunkach pomiaru ograniczenie dotyczy endpointu listy wydarzeń (API z MySQL), a nie nginx ani samego health — bez wyłączania innych czynników (w tym współdzielonych 2 OCPU maszyny, na której k6 konkurował ze stackiem). Dla typowego ruchu akademickiego i demonstracyjnego (dziesiątki współbieżnych użytkowników) zapas pozostaje duży.

14.10 Ograniczenia procesu testowania

W obecnej wersji MVP proces testowania ma świadome ograniczenia:

- brak pełnych testów natywnych mobilnych (np. Detox lub Maestro) — E2E mobilki przez Expo web;
- pomiary k6 skupiły się na zasobach odczytowych i istniejących danych; nie wykonano formalnego scenariusza odbiorczego KP/DZW „ok. 10 000 rekordów w katalogu / 50 RPS” jako zautomatyzowanego wymogu odbiorczego;
- k6 działał na tej samej VM co stack (konkurencja o 2 OCPU), więc wynik to dolna granica pojemności originu — szczegóły w raporcie [10].

Rozdział 15

Prezentacja systemu

Rozdział porządkuje sposób pokazania RacePortal na obronie i w odbiorze MVP: najpierw mapę funkcjonalności oraz warunki demonstracji, a następnie typowe ścieżki ról (kierowca, organizator, administrator). Szczegółowy scenariusz czasowy utrzymywany jest w dokumentacji projektowej [25].

15.1 Prezentacja głównych funkcjonalności

Celem prezentacji jest wykazanie, że RacePortal działa jako spójny system — a nie zestaw osobnych mocków — na wspólnej powierzchni produkcyjnej lub lokalnej. Demonstracja opiera się na środowisku `https://raceportal.pl` (Oracle Always Free + Cloudflare Tunnel [11]) albo na stacku `docker compose` [15] (web, API, MySQL, backup) uruchomionym lokalnie. Przed pokazem weryfikowany jest health API (`/api/health`) oraz dostępność kont demonstracyjnych (seed).

Mapa funkcjonalności pokazywanych w MVP

W toku prezentacji eksponowane są następujące bloki produktu (zgodnie z zakresem MVP [9]):

- **katalog publiczny** — trzy widoki wydarzeń (lista, mapa Leaflet [17], kalendarz) na wspólnych filtrach (kategoria, województwo, data, nazwa); wejście bez logowania;
- **szczegóły wydarzenia** — opis, wymagania udziału, opcjonalne załączniki regulaminu, wyznaczanie trasy dojazdu (OSRM [16]);
- **konto i role** — rejestracja/logowanie, JWT + RBAC (`USER` / `ORGANIZER` / `ADMIN`); wniosek o rolę organizatora;
- **garaż i** `carMatch` — pojazdy kierowcy z parametrami; dopasowanie i filtr katalogu „pod moje auto”;

- **zgłoszenia** — składanie zapisu, statusy (`PENDING` / `ACCEPTED` / `CONFIRMED` / `CANCELED`), model „zapłać później” z dowodem przelewu; maile transakcyjne (Mailbux);
- **panel organizatora** — tworzenie/edycja wydarzenia (w tym wymagania techniczne), obsługa zgłoszeń, lista startowa / wyniki w zakresie MVP;
- **panel administratora / backoffice** — moderacja wydarzeń i wniosków o rolę, zarządzanie rolami kont;
- **ścieżka mobilna** — PWA weba oraz wizytówka Expo z CTA do zapisu na webie (bez obiecywania parytetu funkcji ze sklepami aplikacji).

Kolejność narracji prezentacji

Przyjęto układ dwuczęściowy: krótki kontekst problemowy i architektoniczny, a następnie demonstracja na żywo. W części merytorycznej podkreślane są: rozproszenie informacji o imprezach motorsportowych w Polsce, przewaga zintegrowanego garażu z `carMatch` oraz trójwarstwowy stack (React/Vite [1, 2] $\rightarrow$ Spring Boot [4] $\rightarrow$ MySQL [6]) z konteneryzacją i wystawieniem HTTPS na brzegu Cloudflare. W części demonstracyjnej na żywo unika się długiej rejestracji „od zera” (zależność od SMTP) na rzecz kont demonstracyjnych (seed) oraz pomija się wykład o szczegółach JWT — te mechanizmy opisano w rozdz. 13.

Typowy przebieg pokazów funkcjonalnych (szczegóły ról — kolejne podrozdziały):

1. **gość** — strona główna i katalog w trzech widokach z filtrami; wejście w detal wydarzenia;
2. **kierowca** — garaż, filtr pod pojazd, trasa, zgłoszenie, podgląd statusu na dashboardzie;
3. **organizator** — utworzenie lub wybór wydarzenia, panel zgłoszeń, zmiana statusu z potwierdzeniem mailowym;
4. **administrator** (gdy pozwala czas) — moderacja publikacji / wniosku organizatora;
5. **domknięcie** — PWA lub skrót mobilny CTA oraz uczciwe wskazanie świadomych ograniczeń MVP (brak PSP, brak sklepów aplikacji, Expo jako wizytówka).

Kryteria udanej demonstracji

Prezentację uznaje się za udaną, gdy widzowie mogą potwierdzić: (1) publiczny katalog działa bez konta; (2) kierowca przechodzi spójną ścieżkę garaż $\rightarrow$ dopasowanie $\rightarrow$ zapis; (3) organizator steruje statusami zgłoszeń w swoim wydarzeniu; (4) system jest uruchomiony jako złożony stack (Compose / produkcja), a nie pojedyncza strona statyczna. Plan awaryjny obejmuje krótkie nagranie ścieżki krytycznej oraz pominięcie opcjonalnych scen (admin, rejestracja z mailem), jeśli infrastruktura lub SMTP są niestabilne [25].

15.2 Scenariusz użycia z perspektywy kierowcy

Login seed → garaż → filtr/katalog → zapis → dashboard statusów.

15.3 Scenariusz użycia z perspektywy organizatora

Login org → wydarzenie (presety torów) → zgłoszenia → zmiana statusu → podgląd maila na skrzynce (Mailbux).

15.4 Scenariusz użycia z perspektywy administratora

Login admin → moderacja wydarzenia / wniosku organizatora / ról.

15.5 Podsumowanie działania systemu

System realizuje spójny obieg informacji o wydarzeniach i zapisach w środowisku uruchamialnym lokalnie oraz pod adresem produkcyjnym za Cloudflare Tunnel.

Rozdział 16

Nakład pracy

16.1 Liczba godzin przeznaczona na realizację projektu

Nakład sumaryczny (osobogodziny): **[do uzupełnienia przez zespół]**.

16.2 Podział zadań w zespole

Skrót obszarów — rozdz. 17. Szczegółowa tabela godzin per osoba: **[do uzupełnienia przez zespół]**.

16.3 Narzędzia wspierające zarządzanie pracą

Git/GitHub, dokumentacja Markdown w `docs/`, Docker, Cursor/IDE, Playwright [23], Figma Make, Cloudflare Dashboard; skład pracy: LaTeX [26].

16.4 Problemy organizacyjne i ich rozwiązanie

M.in. rozrost zakresu mobile — decyzja o wizytówce; luki Spec — rejestr `spec-conformity.md`; stabilizacja testów względem seedu/archiwizacji; uproszczenie hostingu (Oracle + Tunnel zamiast GCP Pages).

Rozdział 17

Wkład własny

Rozdział opisuje, kto odpowiadał za które obszary prac projektowych i implementacyjnych (imię i nazwisko). Podział odzwierciedla faktyczny udział zespołu; największy wkład merytoryczny i projektowy w warstwie produktu/UI wniosła Natalia Otrombke.

17.1 Natalia Otrombke

- projekt UI/UX w Figma Make (**Advanced UI/UX for RACEPORTAL**) — ekrany katalogu, auth, dashboard, paneli;
- przeniesienie i dopracowanie frontendu React (motyw, layout, spójność marki, kluczowe ścieżki UX);
- udział w definicji flow użytkownika (gość $\rightarrow$ kierowca $\rightarrow$ organizator) oraz w materiałach koncepcyjnych ról.

17.2 Oliwier Kasprowicz

- prace analityczne i dokumentacyjne (wymagania, prezentacje zespołowe);
- wsparcie przy diagramach i scenariuszach użycia;
- udział w przeglądach zakresu MVP i odbiorze funkcjonalności.

17.3 Michał Gutowski

- odpowiedzialność za projektowanie architektury procesów, tworzenie zaawansowanych materiałów projektowych oraz diagramów modelujących system (m.in. rich picture, szczegółowe diagramy przepływów);

- zbieranie i analiza wymagań funkcjonalnych poprzez bezpośredni kontakt z docelowymi użytkownikami platformy (kierowcami oraz organizatorami wydarzeń motorsportowych) w celu pozyskania kluczowej wiedzy dziedzinowej;
- aktywny udział w pracach programistycznych nad warstwą backendową systemu (implementacja logiki biznesowej API);
- wsparcie w tworzeniu dokumentacji projektowej oraz opracowywanie i testowanie scenariuszy demonstracyjnych przed wdrożeniem;
- udział w integracji komponentów oraz weryfikacji poprawności głównych ścieżek użytkownika.

17.4 Miłosz Parkitny

- modelowanie i diagramy (m.in. przypadki użycia, podział funkcjonalności, model danych);
- wsparcie analizy interesariuszy i harmonogramów;
- udział w pracach implementacyjnych / poprawkach w repozytorium.

17.5 Wojciech Wronisz

- pełnienie roli kierownika projektu w Dokumentacji Założeń Wstępnych (DZW) oraz — w praktyce repozytorium — odpowiedzialność za obszar fullstack, testów automatycznych i bezpieczeństwa aplikacji;
- prace nad warstwą API Spring Boot [4]: kontrakt REST współdzielony z webem i mobile, integracja z MySQL, inicjalizacja danych demonstracyjnych (`DataInitializer` / seed), wysyłka maili transakcyjnych (SMTP Mailbux);
- projekt i utwardzenie uwierzytelniania oraz autoryzacji: JWT [14] (access/refresh, `tokenVersion`), ciasteczka httpOnly (`rp_at`/`rp_rt`), RBAC ról `USER`/`ORGANIZER`/`ADMIN`, Spring Security [5];
- remedacje po testach penetracyjnych powierzchni HTTP(S) produkcji [22]: m.in. limity prób logowania (`AttemptRateLimiter`), walidacja URL (`SafeHttpUrls`), spójność HSTS/HTTPS za Cloudflare Tunnel, wyrównanie kosztu odpowiedzi przy logowaniu/resecie hasła, zaostrzenie CSP i ograniczenie wycieków informacyjnych;

- konteneryzacja i wdrożenie: Docker Compose [15] (web, API, MySQL, backup), produkcja na Oracle Always Free z Cloudflare Tunnel [11] i nginx, konfiguracja domeny `https://raceportal.pl` oraz backoffice;
- ścieżka mobilna i PWA: wizytówka Expo (katalog + CTA do weba), instalowalność weba jako PWA, utrzymanie spójności kontraktu typów (`packages/api-types`);
- weryfikacja jakości: testy integracyjne API (JUnit/MockMvc), E2E Playwright [23], pomiary obciążenia Grafana k6 [24, 10] na VM oraz dokumentacja uruchomienia, wdrożenia i odbioru w `docs/` (m.in. scenariusz prezentacji [25]);
- udział w redakcji pracy dyplomowej (LaTeX [26]): rozdziały dotyczące SWOT, projektu/realizacji, mobile, bezpieczeństwa (w tym pentest), prezentacji MVP, a także uzupełnianie słownika pojęć i bibliografii.

Rozdział 18

Podsumowanie

18.1 Największe wyzwania i sposób ich rozwiązania

Podczas realizacji i wdrożenia produkcyjnego systemu RacePortal zespół napotkał szereg istotnych wyzwań inżynierskich, zwłaszcza w obszarze infrastruktury, konfiguracji bezpiecznego tunelowania oraz zabezpieczeń sieciowych. Do najważniejszych z nich należały:

- **Wyzwania związane z wdrożeniem (deployment) i architekturą chmurową:** Początkowe próby rozproszenia aplikacji (np. frontend hostowany na zewnętrznych platformach typu Pages, a backend na maszynie wirtualnej) generowały problemy z konfiguracją polityk CORS, routingu oraz spójności środowiskowej. Wyzwaniem było również zapewnienie prostoty uruchomienia produkcyjnego na darmowej maszynie wirtualnej (Oracle Cloud Always Free ARM). *Sposób rozwiązania:* Zdecydowano się na pełną konteneryzację całego środowiska za pomocą Docker Compose (`web`, `api`, `mysql`, `backup`) oraz sprowadzenie infrastruktury produkcyjnej do jednej, spójnej maszyny wirtualnej zarządzanej kodem konfiguracyjnym.

- **Konfiguracja bezpiecznego tunelowania w Cloudflare:** Wystawienie lokalnie skonteneryzowanej aplikacji na publiczny internet przy jednoczesnym zachowaniu wysokich standardów bezpieczeństwa i szyfrowania bez konieczności skomplikowanego zarządzania certyfikatami SSL bezpośrednio na serwerze stanowiło duże wyzwanie sieciowe. *Sposób rozwiązania:* Zastosowano technologię **Cloudflare Tunnel** (`cloudflared`). Tunel bezpiecznie spina maszynę wirtualną z brzegiem sieci Cloudflare, przejmując całkowitą obsługę rekordów DNS, terminację protokołu TLS (HTTPS) oraz ochronę przed atakami DDoS (WAF). Dzięki temu ruch do lokalnego serwera Nginx (działającego na porcie `:8081`) jest w pełni szyfrowany i odizolowany od bezpośrednich skanów portów z zewnątrz.

- **Zabezpieczenia strony pod kątem cybersecurity i ochrony zasobów:** Projektując platformę obsługującą dane użytkowników, konta ról oraz pojazdy, należało zabezpieczyć

system przed typowymi zagrożeniami webowymi (takimi jak próby przejęcia sesji, ataki typu brute-force na panele logowania czy nieuprawniony dostęp do zasobów innych użytkowników — IDOR). *Sposób rozwiązania:* Wdrożono bezstanowe uwierzytelnianie oparte o tokeny JSON Web Token (JWT) w bezpiecznych ciasteczkach / nagłówkach oraz kontrolę dostępu opartą o role (RBAC) na poziomie kontrolerów API. Dodatkowo kluczowe ścieżki uwierzytelniania zabezpieczono limitowaniem częstotliwości zapytań (rate limiting), a na warstwie serwera proxy (nginx) ustawiono nagłówki bezpieczeństwa, w tym politykę ochrony treści (CSP) oraz nagłówki zapobiegające osadzaniu strony w ramkach (X-Frame-Options).

18.2 Zmiany i adaptacje w trakcie realizacji

Realizacja RacePortal nie przebiegała jako sztywne wykonanie pierwotnego planu „punkt po punkcie". Zakres MVP [9], ograniczenia budżetowe oraz wnioski z integracji i testów wymusiły kilka świadomych adaptacji. Poniżej zestawiono najważniejsze zmiany względem wczesnych założeń (KP/DZW, pierwsze prototypy UI i eksperymenty infrastrukturalne) wraz z uzasadnieniem.

- **Aplikacja mobilna: od równoważności funkcji do wizytówki.** Wczesne dokumenty i szkice zakładały klienta mobilnego zbliżonego funkcjonalnie do weba (konto, garaż, zapis). W toku prac okazało się, że utrzymanie pełnej równoważności funkcji na Expo i na React jednocześnie przekracza ramy czasowe zespołu oraz grozi rozjazdem kontraktu API. Przyjęto model wizytówki: katalog (lista / mapa / kalendarz) + CTA do zapisu na webie, a pełną ścieżkę transakcyjną domknięto w PWA weba. Efektem jest działająca demonstracja mobilna bez obietnicy publikacji w sklepach aplikacji.

- **Hosting: od rozproszonego stacku do jednej VM z Tunelem.** Pierwsze eksperymenty (m.in. frontend na Cloudflare Pages / podobnych platformach, backend osobno) generowały problemy CORS, rozjazdu originów i złożoności wdrożenia. Zespół przeszedł na jeden Compose [15] na Oracle Always Free oraz publiczny dostęp przez Cloudflare Tunnel [11] (apex `https://raceportal.pl`, backoffice na osobnym hoście). Deploy sprowadza się do aktualizacji kodu i przebudowy kontenerów; TLS/HSTS terminowane są na brzegu Cloudflare.

- **Sesje: od JWT w `localStorage` do ciasteczek httpOnly.** Wczesny klient webowy przechowywał token w `localStorage`. Po analizie zagrożeń i testach penetracyjnych [22] sesję przeniesiono do ciasteczek `rp_at`/`rp_rt` (httpOnly, rotacja refresh, `tokenVersion`), co ogranicza skutki XSS względem modelu opartego o JavaScriptowy odczyt tokenu.

- **Wspólny kontrakt typów (`packages/api-types`).** Rozrost monorepo (web + mobile + API) ujawnił ryzyko rozjazdu pól JSON. Wprowadzono współdzielony pakiet TypeScript [3] generujący/utrzymujący typy kontraktu REST, co zmniejszyło liczbę błędów integracyjnych i uprościło testy jednostkowe po stronie klientów.

- **Mapy i trasa: Leaflet + OSRM jako domyślny wariant.** KP/DZW wskazywały Google Maps API jako integrację mapową. W MVP domyślnie przyjęto Leaflet [17] (markery katalogu) oraz publiczny OSRM [16] do trasy dojazdu, z opcją Google przy skonfigurowanym kluczu. Zmiana ograniczyła koszty i zależność od płatnego klucza przy zachowaniu demonstracyjnej wartości mapy.

- **Płatności: model „zapłać później" zamiast natychmiastowego PSP.** Zamiast wdrażać bramkę płatności online w ramach akademickiego MVP pozostawiono statusy zgłoszeń i opcjonalny dowód przelewu po stronie organizatora. Decyzja wynika z kosztów onboardingu PSP, wymogów prawnych i priorytetu domknięcia katalogu oraz zapisów.

- **Media: od samych URL-i do lokalnego uploadu z limitami.** Wczesne założenie „same odnośniki zewnętrzne" rozszerzono o upload JPG/PNG/WebP/PDF na dysk serwera (garaż, grafika wydarzenia, regulamin, dowód płatności), z weryfikacją typu po sygnaturze i limitem rozmiaru. Jednocześnie pozostawiono możliwość wskazania bezpiecznego URL http(s) — kompromis między wygodą demonstracji a brakiem zewnętrznego object storage.

- **Doprecyzowanie formularzy i zgodności ze Spec.** W trakcie implementacji ujawniono różnice między dokumentami wymagań a faktycznym kontraktem pól (wymagane vs opcjonalne). Zespół prowadzi rejestr zgodności [27] i iteracyjnie dopinał walidację DTO oraz UI, zamiast „na siłę" odtwarzać każdy historyczny zapis Spec bez odniesienia do MVP.

- **Bezpieczeństwo jako pętla remedacji, nie jednorazowa checklista.** Po wystawieniu produkcji przeprowadzono testy penetracyjne i wdrożono remedacje (HSTS, rate limit, SafeHttpUrls, timing-safe login, CSP itd.) w cyklu odkrycie $\rightarrow$ naprawa $\rightarrow$ weryfikacja [22]. To zmiana procesu: konfiguracja operatorska (sekret JWT, seed na ekspozycji publicznej) stała się częścią odbioru, a nie wyłącznie tematem kodu.

Adaptacje te nie zmieniają celu produktu (centralny katalog i ujednolicone zapisy), lecz porządkują sposób dojścia do działającego MVP w ograniczeniach czasu, budżetu i bezpieczeństwa opisanych w sąsiednich podrozdziałach.

18.3 Ograniczenia końcowej wersji systemu

Oddawana wersja RacePortal domyka założony zakres MVP: publiczny katalog wydarzeń, konta i role, wirtualny garaż, proces zgłoszeń ze statusami, panele organizatora i administratora, powiadomienia e-mail, lokalny upload mediów oraz mobilną wizytówkę katalogu. Jednocześnie praca inżynierska wymaga jasnego wskazania granic rozwiązania — zarówno tych wynikających ze świadomego zawężenia zakresu, jak i tych wynikających z przyjętego modelu wdrożenia demonstracyjnego. Opis poniżej jest spójny z kartą projektu oraz z podrozdziałem 1.1; nie stanowi katalogu usterek, lecz zestaw ograniczeń, których zespół nie deklarował jako zrealizowanych.

Najistotniejszym ograniczeniem biznesowym pozostaje brak natywnej bramki płatności online (PSP). System umożliwia oznaczanie statusów opłat i dołączanie potwierdzenia przelewu, lecz nie pobiera składek startowych, nie rozlicza zwrotów i nie prowadzi księgowości wydarzenia. Integracja PSP została odłożona ze względu na koszty onboardingu, wymogi identyfikacji płatnika oraz odpowiedzialność prawną nieproporcjonalną do celów akademickiego prototypu.

W obszarze treści i plików końcowa wersja wykracza poza wczesne założenie „same referencje URL". Aplikacja webowa oraz API umożliwiają lokalny upload obrazów (JPG, PNG, WebP) i dokumentów PDF z limitem rozmiaru oraz weryfikacją typu po sygnaturze pliku. Mechanizm ten obsługuje m.in. zdjęcia pojazdów w garażu, grafikę wydarzenia, regulaminy PDF oraz opcjonalny dowód płatności przy zgłoszeniu; równolegle nadal dopuszczalne jest wskazanie zewnętrznego adresu http(s). Ograniczeniem pozostaje natomiast warstwa infrastrukturalna przechowywania: pliki trafiają na dysk serwera aplikacji, bez zewnętrznego object storage, bez skanowania antywirusowego, bez polityk retencji i bez podpisanych odnośników o czasowo ograniczonym dostępie. Nie zaimplementowano także galerii albumów po imprezie ani uploadu mediów z klienta mobilnego, który w MVP pełni wyłącznie rolę wizytówki.

Komunikacja z użytkownikiem opiera się na poczcie elektronicznej (potwierdzenia rejestracji, weryfikacja konta, zmiany statusów zgłoszeń) oraz uzupełniających powiadomieniach w aplikacji webowej. Brakuje natywnych powiadomień push oraz narzędzi do kampanii marketingowych. Świadomie zawężono również zakres aplikacji mobilnej: klient Expo udostępnia przegląd katalogu (lista, mapa, kalendarz) z przekierowaniem do serwisu webowego w celu zapisu, lecz nie obejmuje logowania, garażu, zgłoszeń ani paneli ról i nie został opublikowany w sklepach App Store / Google Play. Decyzja ta pozwoliła domknąć pełną ścieżkę zapisu po stronie weba i PWA zamiast dublować ten sam zakres na dwóch klientach.

Zależności zewnętrzne dotyczą przede wszystkim map i tras. Domyślne wyznaczanie trasy korzysta z publicznego OSRM (z możliwością Google Maps API przy kluczu), co oznacza brak gwarancji dostępności i SLA poza kontrolą zespołu. Katalog torów jest seedowany i rozszerzalny, lecz nie pretenduje do roli oficjalnego, kompletnego rejestru obiektów motorsportowych

w Polsce. Podobnie kompletność oferty wydarzeń zależy od aktywności organizatorów i moderacji administratora — platforma nie scrapuje automatycznie zewnętrznych źródeł informacji.

W warstwie bezpieczeństwa przyjęto standard adekwatny do MVP demonstracyjnego: uwierzytelnianie JWT w ciasteczkach HTTP-only, kontrola dostępu RBAC, ograniczanie liczby prób, walidacja wejścia oraz terminacja TLS/HSTS na Cloudflare Tunnel przed aplikacją. Poza zakresem pozostają m.in. pełny program testów penetracyjnych, zaawansowany WAF, dedykowany menedżer sekretów oraz własny certyfikat TLS utrzymywany bezpośrednio na hoście aplikacji. Są to ograniczenia dojrzałości operacyjnej, a nie brak podstawowych mechanizmów ochrony kont i zasobów opisanych w rozdziale poświęconym bezpieczeństwu.

Wydajność stacku produkcyjnego została zweryfikowana pomiarami Grafana k6 [24] na Oracle VM (25.10.2026); raport i tabele wyników znajdują się w `docs/testy/wyniki/k6-vm-rps.md`. Dlatego ograniczeń wydajnościowych (RPS) nie ujmuje się tu jako otwartej luki odbiorczej — należą one do udokumentowanych wyników testowania, a nie do listy niedomkniętych ograniczeń MVP.

18.3.1 Ograniczenia budżetowe

Projekt realizowano przy budżecie bliskim zeru, typowym dla akademickiego MVP. Hosting produkcyjny oparto o Oracle Always Free (maszyna ARM z limitami OCPU i pamięci), publiczny dostęp — o Cloudflare Tunnel z terminacją TLS po stronie Cloudflare, a pocztę — o SMTP Mailbux w konfiguracji niskokosztowej. Świadomie unikano płatnych bramek PSP, komercyjnego object storage, osobnego WAF oraz płatnych planów mapowych jako warunków działania systemu. Skutkiem budżetowym jest m.in. lokalny upload na dysk VM zamiast S3/MinIO, zależność od publicznych lub opcjonalnie konfigurowanych usług map oraz brak środków na ciągły monitoring klasy APM. Ograniczenie budżetowe nie unieważnia działania MVP, lecz wyznacza pułap skalowania i poziom utwardzenia infrastruktury możliwy do utrzymania bez stałych kosztów licencyjnych.

18.3.2 Ograniczenia czasowe

Ramy czasowe wynikały z kalendarza pracy inżynierskiej i organizacji zespołu pięcioosobowego: konieczność domknięcia zakresu MVP przed terminem odbiorczym karty projektu, równoległa praca nad dokumentacją, testami i wdrożeniem oraz historyczne okno przedmiotu PRO (ok. 15 tygodni) jako kontekst planistyczny, a nie formalny Gantt dyplomu. Ograniczenie czasu wymusiło priorytety — pełna ścieżka zapisu na webie i PWA kosztem równoważności funkcji aplikacji mobilnej, odłożenie PSP i sklepów mobilnych, a także świadome zawężenie hardeningu produkcyjnego. Termin karty projektu (31.01.2026) traktowano jako ograniczenie planistyczne odbioru akademickiego, natomiast faktyczne domknięcie funkcji i pomiarów dokumen-

towano w `docs/changes.md`. W efekcie końcowa wersja jest kompletna względem zadeklarowanego MVP, lecz nie obejmuje planu rozwoju wykraczającego poza ten horyzont czasowy.

18.4 Możliwości dalszego rozwoju

MVP RacePortal realizuje założone cele funkcjonalne i procesowe, ale zostawia miejsce na dalszą rozbudowę. Na podstawie doświadczeń z wdrożenia, uwag użytkowników oraz braków obecnej wersji wskazano m.in. następujące kierunki:

- **Integracja z profesjonalnymi bramkami płatności (PSP):** Wprowadzenie automatycznych płatności online (np. przelewy natychmiastowe, BLIK, karty płatnicze) zamiast obecnego mechanizmu weryfikacji na podstawie przesyłanego adresu URL dowodu wpłaty. Umożliwi to pełną automatyzację rozliczeń finansowych między kierowcami a organizatorami oraz natychmiastowe potwierdzanie opłacenia wpisowego.
- **Autorski pomiar czasowy i wyniki na żywo:** Wdrożenie dedykowanego modułu pomiaru czasu dla organizatorów z możliwością synchronizacji urządzeń pomiarowych bezpośrednio z aplikacją, co umożliwi uczestnikom oraz kibicom podgląd oficjalnych wyników zawodów na żywo.
- **Zaawansowane zarządzanie listami rezerwowymi i waitlistą:** Rozbudowa logiki biznesowej zgłoszeń o automatyczne zarządzanie kolejką rezerwową (*waitlist*). W przypadku rezygnacji jednego z uczestników, system mógłby automatycznie powiadamiać i przesuwać na listę główną kolejnego kierowcę z listy rezerwowej według zdefiniowanych kryteriów.
- **Pełnoprawna aplikacja mobilna i publikacja w sklepach:** Rozszerzenie obecnej wizytówkowej aplikacji mobilnej (Expo) o pełną funkcjonalność konta użytkownika, w tym mobilny wirtualny garaż, składanie zgłoszeń oraz bezpośredni wgląd do statusów, a także przygotowanie aplikacji do oficjalnej dystrybucji w sklepach Google Play oraz App Store.
- **Podstrona Auta:** Stworzenie dedykowanej, publicznej strony każdego samochodu w systemie, prezentującej jego szczegółowe dane techniczne, galerię zdjęć oraz pełną historię startów w ubiegłych i nadchodzących wydarzeniach motorsportowych.
- **Podstrona Kierowcy:** Implementacja publicznego profilu zawodnika, który w jednym miejscu agreguje jego osiągnięcia sportowe, zestawienie posiadanych aut z wirtualnego garażu oraz odnośniki do mediów społecznościowych.

- **Podstrona Obiektów Motorsportowych:** Rozbudowa systemu o kompletny katalog wszystkich torów i obiektów w Polsce, zawierający ich opisy, parametry techniczne oraz precyzyjną lokalizację na zintegrowanej mapie.
- **Podstrona Organizatora:** Wprowadzenie dedykowanego profilu publicznego dla klubów oraz organizatorów wydarzeń, prezentującego ich historię, opis działalności oraz zestawienie organizowanych przez nich imprez motorsportowych.

18.5 Wnioski końcowe

RacePortal to działające MVP katalogu wydarzeń motorsportowych. System pozwala przeglądać imprezy, prowadzić garaż, składać zgłoszenia oraz obsługiwać role kierowcy, organizatora i administratora. Całość uruchamia się w Docker Compose i nadaje się do prezentacji dyplomowej.

Projekt ma też jasne granice. Nie ma płatności online — rozliczenie wpisowego zostaje po stronie organizatora. Aplikacja mobilna pokazuje głównie katalog i kieruje do zapisu w wersji webowej; nie ma w niej logowania, garażu ani publikacji w sklepach. Nie wykonano też formalnego testu obciążenia przy ok. 10 000 rekordach w katalogu i ok. 50 RPS, który był zapisany w wymaganiach jakościowych.

Zamiast tego 25.10.2026 zrobiono pomiary Grafana k6 na Oracle VM [10]. Podczas testów dla listy wydarzeń uzyskano około 150–200 żądań na sekundę; przy 50 RPS nie było błędów (p95 ok. 4,5 ms). Dla health API i statycznego frontu uzyskano ok. 2000 żądań na sekundę. W porównaniu pomiarów ograniczenie dotyczy listy wydarzeń (API z MySQL), a nie nginx ani health API. Ponieważ k6 działał na tej samej maszynie co aplikacja, wynik traktujemy jako dolną granicę — to nadal nie jest formalny scenariusz 10k rekordów / 50 RPS.

Na potrzeby MVP i obrony taki obraz wydajności wystarcza. Na później zostają m.in. płatności online, rozszerzenie aplikacji mobilnej, formalny test 10k/50 RPS oraz dalsze utwardzenie środowiska produkcyjnego.

Bibliografia

[1] „React Documentation”. https://react.dev/. dostęp: 2026-11-07 (cyt. na ss. 24, 70, 110).

[2] „Vite Documentation”. https://vite.dev/guide/. dostęp: 2026-11-07 (cyt. na ss. 24, 70, 110).

[3] „TypeScript Documentation”. https://www.typescriptlang.org/docs/. dostęp: 2026-11-07 (cyt. na ss. 24, 70, 118).

[4] „Spring Boot Reference Documentation”. https://docs.spring.io/spring-boot/docs/current/reference/html/. dostęp: 2026-11-07 (cyt. na ss. 24, 110, 114).

[5] „Spring Security Reference”. https://docs.spring.io/spring-security/reference/. dostęp: 2026-11-07 (cyt. na ss. 24, 76, 97, 114).

[6] „MySQL 8.4 Reference Manual”. https://dev.mysql.com/doc/refman/8.4/en/. dostęp: 2026-11-07 (cyt. na ss. 24, 110).

[7] „Expo Documentation”. https://docs.expo.dev/. dostęp: 2026-11-07 (cyt. na s. 24).

[8] „RacePortal — repozytorium projektu”. https://github.com/Gucio-009/RacePortal. Branch main; dostęp: 2026-11-07 (cyt. na s. 24).

[9] „RacePortal — zakres MVP”. dokumentacja projektu (docs/MVP.md). stan 28.10.2026; dostęp: 2026-11-07 (cyt. na ss. 52, 109, 117).

[10] „Wyniki load testu k6 — Oracle VM (RPS)”. dokumentacja projektu RacePortal (docs/testy/wyniki/k6-vm-rps.md). sesja pomiarowa 2026-10-25; dostęp: 2026-11-07 (cyt. na ss. 56 i nast., 105, 108, 115, 122).

[11] „Cloudflare Tunnel”. https://developers.cloudflare.com/cloudflare-one/connections/connect-apps/. dostęp: 2026-11-07 (cyt. na ss. 68, 109, 115, 117).

[12] „nginx Documentation”. https://nginx.org/en/docs/. dostęp: 2026-11-07 (cyt. na s. 68).

[13] „Tailwind CSS Documentation”. https://tailwindcss.com/docs. dostęp: 2026-11-07 (cyt. na s. 70).

[14] „JSON Web Token (JWT)”. https://datatracker.ietf.org/doc/html/rfc7519. RFC 7519; dostęp: 2026-11-07 (cyt. na ss. 76, 114).

[15] „Docker Compose overview”. https://docs.docker.com/compose/. dostęp: 2026-11-07 (cyt. na ss. 77, 109, 115, 117).

[16] „Open Source Routing Machine”. http://project-osrm.org/. dostęp: 2026-11-07 (cyt. na ss. 89, 91, 109, 118).

[17] „Leaflet — an open-source JavaScript library for mobile-friendly interactive maps”. https://leafletjs.com/. dostęp: 2026-11-07 (cyt. na ss. 91, 109, 118).

[18] Niels Provos i David Mazières. „bcrypt — A Future-Adaptable Password Scheme”. https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf. USENIX 1999; dostęp: 2026-11-07 (cyt. na s. 98).

[19] „Jakarta Bean Validation”. https://beanvalidation.org/. dostęp: 2026-11-07 (cyt. na s. 100).

[20] „OWASP Top 10 Web Application Security Risks”. https://owasp.org/www-project-top-ten/. dostęp: 2026-11-07 (cyt. na s. 102).

[21] „OWASP Application Security Verification Standard”. https://owasp.org/www-project-application-security-verification-standard/. dostęp: 2026-11-07 (cyt. na s. 102).

[22] „RacePortal — rejestr testów penetracyjnych i remedacji”. dokumentacja projektu (docs/cybersecurity-pentest.md). dostęp: 2026-11-07 (cyt. na ss. 104, 114, 117 i nast.).

[23] „Playwright Documentation”. https://playwright.dev/docs/intro. dostęp: 2026-11-07 (cyt. na ss. 105 i nast., 112, 115).

[24] „Grafana k6 documentation”. https://grafana.com/docs/k6/latest/. dostęp: 2026-11-07 (cyt. na ss. 105, 108, 115, 120).

[25] „RacePortal — scenariusz prezentacji obrony”. dokumentacja projektu (docs/scenariusz-prezentac dostęp: 2026-11-07 (cyt. na ss. 109 i nast., 115).

[26] „LaTeX — A document preparation system”. https://www.latex-project.org/. dostęp: 2026-11-07 (cyt. na ss. 112, 115).

[27] „RacePortal — rejestr zgodności ze Specyfikacją”. dokumentacja projektu (docs/spec-conformity.m dostęp: 2026-11-07 (cyt. na s. 118).

Spis rysunków

4.1 Podział interesariuszy projektu — funkcjonalności dostępne dla ról użytkowników (materiał koncepcyjny). 38

7.1 Diagram przypadków użycia — Platforma motorsportowa (wersja koncepcyjna). 65

8.1 Diagram klas / model danych (ERD) — wersja koncepcyjna projektu. 69

8.2 Prototyp UI RacePortal w Figma Make — strona główna (ciemny motyw motorsportowy, akcent marki). 73

Spis tabel

4.1 Karta udziałowca: Gość / kibic 39
4.2 Karta udziałowca: Kierowca 39
4.3 Karta udziałowca: Organizator 40
4.4 Karta udziałowca: Administrator 40
4.5 Karta udziałowca: Właściciel toru / partner obiektu 41
4.6 Porównanie modelu grup społecznościowych z modelem RacePortal (ujęcie koncepcyjne). 42

6.1 Zakres MVP — funkcje z KP/DZW oraz stan według `docs/MVP.md` (28.10.2026). 53
6.2 Podział ról według DZW (formalny dla tego rozdziału) oraz opis w `README.md`. 54
6.3 Etapy prac — plan z Harmonogramu PRO wobec charakteru przebiegu w repozytorium. 55
6.4 Analiza ryzyka RacePortal — wybrane zagrożenia i sposoby ich ograniczenia. . 57

7.1 Zintegrowany katalog wydarzeń motorsportowych 59
7.2 Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów 59
7.3 Przejrzysty podział ról w systemie 59
7.4 Publiczny katalog i filtrowanie 60
7.5 Szczegóły wydarzenia i wyznaczanie trasy 60
7.6 Zarządzanie profilem i kontem 60
7.7 Wirtualny Garaż Kierowcy 60
7.8 Szybkie aplikowanie na wydarzenia 61
7.9 Skuteczny system powiadomień 61
7.10 Panel Zarządzania dla Organizatora 61
7.11 Panel Administratora i wsparcie użytkowników 61
7.12 Mobilna aplikacja wizytówkowa 62
7.13 Wydajność i responsywność platformy 62
7.14 Dostępność wieloplatformowa (RWD) 62
7.15 Niezawodność i łatwość wdrażania 62
7.16 Bezpieczeństwo danych osobowych i kont 63
7.17 Ścisła kontrola dostępu do zasobów 63

7.18 Odporność na nadużycia platformy 63
7.19 Nowoczesny design branżowy 64
7.20 Intuicyjność i czytelność procesów 64

14.1 Skrót wyników k6 na Oracle VM (25.10.2026) — komfort vs sufit. 108

Załączniki

Płyta CD z następującą zawartością:

- *pliki projektowe* – pliki składające się na całość projektu
  - repozytorium kodu źródłowego wraz z instrukcją zbudowania i uruchomienia projektu
  - źródło pracy inżynierskiej.
- *Otrombke Natalia_Kasprowicz Oliwier_Gutowski Michał_Parkitny Miłosz_Wronisz Wojciech_praca pisemna* – katalog zawierający plik PDF z pracą inżynierską.